

STAR RX72N - rx_bmp280 Library
1.0.0

Generated by Doxygen 1.16.1

1 Test List	1
2 Directory Hierarchy	3
2.1 Directories	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 libs/rx_bmp280/inc Directory Reference	7
4.2 libs Directory Reference	7
4.3 libs/rx_bmp280 Directory Reference	8
4.4 libs/rx_bmp280/src Directory Reference	8
5 File Documentation	9
5.1 libs/rx_bmp280/inc/rx_bmp280.h File Reference	9
5.1.1 Detailed Description	10
5.1.2 Overview	10
5.1.3 Key Specifications	10
5.1.4 Hardware Configuration (STAR Project)	11
5.1.5 Operating Mode	11
5.1.6 Integer Compensation Formulas	11
5.1.7 Data Structure Documentation	12
5.1.7.1 struct bmp280_calib_t	12
5.1.7.2 struct bmp280_data_t	13
5.1.8 Enumeration Type Documentation	14
5.1.8.1 bmp280_temp_limits_t	14
5.1.9 Function Documentation	15
5.1.9.1 rx_bmp280_init()	15
5.1.9.2 rx_bmp280_read()	17
5.2 rx_bmp280.h	20
5.3 libs/rx_bmp280/inc/rx_bmp280_regs.h File Reference	24
5.3.1 Detailed Description	26
5.3.2 Overview	26
5.3.3 Register Map Summary	26
5.3.4 DFRobot Module Hardware	26
5.3.5 Enumeration Type Documentation	27
5.3.5.1 bmp280_adc_buf_t	27
5.3.5.2 bmp280_adc_idx_t	28
5.3.5.3 bmp280_adc_shift_t	28
5.3.5.4 bmp280_calib_idx_t	29
5.3.5.5 bmp280_calib_reg_count_t	30
5.3.5.6 bmp280_calib_shift_t	31
5.3.5.7 bmp280_chip_id_t	31
5.3.5.8 bmp280_config_t	32

5.3.5.9 bmp280_ctrl_meas_t	33
5.3.5.10 bmp280_i2c_addr_t	33
5.3.5.11 bmp280_poll_t	34
5.3.5.12 bmp280_reg_t	35
5.3.5.13 bmp280_status_mask_t	36
5.3.5.14 bmp280_write_idx_t	37
5.3.5.15 bmp280_write_size_t	37
5.4 rx_bmp280_regs.h	38
5.5 libs/rx_bmp280/src/rx_bmp280.c File Reference	43
5.5.1 Detailed Description	45
5.5.2 Overview	45
5.5.3 Integer Compensation Algorithm	45
5.5.4 NASA Power of 10 Compliance	46
5.5.5 Enumeration Type Documentation	47
5.5.5.1 bmp280_adc20_idx_t	47
5.5.5.2 bmp280_adc20_range_t	47
5.5.5.3 bmp280_le16_idx_t	48
5.5.5.4 bmp280_press_range_pa_t	48
5.5.5.5 bmp280_press_range_t	49
5.5.5.6 bmp280_sizeof_t	50
5.5.5.7 bmp280_temp_comp_t	50
5.5.5.8 bmp280_temp_range_t	51
5.5.5.9 press_comp_constants_t	52
5.5.6 Function Documentation	52
5.5.6.1 internal_assemble_adc20()	52
5.5.6.2 internal_compensate_pressure()	54
5.5.6.3 internal_compensate_temp()	55
5.5.6.4 internal_finalize_pressure_q8()	57
5.5.6.5 internal_parse_calibration()	58
5.5.6.6 internal_parse_s16_le()	59
5.5.6.7 internal_parse_u16_le()	60
5.5.6.8 internal_read_and_compensate_adc()	61
5.5.6.9 internal_read_and_validate_calibration()	63
5.5.6.10 internal_read_regs()	64
5.5.6.11 internal_verify_chip_id()	65
5.5.6.12 internal_write_reg()	66
5.5.6.13 rx_bmp280_init()	67
5.5.6.14 rx_bmp280_read()	69
5.5.7 Variable Documentation	70
5.5.7.1 s_bus_name	70
5.5.7.2 s_calib	71
5.5.7.3 s_initialized	71
5.5.7.4 s_manager	72

5.5.7.5 s_tag	72
5.6 rx_bmp280.c	73

Chapter 1

Test List

Global `rx`bmp280`init` (`rx`bus`manager`*manager`)

tests/test_rx_bmp280.c: test_bmp280_init_success, test_bmp280_init_null_manager_returns↵
_error, test_bmp280_init_i2c_error_propagates, test_bmp280_init_invalid_calib_returns_error,
test_bmp280_init_reinit_succeeds

Global `rx`bmp280`read` (`bmp280`data`*out`)

tests/test_rx_bmp280.c: test_bmp280_read_success_forced_mode, test_bmp280_read_status↵
_timeout, test_bmp280_read_i2c_error_propagates, test_bmp280_compensation_known_values,
test_bmp280_read_zero_var1_returns_error

Chapter 2

Directory Hierarchy

2.1 Directories

inc	7
rx_bmp280.h	9
rx_bmp280_regs.h	24
libs	7
rx_bmp280	8
inc	7
rx_bmp280.h	9
rx_bmp280_regs.h	24
src	8
rx_bmp280.c	43
rx_bmp280	8
inc	7
rx_bmp280.h	9
rx_bmp280_regs.h	24
src	8
rx_bmp280.c	43
src	8
rx_bmp280.c	43

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

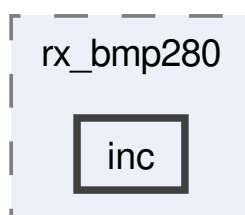
libs/rx_bmp280/inc/ rx_bmp280.h	
BMP280 Digital Barometric Pressure and Temperature Sensor Driver API	9
libs/rx_bmp280/inc/ rx_bmp280_regs.h	
BMP280 Barometric Pressure and Temperature Sensor Register Map	24
libs/rx_bmp280/src/ rx_bmp280.c	
BMP280 Digital Barometric Pressure and Temperature Sensor Driver Implementation	43

Chapter 4

Directory Documentation

4.1 libs/rx_bmp280/inc Directory Reference

Directory dependency graph for inc:

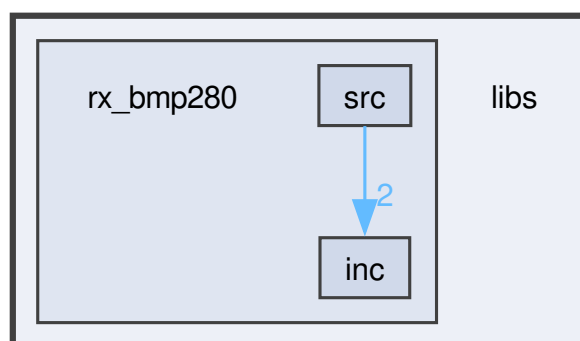


Files

- file [rx_bmp280.h](#)
BMP280 Digital Barometric Pressure and Temperature Sensor Driver API.
- file [rx_bmp280_regs.h](#)
BMP280 Barometric Pressure and Temperature Sensor Register Map.

4.2 libs Directory Reference

Directory dependency graph for libs:

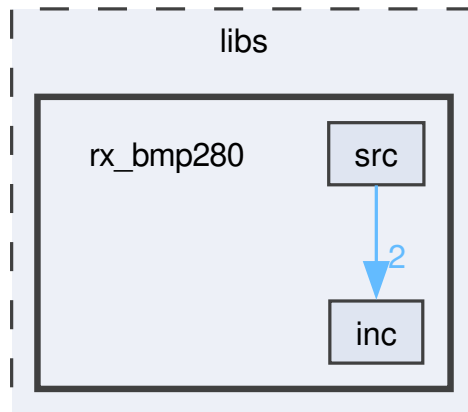


Directories

- directory [rx_bmp280](#)

4.3 libs/rx_bmp280 Directory Reference

Directory dependency graph for rx_bmp280:

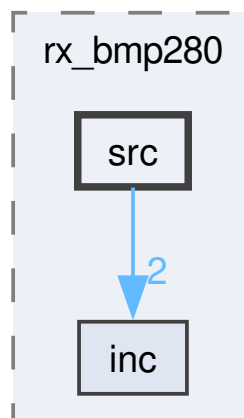


Directories

- directory [inc](#)
- directory [src](#)

4.4 libs/rx_bmp280/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_bmp280.c](#)
BMP280 Digital Barometric Pressure and Temperature Sensor Driver Implementation.

Chapter 5

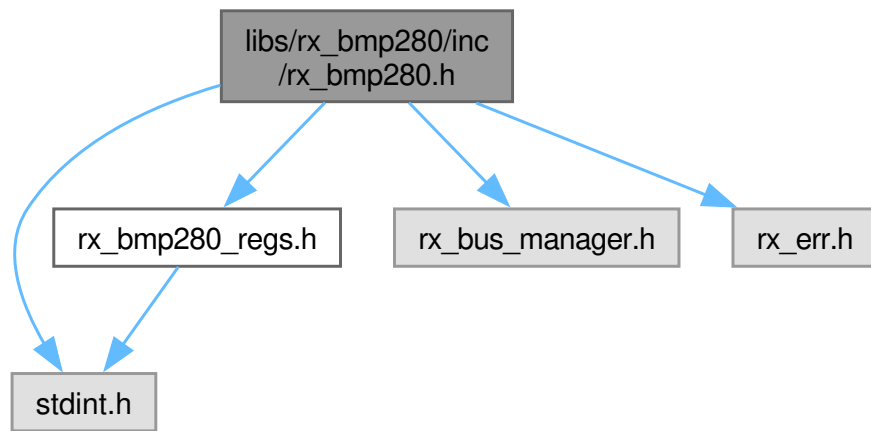
File Documentation

5.1 libs/rx_bmp280/inc/rx_bmp280.h File Reference

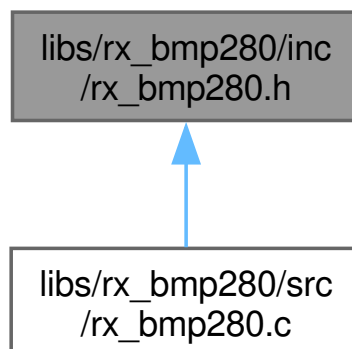
BMP280 Digital Barometric Pressure and Temperature Sensor Driver API.

```
#include <stdint.h>
#include "rx_bmp280_regs.h"
#include "rx_bus_manager.h"
#include "rx_err.h"
```

Include dependency graph for rx_bmp280.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `bmp280_calib_t`
BMP280 factory calibration coefficients.
- struct `bmp280_data_t`
BMP280 compensated measurement output.

Enumerations

- enum `bmp280_temp_limits_t`: `int16_t { k_bmp280_temp_min_cdeg = -4000, k_bmp280_temp_max_cdeg = 8500 }`
BMP280 valid temperature output range limits in centi-degrees Celsius.

Functions

- `rx_err_t rx_bmp280_init (rx_bus_manager_t *manager)`
Initialize BMP280 sensor and read factory calibration coefficients.
- `rx_err_t rx_bmp280_read (bmp280_data_t *out)`
Trigger a forced measurement and read compensated pressure and temperature.

5.1.1 Detailed Description

BMP280 Digital Barometric Pressure and Temperature Sensor Driver API.

5.1.2 Overview

Platform-independent driver for the Bosch BMP280 digital barometric pressure sensor. The BMP280 measures atmospheric pressure and ambient temperature using a MEMS capacitive pressure element with integrated temperature compensation.

This driver uses forced mode operation: each measurement is explicitly triggered, the sensor performs one conversion, and returns to sleep. This minimizes self-heating and provides one-shot measurement semantics suitable for periodic IMU task reads.

5.1.3 Key Specifications

Parameter	Value
Pressure range	300-1100 hPa
Pressure resolution	0.0016 hPa
Temperature range	-40 to +85 degC
Temperature resolution	0.01 degC
I2C speed	Up to 400 kHz (fast mode)

5.1.4 Hardware Configuration (STAR Project)

- Module: DFRobot BMP280 breakout
- I2C Address: 0x76 (SDO pulled LOW)
- I2C Bus: RIIC1 (SCL1=P2.1, SDA1=P2.0), bus name "i2c1_baro"

5.1.5 Operating Mode

The driver uses forced mode (trigger -> measure -> sleep) with:

- Temperature oversampling: 2x
- Pressure oversampling: 16x
- IIR filter: coefficient 2

5.1.6 Integer Compensation Formulas

The BMP280 uses factory-calibrated trimming parameters to compensate raw ADC output. The driver implements the Bosch-provided integer-only compensation formulas (no floating-point math in driver).

Output values:

- temp_centi_degC: temperature * 100 (e.g. 2523 = 25.23 degC)
- press_pa_256: pressure * 256 (e.g. 26624000 = 104000 Pa = 1040 hPa)

See also

[rx_bmp280_regs.h](#) Register map and bit-field constants

[rx_bmp280.c](#) Implementation with Bosch compensation formulas

BMP280 datasheet v1.19 section 4.2.3 for compensation algorithms

NASA Power of 10 Compliance:

- Rule 1 (No goto): [PASS] Structured control flow only
- Rule 3 (No heap): [PASS] Zero dynamic allocation
- Rule 5 (Assertions): [PASS] 2+ preconditions per function
- Rule 7 (Return checks): [PASS] All I2C returns validated
- Rule 8 (Preprocessor): [PASS] C23 typed enums only

Author

STAR Team

Date

2026-03-04

Version

1.0.0

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_bmp280.h](#).

5.1.7 Data Structure Documentation

5.1.7.1 struct bmp280_calib_t

BMP280 factory calibration coefficients.

Trimming parameters read from on-chip OTP during initialization. Used by the Bosch integer compensation algorithm to convert raw ADC values to calibrated temperature and pressure.

All coefficients are little-endian in the register map and stored as parsed C types here. `dig_T1` and `dig_P1` are unsigned; all others are signed two's complement.

Invariant

All values loaded once from BMP280 OTP during init
`dig_T1`, `dig_P1` are positive (unsigned)

See also

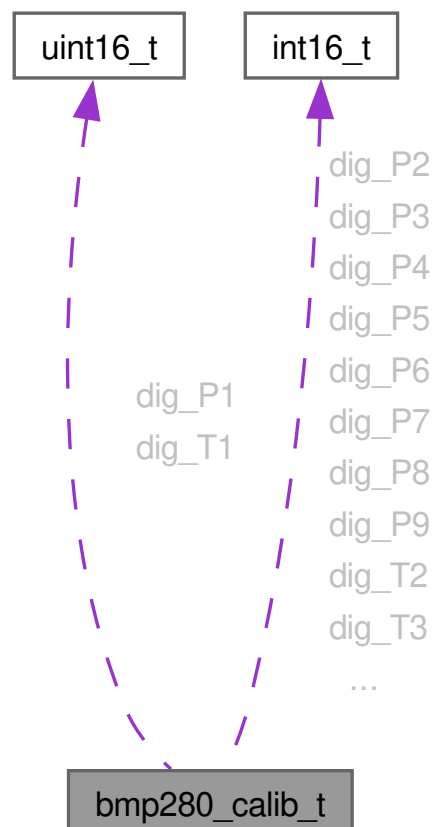
BMP280 datasheet section 4.2.2 for coefficient descriptions
[rx_bmp280_init\(\)](#) Reads and parses these coefficients

Since

Version 1.0.0

Definition at line 136 of file [rx_bmp280.h](#).

Collaboration diagram for `bmp280_calib_t`:



Data Fields

uint16_t	dig_P1	Pressure compensation coefficient P1 (unsigned)
int16_t	dig_P2	Pressure compensation coefficient P2 (signed)
int16_t	dig_P3	Pressure compensation coefficient P3 (signed)
int16_t	dig_P4	Pressure compensation coefficient P4 (signed)
int16_t	dig_P5	Pressure compensation coefficient P5 (signed)
int16_t	dig_P6	Pressure compensation coefficient P6 (signed)
int16_t	dig_P7	Pressure compensation coefficient P7 (signed)
int16_t	dig_P8	Pressure compensation coefficient P8 (signed)
int16_t	dig_P9	Pressure compensation coefficient P9 (signed)
uint16_t	dig_T1	Temperature compensation coefficient T1 (unsigned)
int16_t	dig_T2	Temperature compensation coefficient T2 (signed)
int16_t	dig_T3	Temperature compensation coefficient T3 (signed)

5.1.7.2 struct bmp280`data`

BMP280 compensated measurement output.

Integer-scaled output values for temperature and pressure. Avoids floating-point in the driver; convert to SI units in application:

```
float temp_celsius = (float)data.temp_centigrade / 100.0F;
float pressure_pa = (float)data.press_pa_256 / 256.0F;
float pressure_hpa = pressure_pa / 100.0F;
```

Invariant

temp_centigrade in [-4000, 8500] (operating range -40 to +85 degC)
 press_pa_256 > 0 (absolute pressure always positive)

See also

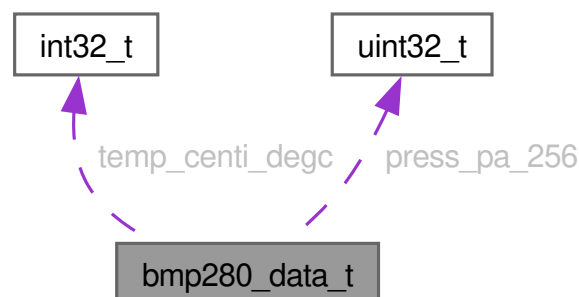
[rx_bmp280_read\(\)](#) Populates this structure

Since

Version 1.0.0

Definition at line 172 of file [rx_bmp280.h](#).

Collaboration diagram for bmp280_data_t:



Data Fields

uint32_t	press_pa_256	Pressure * 256 in Pa (divide by 256.0 for Pa)
int32_t	temp_centi_degc	Temperature * 100 (e.g. 2523 = 25.23 degC)

5.1.8 Enumeration Type Documentation

5.1.8.1 bmp280`temp`limits`t

enum [bmp280_temp_limits_t](#) : int16_t

BMP280 valid temperature output range limits in centi-degrees Celsius.

Authoritative temperature operating range constants for the BMP280 sensor as documented in the Bosch BMP280 datasheet. Values are in units of 0.01 degC (centi-degrees) matching the temp_centi_degc field in [bmp280_data_t](#).

Limit	Value	Physical
k_bmp280_temp_min_cdegc	-4000	-40.00 degC
k_bmp280_temp_max_cdegc	8500	+85.00 degC

See also

[bmp280_data_t](#) Uses temp_centi_degc validated against these limits

[rx_bmp280_read\(\)](#) Applies these range checks after compensation

Since

Version 1.0.0

Enumerator

k_bmp280_temp_min_cdegc	Minimum valid temperature: -40.00 degC
k_bmp280_temp_max_cdegc	Maximum valid temperature: +85.00 degC

Definition at line 105 of file [rx_bmp280.h](#).

```
00105         : int16_t {
00106     k_bmp280_temp_min_cdegc = -4000, /**< Minimum valid temperature: -40.00 degC */
00107     k_bmp280_temp_max_cdegc = 8500,  /**< Maximum valid temperature: +85.00 degC */
00108 } bmp280_temp_limits_t;
```

5.1.9 Function Documentation

5.1.9.1 rx`bmp280`init()

```
rx_err_t rx_bmp280_init (
    rx_bus_manager_t * manager) [nodiscard]
```

Initialize BMP280 sensor and read factory calibration coefficients.

Reads the 24-byte calibration block from OTP (k_bmp280_reg_calib_start (0x88) through k_bmp280_reg_calib_end (0x9F)) and parses the 12 trimming parameters into module-static storage. Then writes the IIR filter configuration to register k_bmp280_reg_config (0xF5).

The ctrl_meas register (0xF4) is NOT written during init; it is written at the start of each [rx_bmp280_read\(\)](#) call to trigger a forced measurement. The bus manager must have "i2c1_baro" registered as the named I2C bus.

Parameters

in	manager	Pointer to initialized bus manager with "i2c1_baro" registered. Must not be NULL.
----	---------	---

Returns

rx_err_t Initialization result

Return values

k_rx_ok	Calibration read, sensor ready for reads
k_rx_err_null_ptr	manager is NULL
k_rx_err_nack	I2C NACK (device not found, check address/wiring)
k_rx_err_timeout	I2C timeout

Precondition

manager non-NULL, "i2c1_baro" bus registered and initialized
BMP280 powered (3.3V) on same I2C bus as BNO055

Postcondition

Calibration coefficients loaded into module-static storage
Driver initialized successfully (s_initialized == true on success)
IIR filter configured with coefficient 2

Note

Not thread-safe; call during single-threaded initialization
Blocks ~2 ms (I2C calibration burst read)

Test tests/test_rx_bmp280.c: test_bmp280_init_success, test_bmp280_init_null_manager ↵
returns_error, test_bmp280_init_i2c_error_propagates, test_bmp280_init_invalid_calib ↵
returns_error, test_bmp280_init_reinit_succeeds

See also

[rx_bmp280_read\(\)](#) Triggered forced measurements after init

Since

Version 1.0.0

Verifies chip ID, reads and validates 24 bytes of factory-calibrated trimming parameters from OTP registers, and configures the IIR filter by writing to the config register 0xF5.

Precondition

manager non-NULL, "i2c1_baro" registered and initialized
BMP280 powered on RIIC1 bus

Postcondition

s_calib contains valid OTP coefficients
s_initialized == true on success

Note

Not thread-safe

See also

[rx_bmp280_read\(\)](#) For triggered measurements

Since

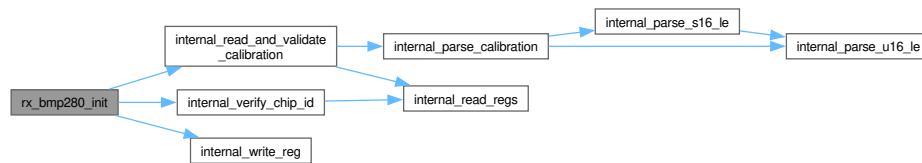
Version 1.0.0

Definition at line 815 of file [rx_bmp280.c](#).

```
00816 {
00817     RX_CHECK_NULL_PTR(manager, s_tag, "Bus manager is NULL");
00818     RX_ASSERT_PRE(s_bus_name != NULL, "Bus name must not be NULL");
00819     s_manager = manager;
00820     s_initialized = false;
00821
00822     rx_err_t err = internal_verify_chip_id();
00823     if (err != k_rx_ok) {
00824         return err;
00825     }
00826
00827     err = internal_read_and_validate_calibration();
00828     if (err != k_rx_ok) {
00829         return err;
00830     }
00831
00832     err = internal_write_reg(k_bmp280_reg_config, k_bmp280_config_val);
00833     if (err != k_rx_ok) {
00834         rx_log_error(s_tag, "Config register write failed");
00835         s_manager = NULL;
00836         return err;
00837     }
00838
00839     s_initialized = true;
00840     rx_log_info(s_tag, "BMP280 initialized, calibration loaded");
00841
00842     return k_rx_ok;
00843 }
00844 }
```

References [internal_read_and_validate_calibration\(\)](#), [internal_verify_chip_id\(\)](#), [internal_write_reg\(\)](#), [k_bmp280_config_val](#), [k_bmp280_reg_config](#), [s_bus_name](#), [s_initialized](#), [s_manager](#), and [s_tag](#).

Here is the call graph for this function:



5.1.9.2 rx`bmp280`read()

```

rx_err_t rx_bmp280_read (
    bmp280_data_t * out) [nodiscard]

```

Trigger a forced measurement and read compensated pressure and temperature.

Measurement sequence:

1. Write ctrl_meas = k_bmp280_ctrl_meas_val (0x55) to k_bmp280_reg_ctrl_meas (0xF4) (osrs←_t=2x, osrs_p=16x, forced mode)
2. Poll status register k_bmp280_reg_status (0xF3) until measuring bit (k_bmp280_status_meas←_mask, bit 3) clears
 - Bounded by k_bmp280_poll_max iterations
3. Read 6 bytes from k_bmp280_reg_press_msb (0xF7): pressure MSB/LSB/XLSB + temperature MSB/LSB/XLSB
4. Assemble 20-bit ADC values: adc_P = (buf[k_bmp280_press_msb_idx] << k_bmp280_shift_←_msb) | (buf[k_bmp280_press_lsb_idx] << k_bmp280_shift_lsb_left) | (buf[k_bmp280_press←_xlsb_idx] >> k_bmp280_shift_xlsb_right) adc_T = (buf[k_bmp280_temp_msb_idx] << k←_bmp280_shift_msb) | (buf[k_bmp280_temp_lsb_idx] << k_bmp280_shift_lsb_left) | (buf[k←_bmp280_temp_xlsb_idx] >> k_bmp280_shift_xlsb_right)
5. Apply Bosch integer compensation algorithm (BMP280 datasheet section 4.2.3)

Parameters

out	out	Output data structure. Must not be NULL.
-----	-----	--

Returns

rx_err_t Read result

Return values

k_rx_ok	Measurement complete, out populated
k_rx_err_null_ptr	out is NULL
k_rx_err_not_initialized	init not called
k_rx_err_nack	I2C communication failure
k_rx_err_timeout	Status poll exceeded k_bmp280_poll_max iterations

Precondition

[rx_bmp280_init\(\)](#) succeeded
out non-NULL

Postcondition

out->temp_centi_degc contains calibrated temperature * 100
out->press_pa_256 contains calibrated pressure * 256
Sensor returns to sleep mode (forced measurement complete)

Note

Not thread-safe; call from single task
Measurement takes ~40 ms with 16x pressure oversampling

Warning

Returns `k_rx_err_timeout` if status bit does not clear within `k_bmp280_poll_max` polls (indicates hardware fault)

Test tests/test_rx_bmp280.c: `test_bmp280_read_success_forced_mode`, `test_bmp280_read_status_timeout`, `test_bmp280_read_i2c_error_propagates`, `test_bmp280_compensation_known_values`, `test_bmp280_read_zero_var1_returns_error`

See also

[rx_bmp280_init\(\)](#) Must be called successfully before [rx_bmp280_read\(\)](#)

Since

Version 1.0.0

Trigger a forced measurement and read compensated pressure and temperature.

Executes the complete forced-mode measurement cycle:

1. Trigger forced measurement (write `ctrl_meas`)
2. Poll status register for completion (bounded loop)
3. Read ADC data, apply compensation, validate range (`internal_read_and_compensate_adc`)

Precondition

[rx_bmp280_init\(\)](#) succeeded
out non-NULL

Postcondition

out populated with compensated measurement on k_rx_ok
 Sensor in sleep mode (forced measurement complete)

Note

Not thread-safe

See also

[internal_read_and_compensate_adc\(\)](#) ADC read, compensation, and range validation

Since

Version 1.0.0

Definition at line 947 of file [rx_bmp280.c](#).

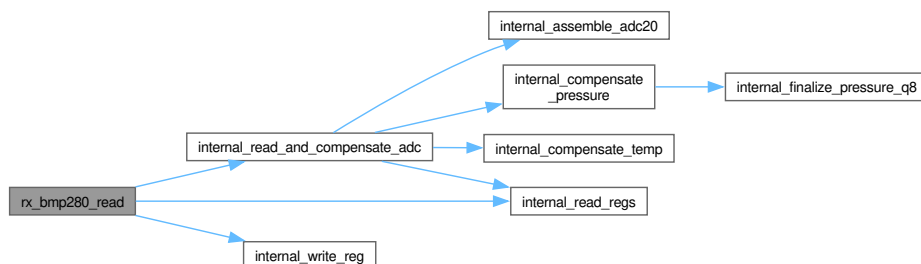
```

00948 {
00949     RX_CHECK_NULL_PTR(out, s_tag, "Output data pointer is NULL");
00950
00951     if (!s_initialized) {
00952         return k_rx_err_not_initialized;
00953     }
00954
00955     /* Step 1: Write ctrl_meas to trigger forced measurement */
00956     rx_err_t err = internal_write_reg(k_bmp280_reg_ctrl_meas, k_bmp280_ctrl_meas_val);
00957     if (err != k_rx_ok) {
00958         rx_log_error(s_tag, "ctrl_meas write failed");
00959         return err;
00960     }
00961
00962     /* Step 2: Poll status register until measuring bit (bit 3) clears */
00963     uint32_t poll_count = 0;
00964     while (poll_count < (uint32_t)k_bmp280_poll_max) {
00965         uint8_t status_byte = 0;
00966         err = internal_read_regs(k_bmp280_reg_status, &status_byte, k_bmp280_single_byte);
00967         if (err != k_rx_ok) {
00968             rx_log_error(s_tag, "Status register read failed");
00969             return err;
00970         }
00971
00972         if ((status_byte & k_bmp280_status_meas_mask) == 0U) {
00973             break; /* Measurement complete */
00974         }
00975
00976         (void)tx_thread_sleep(k_bmp280_poll_sleep_ticks); /* Yield CPU while sensor is measuring */
00977         poll_count++;
00978     }
00979
00980     if (poll_count >= (uint32_t)k_bmp280_poll_max) {
00981         rx_log_error(s_tag, "Status poll timeout");
00982         return k_rx_err_timeout;
00983     }
00984
00985     /* Step 3: Read ADC, apply compensation, validate output range */
00986     return internal_read_and_compensate_adc(out);
00987 }

```

References [internal_read_and_compensate_adc\(\)](#), [internal_read_regs\(\)](#), [internal_write_reg\(\)](#), [k_bmp280_ctrl_meas_val](#), [k_bmp280_poll_max](#), [k_bmp280_poll_sleep_ticks](#), [k_bmp280_reg_ctrl_meas](#), [k_bmp280_reg_status](#), [k_bmp280_single_byte](#), [k_bmp280_status_meas_mask](#), [s_initialized](#), and [s_tag](#).

Here is the call graph for this function:



5.2 rx_bmp280.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_bmp280.h
00003  * @brief BMP280 Digital Barometric Pressure and Temperature Sensor Driver API
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * Platform-independent driver for the Bosch BMP280 digital barometric
00009  * pressure sensor. The BMP280 measures atmospheric pressure and ambient
00010  * temperature using a MEMS capacitive pressure element with integrated
00011  * temperature compensation.
00012  *
00013  * This driver uses forced mode operation: each measurement is explicitly
00014  * triggered, the sensor performs one conversion, and returns to sleep.
00015  * This minimizes self-heating and provides one-shot measurement semantics
00016  * suitable for periodic IMU task reads.
00017  *
00018  * # Key Specifications
00019  *
00020  * | Parameter | Value |
00021  * |-----|-----|
00022  * | Pressure range | 300-1100 hPa |
00023  * | Pressure resolution | 0.0016 hPa |
00024  * | Temperature range | -40 to +85 degC |
00025  * | Temperature resolution | 0.01 degC |
00026  * | I2C speed | Up to 400 kHz (fast mode) |
00027  *
00028  * # Hardware Configuration (STAR Project)
00029  *
00030  * - **Module**: DFRobot BMP280 breakout
00031  * - **I2C Address**: 0x76 (SDO pulled LOW)
00032  * - **I2C Bus**: RIIC1 (SCL1=P2.1, SDA1=P2.0), bus name "i2c1_baro"
00033  *
00034  * # Operating Mode
00035  *
00036  * The driver uses forced mode (trigger -> measure -> sleep) with:
00037  * - Temperature oversampling: 2x
00038  * - Pressure oversampling: 16x
00039  * - IIR filter: coefficient 2
00040  *
00041  * # Integer Compensation Formulas
00042  *
00043  * The BMP280 uses factory-calibrated trimming parameters to compensate
00044  * raw ADC output. The driver implements the Bosch-provided integer-only
00045  * compensation formulas (no floating-point math in driver).
00046  *
00047  * Output values:
00048  * - temp_centi_degC: temperature * 100 (e.g. 2523 = 25.23 degC)
00049  * - press_pa_256: pressure * 256 (e.g. 26624000 = 104000 Pa = 1040 hPa)
00050  *
00051  * @see rx_bmp280_regs.h Register map and bit-field constants
00052  * @see rx_bmp280.c Implementation with Bosch compensation formulas
00053  * @see BMP280 datasheet v1.19 section 4.2.3 for compensation algorithms
00054  *
00055  * @par NASA Power of 10 Compliance:
00056  * - **Rule 1** (No goto): [PASS] Structured control flow only
00057  * - **Rule 3** (No heap): [PASS] Zero dynamic allocation
00058  * - **Rule 5** (Assertions): [PASS] 2+ preconditions per function
00059  * - **Rule 7** (Return checks): [PASS] All I2C returns validated
00060  * - **Rule 8** (Preprocessor): [PASS] C23 typed enums only
00061  *
00062  * @author STAR Team
00063  * @date 2026-03-04
00064  * @version 1.0.0
00065  * @copyright Copyright (c) 2026 Locked Inc.
00066  * SPDX-License-Identifier: MIT
00067  */
00068
00069 #pragma once
00070
00071 #ifdef __cplusplus
00072 extern "C" {
00073 #endif
00074
00075 #include <stdint.h>
00076
00077 #include "rx_bmp280_regs.h"
00078 #include "rx_bus_manager.h"
00079 #include "rx_err.h"
00080
00081 */

```

=====

```

00082 * Public Constants
00083 *
=====
00084 */
00085
00086 /**
00087  * @enum bmp280_temp_limits_t
00088  * @brief BMP280 valid temperature output range limits in centi-degrees Celsius
00089  *
00090  * @details
00091  * Authoritative temperature operating range constants for the BMP280 sensor
00092  * as documented in the Bosch BMP280 datasheet. Values are in units of
00093  * 0.01 degC (centi-degrees) matching the temp_centi_degC field in bmp280_data_t.
00094  *
00095  * | Limit | Value | Physical |
00096  * |-----|-----|-----|
00097  * | k_bmp280_temp_min_cdegC | -4000 | -40.00 degC |
00098  * | k_bmp280_temp_max_cdegC | 8500 | +85.00 degC |
00099  *
00100  * @see bmp280_data_t Uses temp_centi_degC validated against these limits
00101  * @see rx_bmp280_read() Applies these range checks after compensation
00102  *
00103  * @since Version 1.0.0
00104  */
00105 typedef enum : int16_t {
00106     k_bmp280_temp_min_cdegC = -4000, /**< Minimum valid temperature: -40.00 degC */
00107     k_bmp280_temp_max_cdegC = 8500,  /**< Maximum valid temperature: +85.00 degC */
00108 } bmp280_temp_limits_t;
00109
00110 /*
=====
00111 * Data Types
00112 *
=====
00113 */
00114
00115 /**
00116  * @struct bmp280_calib_t
00117  * @brief BMP280 factory calibration coefficients
00118  *
00119  * @details
00120  * Trimming parameters read from on-chip OTP during initialization.
00121  * Used by the Bosch integer compensation algorithm to convert raw
00122  * ADC values to calibrated temperature and pressure.
00123  *
00124  * All coefficients are little-endian in the register map and stored
00125  * as parsed C types here. dig_T1 and dig_P1 are unsigned; all others
00126  * are signed two's complement.
00127  *
00128  * @invariant All values loaded once from BMP280 OTP during init
00129  * @invariant dig_T1, dig_P1 are positive (unsigned)
00130  *
00131  * @see BMP280 datasheet section 4.2.2 for coefficient descriptions
00132  * @see rx_bmp280_init() Reads and parses these coefficients
00133  *
00134  * @since Version 1.0.0
00135  */
00136 typedef struct {
00137     uint16_t dig_T1; /**< Temperature compensation coefficient T1 (unsigned) */
00138     int16_t  dig_T2; /**< Temperature compensation coefficient T2 (signed) */
00139     int16_t  dig_T3; /**< Temperature compensation coefficient T3 (signed) */
00140     uint16_t dig_P1; /**< Pressure compensation coefficient P1 (unsigned) */
00141     int16_t  dig_P2; /**< Pressure compensation coefficient P2 (signed) */
00142     int16_t  dig_P3; /**< Pressure compensation coefficient P3 (signed) */
00143     int16_t  dig_P4; /**< Pressure compensation coefficient P4 (signed) */
00144     int16_t  dig_P5; /**< Pressure compensation coefficient P5 (signed) */
00145     int16_t  dig_P6; /**< Pressure compensation coefficient P6 (signed) */
00146     int16_t  dig_P7; /**< Pressure compensation coefficient P7 (signed) */
00147     int16_t  dig_P8; /**< Pressure compensation coefficient P8 (signed) */
00148     int16_t  dig_P9; /**< Pressure compensation coefficient P9 (signed) */
00149 } bmp280_calib_t;
00150
00151 /**
00152  * @struct bmp280_data_t
00153  * @brief BMP280 compensated measurement output
00154  *
00155  * @details
00156  * Integer-scaled output values for temperature and pressure.
00157  * Avoids floating-point in the driver; convert to SI units in application:
00158  *
00159  * @code
00160  * float temp_celsius = (float)data.temp_centi_degC / 100.0F;
00161  * float pressure_pa   = (float)data.press_pa_256 / 256.0F;
00162  * float pressure_hpa  = pressure_pa / 100.0F;
00163  * @endcode
00164  *
00165  * @invariant temp_centi_degC in [-4000, 8500] (operating range -40 to +85 degC)

```

```

00166 * @invariant press_pa_256 > 0 (absolute pressure always positive)
00167 *
00168 * @see rx_bmp280_read() Populates this structure
00169 *
00170 * @since Version 1.0.0
00171 */
00172 typedef struct {
00173     int32_t temp_centigrade; /**< Temperature * 100 (e.g. 2523 = 25.23 degC) */
00174     uint32_t press_pa_256; /**< Pressure * 256 in Pa (divide by 256.0 for Pa) */
00175 } bmp280_data_t;
00176
00177 /*
=====
00178 * Public API
00179 *
=====
00180 */
00181
00182 /**
00183 * @brief Initialize BMP280 sensor and read factory calibration coefficients
00184 *
00185 * @details
00186 * Reads the 24-byte calibration block from OTP (k_bmp280_reg_calib_start (0x88) through k_bmp280_reg_calib_end
00187 * (0x9F)) and parses
00188 * the 12 trimming parameters into module-static storage.
00189 * Then writes the IIR filter configuration to register k_bmp280_reg_config (0xF5).
00190 *
00191 * The ctrl_meas register (0xF4) is NOT written during init; it is written
00192 * at the start of each rx_bmp280_read() call to trigger a forced measurement.
00193 *
00194 * The bus manager must have "i2c1_baro" registered as the named I2C bus.
00195 *
00196 * @param[in] manager Pointer to initialized bus manager with "i2c1_baro" registered.
00197 * Must not be NULL.
00198 *
00199 * @return rx_err_t Initialization result
00200 * @retval k_rx_ok Calibration read, sensor ready for reads
00201 * @retval k_rx_err_null_ptr manager is NULL
00202 * @retval k_rx_err_nack I2C NACK (device not found, check address/wiring)
00203 * @retval k_rx_err_timeout I2C timeout
00204 *
00205 * @pre manager non-NULL, "i2c1_baro" bus registered and initialized
00206 * @pre BMP280 powered (3.3V) on same I2C bus as BNO055
00207 *
00208 * @post Calibration coefficients loaded into module-static storage
00209 * @post Driver initialized successfully (s_initialized == true on success)
00210 * @post IIR filter configured with coefficient 2
00211 *
00212 * @note Not thread-safe; call during single-threaded initialization
00213 * @note Blocks ~2 ms (I2C calibration burst read)
00214 *
00215 * @test tests/test_rx_bmp280.c: test_bmp280_init_success, test_bmp280_init_null_manager_returns_error,
00216 * test_bmp280_init_i2c_error_propagates, test_bmp280_init_invalid_calib_returns_error,
00217 * test_bmp280_init_reinit_succeeds
00218 *
00219 * @see rx_bmp280_read() Triggered forced measurements after init
00220 *
00221 * @since Version 1.0.0
00222 */
00223 [[nodiscard]] rx_err_t rx_bmp280_init(rx_bus_manager_t* manager);
00224
00225 /**
00226 * @brief Trigger a forced measurement and read compensated pressure and temperature
00227 *
00228 * @details
00229 * Measurement sequence:
00230 * 1. Write ctrl_meas = k_bmp280_ctrl_meas_val (0x55) to k_bmp280_reg_ctrl_meas (0xF4) (osrs_t=2x, osrs_p=16x,
00231 * forced mode)
00232 * 2. Poll status register k_bmp280_reg_status (0xF3) until measuring bit (k_bmp280_status_meas_mask, bit 3) clears
00233 * - Bounded by k_bmp280_poll_max iterations
00234 * 3. Read 6 bytes from k_bmp280_reg_press_msb (0xF7): pressure MSB/LSB/XLSB + temperature MSB/LSB/XLSB
00235 * 4. Assemble 20-bit ADC values:
00236 * adc_P = (buf[k_bmp280_press_msb_idx] « k_bmp280_shift_msb) |
00237 * (buf[k_bmp280_press_lsb_idx] « k_bmp280_shift_lsb_left) |
00238 * (buf[k_bmp280_press_xlsb_idx] » k_bmp280_shift_xlsb_right)
00239 * adc_T = (buf[k_bmp280_temp_msb_idx] « k_bmp280_shift_msb) |
00240 * (buf[k_bmp280_temp_lsb_idx] « k_bmp280_shift_lsb_left) |
00241 * (buf[k_bmp280_temp_xlsb_idx] » k_bmp280_shift_xlsb_right)
00242 * 5. Apply Bosch integer compensation algorithm (BMP280 datasheet section 4.2.3)
00243 *
00244 * @param[out] out Output data structure. Must not be NULL.
00245 *
00246 * @return rx_err_t Read result
00247 * @retval k_rx_ok Measurement complete, out populated
00248 * @retval k_rx_err_null_ptr out is NULL
00249 * @retval k_rx_err_not_initialized init not called
00250 * @retval k_rx_err_nack I2C communication failure
00251 * @retval k_rx_err_timeout Status poll exceeded k_bmp280_poll_max iterations

```

```

00249 *
00250 * @pre rx_bmp280_init() succeeded
00251 * @pre out non-NULL
00252 *
00253 * @post out->temp_centi_degC contains calibrated temperature * 100
00254 * @post out->press_pa_256 contains calibrated pressure * 256
00255 * @post Sensor returns to sleep mode (forced measurement complete)
00256 *
00257 * @note Not thread-safe; call from single task
00258 * @note Measurement takes ~40 ms with 16x pressure oversampling
00259 * @warning Returns k_rx_err_timeout if status bit does not clear within
00260 *         k_bmp280_poll_max polls (indicates hardware fault)
00261 *
00262 * @test tests/test_rx_bmp280.c: test_bmp280_read_success_forced_mode, test_bmp280_read_status_timeout,
00263 *         test_bmp280_read_i2c_error_propagates, test_bmp280_compensation_known_values,
00264 *         test_bmp280_read_zero_var1_returns_error
00265 *
00266 * @see rx_bmp280_init() Must be called successfully before rx_bmp280_read()
00267 *
00268 * @since Version 1.0.0
00269 */
00270 [[nodiscard]] rx_err_t rx_bmp280_read(bmp280_data_t* out);
00271
00272 #ifdef UNIT_TEST
00273 /**
00274 * @brief Reset BMP280 driver static state for unit test isolation
00275 *
00276 * @details
00277 * Clears s_initialized to false and zeroes the calibration struct.
00278 * Call from setUp() before each test so tests can run in any order
00279 * without depending on prior test state. Available only in test builds
00280 * Available only in UNIT_TEST builds.
00281 *
00282 * @pre None (may be called before init)
00283 * @post s_initialized == false; driver in pre-init state
00284 *
00285 * @note Test-only helper; not compiled into production firmware
00286 * @test tests/test_rx_bmp280.c: setUp() calls this function
00287 *
00288 * @since Version 1.0.0
00289 */
00290 void rx_bmp280_test_reset_state(void);
00291
00292 /**
00293 * @brief Set s_initialized and s_manager for direct internal function testing
00294 *
00295 * @details
00296 * Allows tests to bypass the init sequence and inject arbitrary module state
00297 * so internal functions can be called without going through rx_bmp280_init().
00298 *
00299 * @param[in] manager Bus manager pointer to store in s_manager (may be NULL)
00300 * @param[in] init_val Value to write to s_initialized
00301 *
00302 * @pre No concurrent access from other threads (test runs single-threaded)
00303 * @pre Called from test setup after rx_bmp280_test_reset_state()
00304 * @post s_initialized == init_val
00305 * @post s_manager == manager
00306 *
00307 * @note Test-only helper; not compiled into production firmware
00308 * @since Version 1.0.0
00309 */
00310 void rx_bmp280_test_set_state(rx_bus_manager_t* manager, bool init_val);
00311
00312 /**
00313 * @brief Force s_calib.dig_P1 to zero for division-by-zero coverage testing
00314 *
00315 * @details
00316 * Corrupts dig_P1 to zero so that internal_compensate_pressure() hits the
00317 * var1==0 guard when called via internal_read_and_compensate_adc().
00318 *
00319 * @pre rx_bmp280_test_set_state() or rx_bmp280_init() called
00320 * @pre No concurrent access from other threads (test runs single-threaded)
00321 * @post s_calib.dig_P1 == 0; other calibration fields unchanged
00322 * @post internal_compensate_pressure() will return k_rx_err_invalid_state when called
00323 *
00324 * @note Test-only helper; not compiled into production firmware
00325 * @since Version 1.0.0
00326 */
00327 void rx_bmp280_test_zero_calib_p1(void);
00328
00329 /**
00330 * @brief Override s_bus_name for branch coverage testing
00331 *
00332 * @details
00333 * Sets s_bus_name to the given pointer (which may be NULL) so that the
00334 * RX_ASSERT_PRE(s_bus_name != NULL, ...) branches in internal_write_reg
00335 * and rx_bmp280_init can be exercised in unit tests.

```

```

00336 *
00337 * @param[in] name New bus name pointer (may be NULL for assertion testing)
00338 *
00339 * @pre UNIT_TEST defined
00340 * @post s_bus_name == name
00341 *
00342 * @note Test-only helper; not compiled into production firmware
00343 * @since Version 1.0.0
00344 */
00345 void rx_bmp280_test_set_bus_name(const char* name);
00346 #endif /* UNIT_TEST */
00347
00348 #ifdef __cplusplus
00349 }
00350 #endif

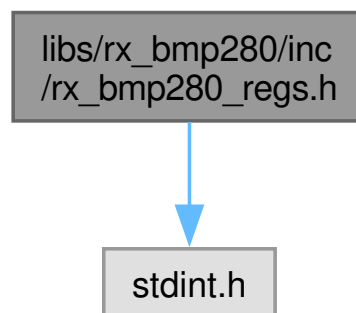
```

5.3 libs/rx_bmp280/inc/rx_bmp280_regs.h File Reference

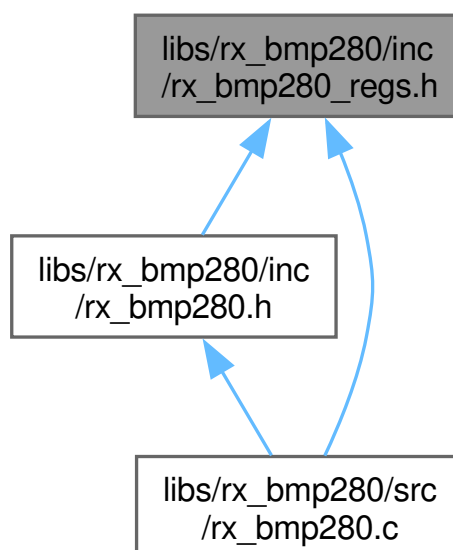
BMP280 Barometric Pressure and Temperature Sensor Register Map.

```
#include <stdint.h>
```

Include dependency graph for rx_bmp280_regs.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `enum bmp280_reg_t` : `uint8_t` {
`k_bmp280_reg_chip_id` = 0xD0 , `k_bmp280_reg_calib_start` = 0x88 , `k_bmp280_reg_calib_end`
`k_bmp280_reg_status` = 0xF3 ,
`k_bmp280_reg_ctrl_meas` = 0xF4 , `k_bmp280_reg_config` = 0xF5 , `k_bmp280_reg_press_msb`
`= 0xF7` , `k_bmp280_reg_press_lsb` = 0xF8 ,
`k_bmp280_reg_press_xlsb` = 0xF9 , `k_bmp280_reg_temp_msb` = 0xFA , `k_bmp280_reg_temp_lsb`
`= 0xFB` , `k_bmp280_reg_temp_xlsb` = 0xFC }
BMP280 register addresses.
- enum `enum bmp280_i2c_addr_t` : `uint8_t` { `k_bmp280_i2c_addr` = 0x76 }
BMP280 I2C device address.
- enum `enum bmp280_chip_id_t` : `uint8_t` { `k_bmp280_chip_id_expected` = 0x58 }
BMP280 expected chip ID value from register 0xD0.
- enum `enum bmp280_ctrl_meas_t` : `uint8_t` { `k_bmp280_ctrl_meas_val` = 0x55 }
BMP280 `ctrl_meas` register value (0xF4).
- enum `enum bmp280_config_t` : `uint8_t` { `k_bmp280_config_val` }
BMP280 config register value (0xF5).
- enum `enum bmp280_status_mask_t` : `uint8_t` { `k_bmp280_status_meas_mask` }
BMP280 status register measuring-bit mask (0xF3).
- enum `enum bmp280_calib_reg_count_t` : `uint8_t` { `k_bmp280_calib_byte_count` = 24 }
BMP280 calibration data size.
- enum `enum bmp280_poll_t` : `uint8_t` { `k_bmp280_poll_max` = 10 , `k_bmp280_poll_sleep_ticks` = 1 }
BMP280 measurement polling loop bound and sleep constant.
- enum `enum bmp280_adc_shift_t` : `uint8_t` { `k_bmp280_shift_msb` = 12 , `k_bmp280_shift_lsb_left` ,
`k_bmp280_shift_xlsb_right` }
BMP280 ADC value bit-shift constants.
- enum `enum bmp280_adc_buf_t` : `uint8_t` { `k_bmp280_adc_buf_size` = 6 }
BMP280 ADC read buffer size.
- enum `enum bmp280_adc_idx_t` : `uint8_t` {
`k_bmp280_press_msb_idx` = 0 , `k_bmp280_press_lsb_idx` = 1 , `k_bmp280_press_xlsb_idx` =
2 , `k_bmp280_temp_msb_idx` = 3 ,
`k_bmp280_temp_lsb_idx` = 4 , `k_bmp280_temp_xlsb_idx` = 5 }
BMP280 ADC buffer byte indices.
- enum `enum bmp280_calib_idx_t` : `uint8_t` {
`k_bmp280_calib_t1_lsb` = 0 , `k_bmp280_calib_t1_msb` = 1 , `k_bmp280_calib_t2_lsb` = 2 ,
`k_bmp280_calib_t2_msb` = 3 ,
`k_bmp280_calib_t3_lsb` = 4 , `k_bmp280_calib_t3_msb` = 5 , `k_bmp280_calib_p1_lsb` = 6 ,
`k_bmp280_calib_p1_msb` = 7 ,
`k_bmp280_calib_p2_lsb` = 8 , `k_bmp280_calib_p2_msb` = 9 , `k_bmp280_calib_p3_lsb` = 10
, `k_bmp280_calib_p3_msb` = 11 ,
`k_bmp280_calib_p4_lsb` = 12 , `k_bmp280_calib_p4_msb` = 13 , `k_bmp280_calib_p5_lsb` =
14 , `k_bmp280_calib_p5_msb` = 15 ,
`k_bmp280_calib_p6_lsb` = 16 , `k_bmp280_calib_p6_msb` = 17 , `k_bmp280_calib_p7_lsb` =
18 , `k_bmp280_calib_p7_msb` = 19 ,
`k_bmp280_calib_p8_lsb` = 20 , `k_bmp280_calib_p8_msb` = 21 , `k_bmp280_calib_p9_lsb` =
22 , `k_bmp280_calib_p9_msb` = 23 }
BMP280 calibration byte indices in 24-byte calib buffer.
- enum `enum bmp280_calib_shift_t` : `uint8_t` { `k_bmp280_byte_shift` = 8 }
Bit shift for little-endian 16-bit calibration coefficient assembly.
- enum `enum bmp280_write_size_t` : `uint8_t` { `k_bmp280_write_buf_size` = 2 , `k_bmp280_read_cmd_size`
= 1 , `k_bmp280_single_byte` = 1 }
I2C write/read buffer size constants for BMP280.

- enum `bmp280_write_idx_t`: `uint8_t { k_bmp280_write_idx_reg = 0, k_bmp280_write_idx_val = 1 }`
I2C write buffer index constants for BMP280 register writes.

5.3.1 Detailed Description

BMP280 Barometric Pressure and Temperature Sensor Register Map.

5.3.2 Overview

Provides all register address definitions and configuration constants for the Bosch BMP280 digital pressure sensor. All constants are defined as C23 typed enumerations following STAR project conventions.

The BMP280 measures atmospheric pressure (300-1100 hPa) and temperature (-40 to +85 degC) using a combined MEMS capacitive pressure cell and digital temperature sensor. Compensation coefficients are trimmed at the factory and stored in on-chip OTP memory (registers 0x88-0x9F).

5.3.3 Register Map Summary

Address	Content
0x88-0x9F	Calibration data (24 bytes)
0xD0	Chip ID (0x60)
0xF3	Status register
0xF4	<code>ctrl_meas</code> (<code>osrs_t</code> , <code>osrs_p</code> , <code>mode</code>)
0xF5	<code>config</code> (<code>t_sb</code> , <code>filter</code> , <code>spi3w_en</code>)
0xF7-0xF9	Pressure ADC (20-bit, MSB/LSB/XLSB)
0xFA-0xFC	Temperature ADC (20-bit, MSB/LSB/XLSB)

5.3.4 DFRobot Module Hardware

- I2C Address: 0x76 (SDO pulled LOW)
- Connected to RIIC1 (SCL1=P2.1, SDA1=P2.0)
- Power: 3.3V

See also

[rx_bmp280.h](#) Public driver API

[rx_bmp280.c](#) Driver implementation

BMP280 datasheet v1.19 for complete register descriptions

NASA Power of 10 Compliance:

- Rule 3: [PASS] No dynamic allocation (constants only)
- Rule 8: [PASS] C23 typed enums for all constants, no macros

Author

STAR Team

Date

2026-03-04

Version

1.0.0

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_bmp280_regs.h](#).

5.3.5 Enumeration Type Documentation

5.3.5.1 bmp280`adc`buf``t`enum [bmp280\\_adc\\_buf\\_t](#) : `uint8_t`

BMP280 ADC read buffer size.

Buffer size for the 6-byte burst read from register 0xF7: `buf[0..2]` = pressure MSB/LSB/XLSB, `buf[3..5]` = temperature MSB/LSB/XLSB

Since

Version 1.0.0

Enumerator

<code>k_bmp280_adc_buf_size</code>	Byte count for 6-byte burst read (press + temp)
------------------------------------	---

Definition at line 297 of file [rx_bmp280_regs.h](#).

```

00297     : uint8_t {
00298     k_bmp280_adc_buf_size = 6, /**< Byte count for 6-byte burst read (press + temp) */
00299 } bmp280_adc_buf_t;
```

5.3.5.2 bmp280`adc`idx``t`

enum `bmp280_adc_idx_t` : `uint8_t`

BMP280 ADC buffer byte indices.

Byte indices into the 6-byte ADC read buffer returned from address 0xF7. Buffer layout: `buf[0]=press_↵
_msb, buf[1]=press_lsb, buf[2]=press_xlsb, buf[3]=temp_msb, buf[4]=temp_lsb, buf[5]=temp_xlsb`

Invariant

All index values in `[0, k_bmp280_adc_buf_size - 1]`

Since

Version 1.0.0

Enumerator

<code>k_bmp280_press_msb_idx</code>	Index of pressure MSB in 6-byte buffer
<code>k_bmp280_press_lsb_idx</code>	Index of pressure LSB in 6-byte buffer
<code>k_bmp280_press_xlsb_idx</code>	Index of pressure XLSB in 6-byte buffer
<code>k_bmp280_temp_msb_idx</code>	Index of temperature MSB in 6-byte buffer
<code>k_bmp280_temp_lsb_idx</code>	Index of temperature LSB in 6-byte buffer
<code>k_bmp280_temp_xlsb_idx</code>	Index of temperature XLSB in 6-byte buffer

Definition at line 314 of file `rx_bmp280_regs.h`.

```
00314         : uint8_t {
00315   k_bmp280_press_msb_idx = 0, /**< Index of pressure MSB in 6-byte buffer */
00316   k_bmp280_press_lsb_idx = 1, /**< Index of pressure LSB in 6-byte buffer */
00317   k_bmp280_press_xlsb_idx = 2, /**< Index of pressure XLSB in 6-byte buffer */
00318   k_bmp280_temp_msb_idx  = 3, /**< Index of temperature MSB in 6-byte buffer */
00319   k_bmp280_temp_lsb_idx  = 4, /**< Index of temperature LSB in 6-byte buffer */
00320   k_bmp280_temp_xlsb_idx = 5, /**< Index of temperature XLSB in 6-byte buffer */
00321 } bmp280_adc_idx_t;
```

5.3.5.3 bmp280`adc`shift``t`

enum `bmp280_adc_shift_t` : `uint8_t`

BMP280 ADC value bit-shift constants.

Shift amounts used to assemble 20-bit ADC values from three 8-bit registers: `adc_val = ((int32_t)msb << 12) | ((int32_t)lsb << k_bmp280_shift_lsb_left) | ((int32_t)xlsb >> k_bmp280_shift_xlsb_↵
right)`

Note: `k_bmp280_shift_lsb_left` and `k_bmp280_shift_xlsb_right` have the same numeric value (4) but represent different operations: left-shift of the LSB byte and right-shift of the XLSB byte respectively.

Since

Version 1.0.0

Enumerator

k_bmp280_shift_msb	Left-shift MSB byte 12 bits for 20-bit ADC assembly (bits[19:12])
k_bmp280_shift_lsb_left	Left-shift LSB byte 4 bits for 20-bit ADC assembly (bits[11:4])
k_bmp280_shift_xlsb_right	Right-shift XLSB byte 4 bits to extract lower nibble (bits[3:0])

Definition at line 279 of file rx_bmp280_regs.h.

```
00279      : uint8_t {
00280  k_bmp280_shift_msb = 12, /**< Left-shift MSB byte 12 bits for 20-bit ADC assembly (bits[19:12]) */
00281  k_bmp280_shift_lsb_left =
00282      4, /**< Left-shift LSB byte 4 bits for 20-bit ADC assembly (bits[11:4]) */
00283  k_bmp280_shift_xlsb_right =
00284      4, /**< Right-shift XLSB byte 4 bits to extract lower nibble (bits[3:0]) */
00285 } bmp280_adc_shift_t;
```

5.3.5.4 bmp280`calib`idx` t

enum [bmp280_calib_idx_t](#) : uint8_t

BMP280 calibration byte indices in 24-byte calib buffer.

Maps the 24-byte calibration buffer to the BMP280's trimming parameter layout. All coefficients are stored little-endian (LSB first).

Since

Version 1.0.0

Enumerator

k_bmp280_calib_t1_lsb	dig_T1 low byte (uint16_t)
k_bmp280_calib_t1_msb	dig_T1 high byte
k_bmp280_calib_t2_lsb	dig_T2 low byte (int16_t)
k_bmp280_calib_t2_msb	dig_T2 high byte
k_bmp280_calib_t3_lsb	dig_T3 low byte (int16_t)
k_bmp280_calib_t3_msb	dig_T3 high byte
k_bmp280_calib_p1_lsb	dig_P1 low byte (uint16_t)
k_bmp280_calib_p1_msb	dig_P1 high byte
k_bmp280_calib_p2_lsb	dig_P2 low byte (int16_t)
k_bmp280_calib_p2_msb	dig_P2 high byte
k_bmp280_calib_p3_lsb	dig_P3 low byte (int16_t)
k_bmp280_calib_p3_msb	dig_P3 high byte
k_bmp280_calib_p4_lsb	dig_P4 low byte (int16_t)
k_bmp280_calib_p4_msb	dig_P4 high byte
k_bmp280_calib_p5_lsb	dig_P5 low byte (int16_t)
k_bmp280_calib_p5_msb	dig_P5 high byte

k_bmp280_calib_p6_lsb	dig_P6 low byte (int16_t)
k_bmp280_calib_p6_msb	dig_P6 high byte
k_bmp280_calib_p7_lsb	dig_P7 low byte (int16_t)
k_bmp280_calib_p7_msb	dig_P7 high byte
k_bmp280_calib_p8_lsb	dig_P8 low byte (int16_t)
k_bmp280_calib_p8_msb	dig_P8 high byte
k_bmp280_calib_p9_lsb	dig_P9 low byte (int16_t)
k_bmp280_calib_p9_msb	dig_P9 high byte

Definition at line 333 of file `rx_bmp280_regs.h`.

```

00333      : uint8_t {
00334  k_bmp280_calib_t1_lsb = 0, /**< dig_T1 low byte (uint16_t) */
00335  k_bmp280_calib_t1_msb = 1, /**< dig_T1 high byte */
00336  k_bmp280_calib_t2_lsb = 2, /**< dig_T2 low byte (int16_t) */
00337  k_bmp280_calib_t2_msb = 3, /**< dig_T2 high byte */
00338  k_bmp280_calib_t3_lsb = 4, /**< dig_T3 low byte (int16_t) */
00339  k_bmp280_calib_t3_msb = 5, /**< dig_T3 high byte */
00340  k_bmp280_calib_p1_lsb = 6, /**< dig_P1 low byte (uint16_t) */
00341  k_bmp280_calib_p1_msb = 7, /**< dig_P1 high byte */
00342  k_bmp280_calib_p2_lsb = 8, /**< dig_P2 low byte (int16_t) */
00343  k_bmp280_calib_p2_msb = 9, /**< dig_P2 high byte */
00344  k_bmp280_calib_p3_lsb = 10, /**< dig_P3 low byte (int16_t) */
00345  k_bmp280_calib_p3_msb = 11, /**< dig_P3 high byte */
00346  k_bmp280_calib_p4_lsb = 12, /**< dig_P4 low byte (int16_t) */
00347  k_bmp280_calib_p4_msb = 13, /**< dig_P4 high byte */
00348  k_bmp280_calib_p5_lsb = 14, /**< dig_P5 low byte (int16_t) */
00349  k_bmp280_calib_p5_msb = 15, /**< dig_P5 high byte */
00350  k_bmp280_calib_p6_lsb = 16, /**< dig_P6 low byte (int16_t) */
00351  k_bmp280_calib_p6_msb = 17, /**< dig_P6 high byte */
00352  k_bmp280_calib_p7_lsb = 18, /**< dig_P7 low byte (int16_t) */
00353  k_bmp280_calib_p7_msb = 19, /**< dig_P7 high byte */
00354  k_bmp280_calib_p8_lsb = 20, /**< dig_P8 low byte (int16_t) */
00355  k_bmp280_calib_p8_msb = 21, /**< dig_P8 high byte */
00356  k_bmp280_calib_p9_lsb = 22, /**< dig_P9 low byte (int16_t) */
00357  k_bmp280_calib_p9_msb = 23, /**< dig_P9 high byte */
00358 } bmp280_calib_idx_t;

```

5.3.5.5 bmp280_calib_reg_count_t

```
enum bmp280_calib_reg_count_t : uint8_t
```

BMP280 calibration data size.

The calibration block at 0x88 contains 24 bytes encoding three temperature coefficients and nine pressure coefficients:

- T1 (uint16), T2 (int16), T3 (int16) at 0x88-0x8D
- P1 (uint16)..P9 (int16) at 0x8E-0x9F

Invariant

Calibration bytes always valid (factory programmed in OTP)

Since

Version 1.0.0

Enumerator

k_bmp280_calib_byte_count	Number of calibration coefficient bytes (0x88-0x9F)
---------------------------	---

Definition at line 233 of file [rx_bmp280_regs.h](#).

```
00233         : uint8_t {
00234   k_bmp280_calib_byte_count = 24, /**< Number of calibration coefficient bytes (0x88-0x9F) */
00235 } bmp280_calib_reg_count_t;
```

5.3.5.6 bmp280`calib`shift`[t](#)

```
enum bmp280\_calib\_shift\_t : uint8_t
```

Bit shift for little-endian 16-bit calibration coefficient assembly.

Calibration coefficients are stored little-endian (LSB first). The MSB must be shifted left by 8 bits to form a 16-bit value.

Since

Version 1.0.0

Enumerator

k_bmp280_byte_shift	Shift MSB byte left 8 bits for 16-bit assembly
---------------------	--

Definition at line 370 of file [rx_bmp280_regs.h](#).

```
00370         : uint8_t {
00371   k_bmp280_byte_shift = 8, /**< Shift MSB byte left 8 bits for 16-bit assembly */
00372 } bmp280_calib_shift_t;
```

5.3.5.7 bmp280`chip`id`[t](#)

```
enum bmp280\_chip\_id\_t : uint8_t
```

BMP280 expected chip ID value from register 0xD0.

The BMP280 always reads 0x60 from register k_bmp280_reg_chip_id (0xD0). This constant is used during initialization to verify that the correct device is present on the I2C bus before reading calibration data.

Invariant

The BMP280 chip ID register always reads 0x58 on authentic BMP280 devices; 0x60 is the BME280 (pressure + humidity) ID and must not be accepted here.

Since

Version 1.0.0

Enumerator

k_bmp280_chip_id_expected	BMP280 chip ID at reg 0xD0 (DS 5.2.1)
---------------------------	---------------------------------------

Definition at line 142 of file [rx_bmp280_regs.h](#).

```
00142         : uint8_t {
00143   k_bmp280_chip_id_expected = 0x58, /**< BMP280 chip ID at reg 0xD0 (DS 5.2.1) */
00144 } bmp280_chip_id_t;
```

5.3.5.8 bmp280_config_t

enum [bmp280_config_t](#) : uint8_t

BMP280 config register value (0xF5).

Pre-computed register value for IIR filter and standby configuration:

config (0xF5) = 0x04:

- t_sb[7:5] = 000 = 0.5 ms standby time (not used in forced mode)
- filter[4:2] = 001 = IIR filter coefficient 2
- spi3w_en[0] = 0 = SPI disabled

Invariant

k_bmp280_config_val sets filter coefficient 2 to reduce noise on pressure readings
 SPI disabled bit ([0] = 0) ensures I2C mode remains active

See also

[bmp280_ctrl_meas_t](#) Oversampling and mode configuration register value

[bmp280_status_mask_t](#) Status register measuring-bit mask

Since

Version 1.0.0

Enumerator

k_bmp280_config_val	config: filter coef=2 (filter[4:2]=001), standby=0.5ms, SPI disabled
---------------------	--

Definition at line 190 of file [rx_bmp280_regs.h](#).

```
00190         : uint8_t {
00191   k_bmp280_config_val =
00192     0x04, /**< config: filter coef=2 (filter[4:2]=001), standby=0.5ms, SPI disabled */
00193 } bmp280_config_t;
```

5.3.5.9 bmp280`ctrl`meas`t

enum [bmp280_ctrl_meas_t](#) : uint8_t

BMP280 ctrl_meas register value (0xF4).

Pre-computed register value for the required measurement configuration:

ctrl_meas (0xF4) = 0x55:

- osrs_t[7:5] = 010 = 2x oversampling for temperature
- osrs_p[4:2] = 101 = 16x oversampling for pressure
- mode[1:0] = 01 = Forced mode (one measurement, then sleep)

Invariant

k_bmp280_ctrl_meas_val encodes osrs_t=2x, osrs_p=16x, forced mode exactly

Forced mode bits ([1:0] = 01) ensure sensor returns to sleep after each measurement

See also

[bmp280_config_t](#) IIR filter and standby configuration register value

[bmp280_status_mask_t](#) Status register measuring-bit mask

Since

Version 1.0.0

Enumerator

k_bmp280_ctrl_meas_val	ctrl_meas: osrs_t=2x, osrs_p=16x, forced mode
------------------------	---

Definition at line 166 of file [rx_bmp280_regs.h](#).

```
00166      : uint8_t {
00167  k_bmp280_ctrl_meas_val = 0x55, /**< ctrl_meas: osrs_t=2x, osrs_p=16x, forced mode */
00168 } bmp280_ctrl_meas_t;
```

5.3.5.10 bmp280`i2c`addr`t

enum [bmp280_i2c_addr_t](#) : uint8_t

BMP280 I2C device address.

The I2C address is selected by the SDO pin:

- SDO = LOW (DFRobot module default): 0x76
- SDO = HIGH: 0x77

Invariant

`k_bmp280_i2c_addr` is a valid 7-bit I2C address (0x00..0x7F)

Value is constant and determined solely by the SDO pin strapping

Since

Version 1.0.0

Enumerator

<code>k_bmp280_i2c_addr</code>	I2C device address (SDO=LOW, DFRobot module)
--------------------------------	--

Definition at line 123 of file [rx_bmp280_regs.h](#).

```
00123     : uint8_t {
00124   k_bmp280_i2c_addr = 0x76, /**< I2C device address (SDO=LOW, DFRobot module) */
00125 } bmp280_i2c_addr_t;
```

5.3.5.11 `bmp280_poll_t`

enum [bmp280_poll_t](#) : uint8_t

BMP280 measurement polling loop bound and sleep constant.

Maximum number of status register polls when waiting for a forced measurement to complete. With 16x oversampling, maximum measurement time is approximately 40 ms. Each poll sleeps `k_bmp280_poll_sleep_ticks` between reads to yield CPU time. At 1 tick (10 ms) per poll, 10 iterations provides ~100 ms maximum poll time, well under the 900 ms IMU task IWDT watchdog timeout.

Invariant

`k_bmp280_poll_max` > 0 (loop must terminate)

`k_bmp280_poll_sleep_ticks` >= 1 (must yield at least one tick)

`k_bmp280_poll_max` * `k_bmp280_poll_sleep_ticks` * 10 < 900 (must not exceed IWDT timeout)

Since

Version 1.0.0

Enumerator

<code>k_bmp280_poll_max</code>	Maximum status poll iterations (~100 ms at 100 Hz tick)
<code>k_bmp280_poll_sleep_ticks</code>	Ticks to sleep between polls (1 tick = 10 ms at 100 Hz)

Definition at line 255 of file [rx_bmp280_regs.h](#).

```
00255     : uint8_t {
00256   k_bmp280_poll_max      = 10, /**< Maximum status poll iterations (~100 ms at 100 Hz tick) */
00257   k_bmp280_poll_sleep_ticks = 1, /**< Ticks to sleep between polls (1 tick = 10 ms at 100 Hz) */
00258 } bmp280_poll_t;
```


5.3.5.12 bmp280`reg`t

enum `bmp280_reg_t` : `uint8_t`

BMP280 register addresses.

Register addresses for all BMP280 control, configuration, and output registers. Calibration data (0x88-0x9F) is accessed as a 24-byte burst read starting at `k_bmp280_reg_calib_start` .

ADC output registers (0xF7-0xFC) contain 20-bit raw values:

- Pressure: bits[19:4] in `PRESS_MSB[7:0]` and `PRESS_LSB[7:0]` and bits[3:0] in `PRESS_XLSB[7:4]`
- Temperature: same layout at `TEMP_MSB/LSB/XLSB`

Invariant

`k_bmp280_reg_calib_start` contains factory-calibrated coefficients

0xF7 output is valid only after measurement completes

Precondition

BMP280 has completed power-on reset (POR) sequence

I2C bus interface is initialized and BMP280 is addressable

Postcondition

Registers accessible after BMP280 power-on reset sequence completes

All addresses remain constant for the lifetime of the device

Since

Version 1.0.0

Enumerator

<code> k_bmp280_reg_chip_id </code>	Chip ID register: contains device ID 0x60 for BMP280
<code> k_bmp280_reg_calib_start </code>	Start of 24-byte calibration coefficient block
<code> k_bmp280_reg_calib_end </code>	Last byte of 24-byte calibration coefficient block (0x88-0x9F)
<code> k_bmp280_reg_status </code>	Status: bit 3 = measuring, bit 0 = <code> im_update </code>
<code> k_bmp280_reg_ctrl_meas </code>	Control: <code> osrs_t[7:5] </code> <code> osrs_p[4:2] </code> <code> mode[1:0] </code>
<code> k_bmp280_reg_config </code>	Config: <code> t_sb[7:5] </code> <code> filter[4:2] </code> <code> spi3w_en[0] </code>
<code> k_bmp280_reg_press_msb </code>	Pressure MSB: <code> press_raw[19:12] </code>

k_bmp280_reg_press_lsb	Pressure LSB: press_raw[11:4]
k_bmp280_reg_press_xlsb	Pressure XLSB: press_raw[3:0] in bits [7:4]
k_bmp280_reg_temp_msb	Temperature MSB: temp_raw[19:12]
k_bmp280_reg_temp_lsb	Temperature LSB: temp_raw[11:4]
k_bmp280_reg_temp_xlsb	Temperature XLSB: temp_raw[3:0] in bits [7:4]

Definition at line 88 of file [rx_bmp280_regs.h](#).

```

00088      : uint8_t {
00089  k_bmp280_reg_chip_id    = 0xD0, /**< Chip ID register: contains device ID 0x60 for BMP280 */
00090  k_bmp280_reg_calib_start = 0x88, /**< Start of 24-byte calibration coefficient block */
00091  k_bmp280_reg_calib_end  =
00092    0x9F, /**< Last byte of 24-byte calibration coefficient block (0x88-0x9F) */
00093  k_bmp280_reg_status    = 0xF3, /**< Status: bit 3 = measuring, bit 0 = im_update */
00094  k_bmp280_reg_ctrl_meas = 0xF4, /**< Control: osrs_t[7:5] osrs_p[4:2] mode[1:0] */
00095  k_bmp280_reg_config     = 0xF5, /**< Config: t_sb[7:5] filter[4:2] spi3w_en[0] */
00096  k_bmp280_reg_press_msb  = 0xF7, /**< Pressure MSB: press_raw[19:12] */
00097  k_bmp280_reg_press_lsb  = 0xF8, /**< Pressure LSB: press_raw[11:4] */
00098  k_bmp280_reg_press_xlsb = 0xF9, /**< Pressure XLSB: press_raw[3:0] in bits [7:4] */
00099  k_bmp280_reg_temp_msb   = 0xFA, /**< Temperature MSB: temp_raw[19:12] */
00100  k_bmp280_reg_temp_lsb   = 0xFB, /**< Temperature LSB: temp_raw[11:4] */
00101  k_bmp280_reg_temp_xlsb = 0xFC, /**< Temperature XLSB: temp_raw[3:0] in bits [7:4] */
00102 } bmp280_reg_t;

```

5.3.5.13 bmp280_status_mask_t

```
enum bmp280\_status\_mask\_t : uint8_t
```

BMP280 status register measuring-bit mask (0xF3).

Bitmask for the measuring bit in the BMP280 status register:

status (0xF3) measuring bit:

- Bit 3 = 1 while measurement in progress
- Bit 3 = 0 when measurement complete (ADC data ready)

Invariant

k_bmp280_status_meas_mask isolates exactly bit 3 of the status register

Value is read-only; writing this mask to the sensor has no effect

See also

[bmp280_ctrl_meas_t](#) Oversampling and mode configuration register value

[bmp280_config_t](#) IIR filter and standby configuration register value

Since

Version 1.0.0

Enumerator

k_bmp280_status_meas_mask	Status measuring bit: bit 3 set while measurement in progress
---------------------------	---

Definition at line 214 of file [rx_bmp280_regs.h](#).

```
00214         : uint8_t {
00215     k_bmp280_status_meas_mask =
00216         0x08, /**< Status measuring bit: bit 3 set while measurement in progress */
00217 } bmp280_status_mask_t;
```

5.3.5.14 bmp280`write`idx` t

```
enum bmp280\_write\_idx\_t : uint8_t
```

I2C write buffer index constants for BMP280 register writes.

Index constants for accessing fields within the 2-byte I2C write buffer [register_address, data_byte] used by [internal_write_reg\(\)](#).

See also

[bmp280_write_size_t](#) Size constants for the same buffer

Since

Version 1.0.0

Enumerator

k_bmp280_write_idx_reg	Index of register address in write buffer
k_bmp280_write_idx_val	Index of register value in write buffer

Definition at line 404 of file [rx_bmp280_regs.h](#).

```
00404         : uint8_t {
00405     k_bmp280_write_idx_reg = 0, /**< Index of register address in write buffer */
00406     k_bmp280_write_idx_val = 1, /**< Index of register value in write buffer */
00407 } bmp280_write_idx_t;
```

5.3.5.15 bmp280`write`size` t

```
enum bmp280\_write\_size\_t : uint8_t
```

I2C write/read buffer size constants for BMP280.

BMP280 register write sends [register_address, data_byte] as 2 bytes. Size constants for determining transaction lengths.

Invariant

`k_bmp280_write_buf_size == k_bmp280_write_idx_val + 1` (buffer exactly fits register + value)

Since

Version 1.0.0

Enumerator

<code>k_bmp280_write_buf_size</code>	Size of [reg, val] write buffer
<code>k_bmp280_read_cmd_size</code>	Size of register address read command
<code>k_bmp280_single_byte</code>	Size for single byte reads

Definition at line 386 of file `rx_bmp280_regs.h`.

```
00386      : uint8_t {
00387  k_bmp280_write_buf_size = 2, /**< Size of [reg, val] write buffer */
00388  k_bmp280_read_cmd_size = 1, /**< Size of register address read command */
00389  k_bmp280_single_byte   = 1, /**< Size for single byte reads */
00390 } bmp280_write_size_t;
```

5.4 rx_bmp280_regs.h

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_bmp280_regs.h
00003  * @brief BMP280 Barometric Pressure and Temperature Sensor Register Map
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * Provides all register address definitions and configuration constants for the
00009  * Bosch BMP280 digital pressure sensor. All constants are defined as C23 typed
00010  * enumerations following STAR project conventions.
00011  *
00012  * The BMP280 measures atmospheric pressure (300-1100 hPa) and temperature
00013  * (-40 to +85 degC) using a combined MEMS capacitive pressure cell and digital
00014  * temperature sensor. Compensation coefficients are trimmed at the factory and
00015  * stored in on-chip OTP memory (registers 0x88-0x9F).
00016  *
00017  * # Register Map Summary
00018  *
00019  * | Address | Content |
00020  * |-----|-----|
00021  * | 0x88-0x9F | Calibration data (24 bytes) |
00022  * | 0xD0 | Chip ID (0x60) |
00023  * | 0xF3 | Status register |
00024  * | 0xF4 | ctrl_meas (osrs_t, osrs_p, mode) |
00025  * | 0xF5 | config (t_sb, filter, spi3w_en) |
00026  * | 0xF7-0xF9 | Pressure ADC (20-bit, MSB/LSB/XLSB) |
00027  * | 0xFA-0xFC | Temperature ADC (20-bit, MSB/LSB/XLSB) |
00028  *
00029  * # DFRobot Module Hardware
00030  *
00031  * - I2C Address: 0x76 (SDO pulled LOW)
00032  * - Connected to RIIC1 (SCL1=P2.1, SDA1=P2.0)
00033  * - Power: 3.3V
00034  *
00035  * @see rx_bmp280.h Public driver API
00036  * @see rx_bmp280.c Driver implementation
00037  * @see BMP280 datasheet v1.19 for complete register descriptions
00038  *
00039  * @par NASA Power of 10 Compliance:
00040  * - **Rule 3:** [PASS] No dynamic allocation (constants only)
00041  * - **Rule 8:** [PASS] C23 typed enums for all constants, no macros
00042  *
00043  * @author STAR Team
00044  * @date 2026-03-04
00045  * @version 1.0.0
00046  * @copyright Copyright (c) 2026 Locked Inc.
```

```

00047 * SPDX-License-Identifier: MIT
00048 */
00049
00050 #pragma once
00051
00052 #include <stdint.h>
00053
00054 #ifdef __cplusplus
00055 extern "C" {
00056 #endif
00057
00058 /*
=====
00059 * Register Address Map
00060 *
=====
00061 */
00062
00063 /**
00064 * @enum bmp280_reg_t
00065 * @brief BMP280 register addresses
00066 *
00067 * @details
00068 * Register addresses for all BMP280 control, configuration, and output
00069 * registers. Calibration data (0x88-0x9F) is accessed as a 24-byte burst
00070 * read starting at k_bmp280_reg_calib_start.
00071 *
00072 * ADC output registers (0xF7-0xFC) contain 20-bit raw values:
00073 * - Pressure: bits[19:4] in PRESS_MSB[7:0] and PRESS_LSB[7:0] and
00074 *   bits[3:0] in PRESS_XLSB[7:4]
00075 * - Temperature: same layout at TEMP_MSB/LSB/XLSB
00076 *
00077 * @invariant k_bmp280_reg_calib_start contains factory-calibrated coefficients
00078 * @invariant 0xF7 output is valid only after measurement completes
00079 *
00080 * @pre BMP280 has completed power-on reset (POR) sequence
00081 * @pre I2C bus interface is initialized and BMP280 is addressable
00082 *
00083 * @post Registers accessible after BMP280 power-on reset sequence completes
00084 * @post All addresses remain constant for the lifetime of the device
00085 *
00086 * @since Version 1.0.0
00087 */
00088 typedef enum : uint8_t {
00089     k_bmp280_reg_chip_id      = 0xD0, /**< Chip ID register: contains device ID 0x60 for BMP280 */
00090     k_bmp280_reg_calib_start = 0x88, /**< Start of 24-byte calibration coefficient block */
00091     k_bmp280_reg_calib_end   =
00092         0x9F, /**< Last byte of 24-byte calibration coefficient block (0x88-0x9F) */
00093     k_bmp280_reg_status      = 0xF3, /**< Status: bit 3 = measuring, bit 0 = im_update */
00094     k_bmp280_reg_ctrl_meas   = 0xF4, /**< Control: osrs_t[7:5] osrs_p[4:2] mode[1:0] */
00095     k_bmp280_reg_config      = 0xF5, /**< Config: t_sb[7:5] filter[4:2] spi3w_en[0] */
00096     k_bmp280_reg_press_msb   = 0xF7, /**< Pressure MSB: press_raw[19:12] */
00097     k_bmp280_reg_press_lsb   = 0xF8, /**< Pressure LSB: press_raw[11:4] */
00098     k_bmp280_reg_press_xlsb  = 0xF9, /**< Pressure XLSB: press_raw[3:0] in bits [7:4] */
00099     k_bmp280_reg_temp_msb    = 0xFA, /**< Temperature MSB: temp_raw[19:12] */
00100     k_bmp280_reg_temp_lsb    = 0xFB, /**< Temperature LSB: temp_raw[11:4] */
00101     k_bmp280_reg_temp_xlsb   = 0xFC, /**< Temperature XLSB: temp_raw[3:0] in bits [7:4] */
00102 } bmp280_reg_t;
00103
00104 /*
=====
00105 * Configuration Constants
00106 *
=====
00107 */
00108
00109 /**
00110 * @enum bmp280_i2c_addr_t
00111 * @brief BMP280 I2C device address
00112 *
00113 * @details
00114 * The I2C address is selected by the SDO pin:
00115 * - SDO = LOW (DFRobot module default): 0x76
00116 * - SDO = HIGH: 0x77
00117 *
00118 * @invariant k_bmp280_i2c_addr is a valid 7-bit I2C address (0x00..0x7F)
00119 * @invariant Value is constant and determined solely by the SDO pin strapping
00120 *
00121 * @since Version 1.0.0
00122 */
00123 typedef enum : uint8_t {
00124     k_bmp280_i2c_addr = 0x76, /**< I2C device address (SDO=LOW, DFRobot module) */
00125 } bmp280_i2c_addr_t;
00126
00127 /**
00128 * @enum bmp280_chip_id_t
00129 * @brief BMP280 expected chip ID value from register 0xD0

```

```

00130 *
00131 * @details
00132 * The BMP280 always reads 0x60 from register k_bmp280_reg_chip_id (0xD0).
00133 * This constant is used during initialization to verify that the correct
00134 * device is present on the I2C bus before reading calibration data.
00135 *
00136 * @invariant The BMP280 chip ID register always reads 0x58 on authentic BMP280
00137 * devices; 0x60 is the BME280 (pressure + humidity) ID and must not
00138 * be accepted here.
00139 *
00140 * @since Version 1.0.0
00141 */
00142 typedef enum : uint8_t {
00143     k_bmp280_chip_id_expected = 0x58, /**< BMP280 chip ID at reg 0xD0 (DS 5.2.1) */
00144 } bmp280_chip_id_t;
00145
00146 /**
00147 * @enum bmp280_ctrl_meas_t
00148 * @brief BMP280 ctrl_meas register value (0xF4)
00149 *
00150 * @details
00151 * Pre-computed register value for the required measurement configuration:
00152 *
00153 * **ctrl_meas (0xF4) = 0x55:**
00154 * - osrs_t[7:5] = 010 = 2x oversampling for temperature
00155 * - osrs_p[4:2] = 101 = 16x oversampling for pressure
00156 * - mode[1:0] = 01 = Forced mode (one measurement, then sleep)
00157 *
00158 * @invariant k_bmp280_ctrl_meas_val encodes osrs_t=2x, osrs_p=16x, forced mode exactly
00159 * @invariant Forced mode bits ([1:0] = 01) ensure sensor returns to sleep after each measurement
00160 *
00161 * @see bmp280_config_t IIR filter and standby configuration register value
00162 * @see bmp280_status_mask_t Status register measuring-bit mask
00163 *
00164 * @since Version 1.0.0
00165 */
00166 typedef enum : uint8_t {
00167     k_bmp280_ctrl_meas_val = 0x55, /**< ctrl_meas: osrs_t=2x, osrs_p=16x, forced mode */
00168 } bmp280_ctrl_meas_t;
00169
00170 /**
00171 * @enum bmp280_config_t
00172 * @brief BMP280 config register value (0xF5)
00173 *
00174 * @details
00175 * Pre-computed register value for IIR filter and standby configuration:
00176 *
00177 * **config (0xF5) = 0x04:**
00178 * - t_sb[7:5] = 000 = 0.5 ms standby time (not used in forced mode)
00179 * - filter[4:2] = 001 = IIR filter coefficient 2
00180 * - spi3w_en[0] = 0 = SPI disabled
00181 *
00182 * @invariant k_bmp280_config_val sets filter coefficient 2 to reduce noise on pressure readings
00183 * @invariant SPI disabled bit ([0] = 0) ensures I2C mode remains active
00184 *
00185 * @see bmp280_ctrl_meas_t Oversampling and mode configuration register value
00186 * @see bmp280_status_mask_t Status register measuring-bit mask
00187 *
00188 * @since Version 1.0.0
00189 */
00190 typedef enum : uint8_t {
00191     k_bmp280_config_val =
00192         0x04, /**< config: filter coef=2 (filter[4:2]=001), standby=0.5ms, SPI disabled */
00193 } bmp280_config_t;
00194
00195 /**
00196 * @enum bmp280_status_mask_t
00197 * @brief BMP280 status register measuring-bit mask (0xF3)
00198 *
00199 * @details
00200 * Bitmask for the measuring bit in the BMP280 status register:
00201 *
00202 * **status (0xF3) measuring bit:**
00203 * - Bit 3 = 1 while measurement in progress
00204 * - Bit 3 = 0 when measurement complete (ADC data ready)
00205 *
00206 * @invariant k_bmp280_status_meas_mask isolates exactly bit 3 of the status register
00207 * @invariant Value is read-only; writing this mask to the sensor has no effect
00208 *
00209 * @see bmp280_ctrl_meas_t Oversampling and mode configuration register value
00210 * @see bmp280_config_t IIR filter and standby configuration register value
00211 *
00212 * @since Version 1.0.0
00213 */
00214 typedef enum : uint8_t {
00215     k_bmp280_status_meas_mask =
00216         0x08, /**< Status measuring bit: bit 3 set while measurement in progress */

```

```

00217 } bmp280_status_mask_t;
00218
00219 /**
00220  * @enum bmp280_calib_reg_count_t
00221  * @brief BMP280 calibration data size
00222  *
00223  * @details
00224  * The calibration block at 0x88 contains 24 bytes encoding three temperature
00225  * coefficients and nine pressure coefficients:
00226  * - T1 (uint16), T2 (int16), T3 (int16) at 0x88-0x8D
00227  * - P1 (uint16)..P9 (int16) at 0x8E-0x9F
00228  *
00229  * @invariant Calibration bytes always valid (factory programmed in OTP)
00230  *
00231  * @since Version 1.0.0
00232  */
00233 typedef enum : uint8_t {
00234     k_bmp280_calib_byte_count = 24, /**< Number of calibration coefficient bytes (0x88-0x9F) */
00235 } bmp280_calib_reg_count_t;
00236
00237 /**
00238  * @enum bmp280_poll_t
00239  * @brief BMP280 measurement polling loop bound and sleep constant
00240  *
00241  * @details
00242  * Maximum number of status register polls when waiting for a forced
00243  * measurement to complete. With 16x oversampling, maximum measurement
00244  * time is approximately 40 ms. Each poll sleeps k_bmp280_poll_sleep_ticks
00245  * between reads to yield CPU time. At 1 tick (10 ms) per poll, 10
00246  * iterations provides ~100 ms maximum poll time, well under the 900 ms
00247  * IMU task IWDWT watchdog timeout.
00248  *
00249  * @invariant k_bmp280_poll_max > 0 (loop must terminate)
00250  * @invariant k_bmp280_poll_sleep_ticks >= 1 (must yield at least one tick)
00251  * @invariant k_bmp280_poll_max * k_bmp280_poll_sleep_ticks * 10 < 900 (must not exceed IWDWT timeout)
00252  *
00253  * @since Version 1.0.0
00254  */
00255 typedef enum : uint8_t {
00256     k_bmp280_poll_max = 10, /**< Maximum status poll iterations (~100 ms at 100 Hz tick) */
00257     k_bmp280_poll_sleep_ticks = 1, /**< Ticks to sleep between polls (1 tick = 10 ms at 100 Hz) */
00258 } bmp280_poll_t;
00259
00260 /*
=====
00261  * ADC Assembly Constants
00262  *
=====
00263  */
00264
00265 /**
00266  * @enum bmp280_adc_shift_t
00267  * @brief BMP280 ADC value bit-shift constants
00268  *
00269  * @details
00270  * Shift amounts used to assemble 20-bit ADC values from three 8-bit registers:
00271  * adc_val = ((int32_t)msb « 12) | ((int32_t)lsb « k_bmp280_shift_lsb_left) | ((int32_t)xlsb «
k_bmp280_shift_xlsb_right)
00272  *
00273  * Note: k_bmp280_shift_lsb_left and k_bmp280_shift_xlsb_right have the same numeric
00274  * value (4) but represent different operations: left-shift of the LSB byte and
00275  * right-shift of the XLSB byte respectively.
00276  *
00277  * @since Version 1.0.0
00278  */
00279 typedef enum : uint8_t {
00280     k_bmp280_shift_msb = 12, /**< Left-shift MSB byte 12 bits for 20-bit ADC assembly (bits[19:12]) */
00281     k_bmp280_shift_lsb_left =
00282         4, /**< Left-shift LSB byte 4 bits for 20-bit ADC assembly (bits[11:4]) */
00283     k_bmp280_shift_xlsb_right =
00284         4, /**< Right-shift XLSB byte 4 bits to extract lower nibble (bits[3:0]) */
00285 } bmp280_adc_shift_t;
00286
00287 /**
00288  * @enum bmp280_adc_buf_t
00289  * @brief BMP280 ADC read buffer size
00290  *
00291  * @details
00292  * Buffer size for the 6-byte burst read from register 0xF7:
00293  * buf[0..2] = pressure MSB/LSB/XLSB, buf[3..5] = temperature MSB/LSB/XLSB
00294  *
00295  * @since Version 1.0.0
00296  */
00297 typedef enum : uint8_t {
00298     k_bmp280_adc_buf_size = 6, /**< Byte count for 6-byte burst read (press + temp) */
00299 } bmp280_adc_buf_t;
00300

```

```

00301 /**
00302  * @enum bmp280_adc_idx_t
00303  * @brief BMP280 ADC buffer byte indices
00304  *
00305  * @details
00306  * Byte indices into the 6-byte ADC read buffer returned from address 0xF7.
00307  * Buffer layout: buf[0]=press_msb, buf[1]=press_lsb, buf[2]=press_xlsb,
00308  * buf[3]=temp_msb, buf[4]=temp_lsb, buf[5]=temp_xlsb
00309  *
00310  * @invariant All index values in [0, k_bmp280_adc_buf_size - 1]
00311  *
00312  * @since Version 1.0.0
00313  */
00314 typedef enum : uint8_t {
00315     k_bmp280_press_msb_idx = 0, /**< Index of pressure MSB in 6-byte buffer */
00316     k_bmp280_press_lsb_idx = 1, /**< Index of pressure LSB in 6-byte buffer */
00317     k_bmp280_press_xlsb_idx = 2, /**< Index of pressure XLSB in 6-byte buffer */
00318     k_bmp280_temp_msb_idx = 3, /**< Index of temperature MSB in 6-byte buffer */
00319     k_bmp280_temp_lsb_idx = 4, /**< Index of temperature LSB in 6-byte buffer */
00320     k_bmp280_temp_xlsb_idx = 5, /**< Index of temperature XLSB in 6-byte buffer */
00321 } bmp280_adc_idx_t;
00322
00323 /**
00324  * @enum bmp280_calib_idx_t
00325  * @brief BMP280 calibration byte indices in 24-byte calib buffer
00326  *
00327  * @details
00328  * Maps the 24-byte calibration buffer to the BMP280's trimming parameter
00329  * layout. All coefficients are stored little-endian (LSB first).
00330  *
00331  * @since Version 1.0.0
00332  */
00333 typedef enum : uint8_t {
00334     k_bmp280_calib_t1_lsb = 0, /**< dig_T1 low byte (uint16_t) */
00335     k_bmp280_calib_t1_msb = 1, /**< dig_T1 high byte */
00336     k_bmp280_calib_t2_lsb = 2, /**< dig_T2 low byte (int16_t) */
00337     k_bmp280_calib_t2_msb = 3, /**< dig_T2 high byte */
00338     k_bmp280_calib_t3_lsb = 4, /**< dig_T3 low byte (int16_t) */
00339     k_bmp280_calib_t3_msb = 5, /**< dig_T3 high byte */
00340     k_bmp280_calib_p1_lsb = 6, /**< dig_P1 low byte (uint16_t) */
00341     k_bmp280_calib_p1_msb = 7, /**< dig_P1 high byte */
00342     k_bmp280_calib_p2_lsb = 8, /**< dig_P2 low byte (int16_t) */
00343     k_bmp280_calib_p2_msb = 9, /**< dig_P2 high byte */
00344     k_bmp280_calib_p3_lsb = 10, /**< dig_P3 low byte (int16_t) */
00345     k_bmp280_calib_p3_msb = 11, /**< dig_P3 high byte */
00346     k_bmp280_calib_p4_lsb = 12, /**< dig_P4 low byte (int16_t) */
00347     k_bmp280_calib_p4_msb = 13, /**< dig_P4 high byte */
00348     k_bmp280_calib_p5_lsb = 14, /**< dig_P5 low byte (int16_t) */
00349     k_bmp280_calib_p5_msb = 15, /**< dig_P5 high byte */
00350     k_bmp280_calib_p6_lsb = 16, /**< dig_P6 low byte (int16_t) */
00351     k_bmp280_calib_p6_msb = 17, /**< dig_P6 high byte */
00352     k_bmp280_calib_p7_lsb = 18, /**< dig_P7 low byte (int16_t) */
00353     k_bmp280_calib_p7_msb = 19, /**< dig_P7 high byte */
00354     k_bmp280_calib_p8_lsb = 20, /**< dig_P8 low byte (int16_t) */
00355     k_bmp280_calib_p8_msb = 21, /**< dig_P8 high byte */
00356     k_bmp280_calib_p9_lsb = 22, /**< dig_P9 low byte (int16_t) */
00357     k_bmp280_calib_p9_msb = 23, /**< dig_P9 high byte */
00358 } bmp280_calib_idx_t;
00359
00360 /**
00361  * @enum bmp280_calib_shift_t
00362  * @brief Bit shift for little-endian 16-bit calibration coefficient assembly
00363  *
00364  * @details
00365  * Calibration coefficients are stored little-endian (LSB first). The MSB
00366  * must be shifted left by 8 bits to form a 16-bit value.
00367  *
00368  * @since Version 1.0.0
00369  */
00370 typedef enum : uint8_t {
00371     k_bmp280_byte_shift = 8, /**< Shift MSB byte left 8 bits for 16-bit assembly */
00372 } bmp280_calib_shift_t;
00373
00374 /**
00375  * @enum bmp280_write_size_t
00376  * @brief I2C write/read buffer size constants for BMP280
00377  *
00378  * @details
00379  * BMP280 register write sends [register_address, data_byte] as 2 bytes.
00380  * Size constants for determining transaction lengths.
00381  *
00382  * @invariant k_bmp280_write_buf_size == k_bmp280_write_idx_val + 1 (buffer exactly fits register + value)
00383  *
00384  * @since Version 1.0.0
00385  */
00386 typedef enum : uint8_t {
00387     k_bmp280_write_buf_size = 2, /**< Size of [reg, val] write buffer */

```



```

00388 k_bmp280_read_cmd_size = 1, /**< Size of register address read command */
00389 k_bmp280_single_byte    = 1, /**< Size for single byte reads */
00390 } bmp280_write_size_t;
00391
00392 /**
00393  * @enum bmp280_write_idx_t
00394  * @brief I2C write buffer index constants for BMP280 register writes
00395  *
00396  * @details
00397  * Index constants for accessing fields within the 2-byte I2C write buffer
00398  * [register_address, data_byte] used by internal_write_reg().
00399  *
00400  * @see bmp280_write_size_t Size constants for the same buffer
00401  *
00402  * @since Version 1.0.0
00403  */
00404 typedef enum : uint8_t {
00405     k_bmp280_write_idx_reg = 0, /**< Index of register address in write buffer */
00406     k_bmp280_write_idx_val = 1, /**< Index of register value in write buffer */
00407 } bmp280_write_idx_t;
00408
00409 #ifdef __cplusplus
00410 }
00411 #endif

```

5.5 libs/rx`bmp280/src/rx`bmp280.c File Reference

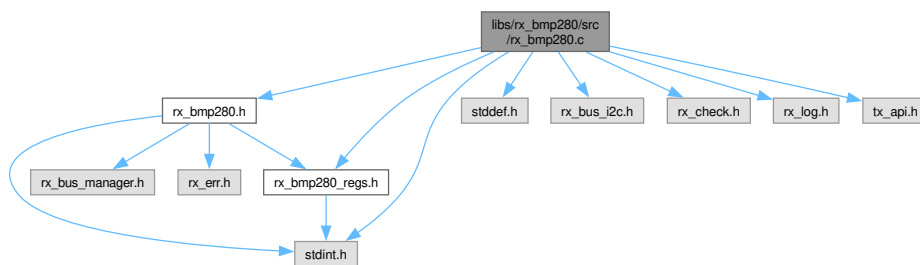
BMP280 Digital Barometric Pressure and Temperature Sensor Driver Implementation.

```

#include "rx_bmp280.h"
#include <stddef.h>
#include <stdint.h>
#include "rx_bmp280_regs.h"
#include "rx_bus_i2c.h"
#include "rx_check.h"
#include "rx_log.h"
#include "tx_api.h"

```

Include dependency graph for rx_bmp280.c:



Enumerations

- enum `bmp280_sizeof_t` : `uint8_t` { `k_bmp280_sizeof_int64` = 8 , `k_bmp280_calib_tail_bytes` = 2 }
Expected sizes for static assertions in this module.
- enum `bmp280_le16_idx_t` : `uint8_t` { `k_bmp280_le16_lsb_idx` = 0 , `k_bmp280_le16_msb_idx` = 1 }
Byte indices for little-endian 16-bit pair (LSB first).
- enum `bmp280_adc20_idx_t` : `uint8_t` { `k_bmp280_adc20_msb_idx` = 0 , `k_bmp280_adc20_lsb_idx` = 1 , `k_bmp280_adc20_xlsb_idx` = 2 }
Byte indices for 3-byte 20-bit ADC buffer (MSB first).
- enum `bmp280_adc20_range_t` : `uint32_t` { `k_bmp280_adc_20bit_max` = 0xFFFFFU }

Valid range for 20-bit ADC values assembled by `internal_assemble_adc20()`.

- enum `bmp280_press_range_pa_t` : `uint32_t` { `k_bmp280_press_min_pa` = 30000U , `k_bmp280_press_max_pa` = 110000U }

BMP280 valid pressure range in base Pascal units.

- enum `bmp280_temp_range_t` : `int32_t` { `k_bmp280_temp_min_cdegc_impl` = -4000 , `k_bmp280_temp_max_cdegc_impl` = 8500 }

BMP280 temperature output range limits for postcondition validation.

- enum `bmp280_press_range_t` : `uint32_t` { `k_bmp280_press_min_pa_256`, `k_bmp280_press_max_pa_256` }

BMP280 pressure output range limits for postcondition validation.

- enum `bmp280_temp_comp_t` : `int32_t` { `k_temp_shift_adc_3` = 3 , `k_temp_shift_t1_1` = 1 , `k_temp_shift_11` = 11 , `k_temp_shift_adc_4` = 4 , `k_temp_shift_12` = 12 , `k_temp_shift_14` = 14 , `k_temp_fine_scale` = 5 , `k_temp_round_add` = 128 , `k_temp_shift_out` = 8 }

Integer shift and scale constants for BMP280 temperature compensation formula.

- enum `press_comp_constants_t` : `int32_t` { `k_press_t_offset` = 128000 , `k_press_p_scale` = 1048576 , `k_press_shift_31` = 31 , `k_press_shift_47` = 47 , `k_press_shift_8` = 8 , `k_press_mul_3125` = 3125 , `k_press_shift_35` = 35 , `k_press_shift_25` = 25 , `k_press_shift_19` = 19 , `k_press_shift_13` = 13 , `k_press_shift_17` = 17 , `k_press_shift_12` = 12 , `k_press_shift_33` = 33 , `k_press_shift_4` = 4 }

Bosch BMP280 pressure compensation formula constants (datasheet appendix A).

Functions

- `RX_STATIC_TESTABLE rx_err_t internal_write_reg (uint8_t reg, uint8_t val)`
Write a single byte to a BMP280 register via I2C.
- `RX_STATIC_TESTABLE rx_err_t internal_read_regs (uint8_t reg, uint8_t *buf, uint8_t len)`
Burst-read consecutive registers from BMP280 via I2C write-read.
- `RX_STATIC_TESTABLE int32_t internal_compensate_temp (int32_t adc_temp, int32_t *t↔_fine_out)`
Apply Bosch integer temperature compensation formula (returns centi-degC, sets `t_fine`).
- `RX_STATIC_TESTABLE rx_err_t internal_compensate_pressure (int32_t adc_pressure, int32_t t_fine, uint32_t *press_out)`
Apply Bosch integer pressure compensation formula (returns Pa*256 in `press_out`).
- `RX_STATIC_TESTABLE uint16_t internal_parse_u16_le (const uint8_t *buf)`
Assemble an unsigned 16-bit little-endian value from two consecutive bytes.
- `RX_STATIC_TESTABLE int16_t internal_parse_s16_le (const uint8_t *buf)`
Assemble a signed 16-bit little-endian value from two consecutive bytes.
- `RX_STATIC_TESTABLE int32_t internal_assemble_adc20 (const uint8_t *buf)`
Assemble a 20-bit signed ADC value from a 3-byte MSB/LSB/XLSB buffer.
- `RX_STATIC_TESTABLE void internal_parse_calibration (const uint8_t *buf)`
Parse 24-byte OTP calibration block into `s_calib` coefficients.
- `RX_STATIC_TESTABLE uint32_t internal_finalize_pressure_q8 (int64_t p, int64_t var1, int64_t var2)`
Apply final Q8 fixed-point scaling and `dig_P7` offset to pressure intermediate.
- `RX_STATIC_TESTABLE rx_err_t internal_read_and_compensate_adc (bmp280_data_t *out)`
Read ADC registers, assemble values, apply compensation, and validate range.

- static rx_err_t [internal_verify_chip_id](#) (void)
Verify BMP280 chip ID register reads the expected value.
- static rx_err_t [internal_read_and_validate_calibration](#) (void)
Read, parse, and validate BMP280 factory calibration coefficients.
- rx_err_t [rx_bmp280_init](#) (rx_bus_manager_t *manager)
Initialize BMP280 sensor and read factory calibration coefficients.
- rx_err_t [rx_bmp280_read](#) (bmp280_data_t *out)
Trigger forced measurement and read compensated pressure and temperature.

Variables

- static const char *const [s_tag](#) = "BMP280"
Log tag for this module.
- static rx_bus_manager_t * [s_manager](#) = NULL
Bus manager pointer stored during [rx_bmp280_init\(\)](#).
- static [bmp280_calib_t](#) [s_calib](#) = {}
Factory calibration coefficients read from BMP280 OTP during init.
- static bool [s_initialized](#) = false
Guard flag: true after successful [rx_bmp280_init\(\)](#).
- static const char *const [s_bus_name](#) = "i2c1_baro"
I2C bus name for BMP280 (RIIC1, device addr 0x76, registered as "i2c1_baro" in main.c).

5.5.1 Detailed Description

BMP280 Digital Barometric Pressure and Temperature Sensor Driver Implementation.

5.5.2 Overview

Complete driver implementation for the Bosch BMP280 barometric pressure sensor connected via I2C on RIIC1. Implements the Bosch integer-only compensation algorithm (datasheet section 4.2.3) for accurate temperature and pressure calculation without floating-point operations.

5.5.3 Integer Compensation Algorithm

The Bosch compensation formulas use 64-bit integer arithmetic for pressure to maintain precision across the full operating range. The intermediate variable `t_fine` is computed from the temperature ADC value and reused in the pressure formula for temperature compensation.

Formula source: BMP280 datasheet v1.19, section 4.2.3 "Compensation formulas in double precision floating point" (Integer version provided in appendix A)

5.5.4 NASA Power of 10 Compliance

Rule	Status	Notes
1. No goto	↵ [PASS]↵	Structured if/while only
2. Bounded loops	↵ [PASS]↵	Status poll bounded by k_bmp280_poll_max
3. No dynamic memory	↵ [PASS]↵	All buffers on stack
4. Short functions	↵ [PASS]↵	Max function ~40 lines
5. Assertions	↵ [PASS]↵	2+ checks per function
6. Data scope	↵ [PASS]↵	Locals at point of use
7. Check returns	↵ [PASS]↵	All I2C returns validated
8. Limit preprocessor	↵ [PASS]↵	C23 typed enums only
9. Pointer restrictions	↵ [WARN]↵	bus_manager function pointers (DIP)
10. Compile warnings	↵ [PASS]↵	-Wall -Wextra -Werror

See also

[rx_bmp280.h](#) Public API

[rx_bmp280_regs.h](#) Register and constant definitions

Author

STAR Team

Date

2026-03-04

Since

Version 1.0.0

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_bmp280.c](#).

5.5.5 Enumeration Type Documentation

5.5.5.1 bmp280`adc20`idx` t

enum [bmp280_adc20_idx_t](#) : uint8_t

Byte indices for 3-byte 20-bit ADC buffer (MSB first).

Index constants for accessing MSB, LSB, and XLSB within a 3-byte ADC buffer. Used by [internal_assemble_adc20\(\)](#).

Since

Version 1.0.0

Enumerator

k_bmp280_adc20_msb_idx	Index of MSB byte in 3-byte ADC buffer
k_bmp280_adc20_lsb_idx	Index of LSB byte in 3-byte ADC buffer
k_bmp280_adc20_xlsb_idx	Index of XLSB byte in 3-byte ADC buffer

Definition at line 104 of file [rx_bmp280.c](#).

```
00104      : uint8_t {
00105  k_bmp280_adc20_msb_idx = 0, /**< Index of MSB byte in 3-byte ADC buffer */
00106  k_bmp280_adc20_lsb_idx = 1, /**< Index of LSB byte in 3-byte ADC buffer */
00107  k_bmp280_adc20_xlsb_idx = 2, /**< Index of XLSB byte in 3-byte ADC buffer */
00108 } bmp280_adc20_idx_t;
```

5.5.5.2 bmp280`adc20`range` t

enum [bmp280_adc20_range_t](#) : uint32_t

Valid range for 20-bit ADC values assembled by [internal_assemble_adc20\(\)](#).

The BMP280 ADC output is a 20-bit non-negative integer. Maximum value is $2^{20} - 1 = 0xFFFFF = 1048575$. Used as a postcondition bound in [internal_assemble_adc20\(\)](#).

Since

Version 1.0.0

Enumerator

k_bmp280_adc_20bit_max	Maximum valid 20-bit ADC value ($2^{20} - 1 = 1048575$)
------------------------	---

Definition at line 121 of file [rx_bmp280.c](#).

```
00121      : uint32_t {
00122  k_bmp280_adc_20bit_max = 0xFFFFFU, /**< Maximum valid 20-bit ADC value ( $2^{20} - 1 = 1048575$ ) */
00123 } bmp280_adc20_range_t;
```

5.5.5.3 bmp280`le16`idx` t

enum [bmp280_le16_idx_t](#) : uint8_t

Byte indices for little-endian 16-bit pair (LSB first).

Index constants for accessing LSB and MSB within a 2-byte little-endian pair. Used by [internal_parse_u16_le\(\)](#) and [internal_parse_s16_le\(\)](#).

Since

Version 1.0.0

Enumerator

k_bmp280_le16_lsb_idx	Index of least-significant byte in a 2-byte LE pair
k_bmp280_le16_msb_idx	Index of most-significant byte in a 2-byte LE pair

Definition at line 89 of file [rx_bmp280.c](#).

```
00089      : uint8_t {
00090  k_bmp280_le16_lsb_idx = 0, /**< Index of least-significant byte in a 2-byte LE pair */
00091  k_bmp280_le16_msb_idx = 1, /**< Index of most-significant byte in a 2-byte LE pair */
00092 } bmp280_le16_idx_t;
```

5.5.5.4 bmp280`press`range`pa` t

enum [bmp280_press_range_pa_t](#) : uint32_t

BMP280 valid pressure range in base Pascal units.

Valid pressure limits per the BMP280 datasheet operating specifications:

- 300 hPa = 30000 Pa (minimum, e.g. high altitude)
- 1100 hPa = 110000 Pa (maximum, e.g. sea level)

These base-Pa values are multiplied by 256 for the fixed-point Pa*256 format used by the Bosch integer compensation formula output. See [bmp280_press_range_t](#) for the Pa*256 scaled limits.

See also

[bmp280_press_range_t](#) Pa*256 scaled limits used in postcondition check

Since

Version 1.0.0

Enumerator

k_bmp280_press_min_pa	Minimum valid pressure: 300 hPa in Pa
-----------------------	---------------------------------------

<code>k_bmp280_press_max_pa</code>	Maximum valid pressure: 1100 hPa in Pa
------------------------------------	--

Definition at line 142 of file `rx_bmp280.c`.

```
00142      : uint32_t {
00143  k_bmp280_press_min_pa = 30000U, /**< Minimum valid pressure: 300 hPa in Pa */
00144  k_bmp280_press_max_pa = 110000U, /**< Maximum valid pressure: 1100 hPa in Pa */
00145 } bmp280_press_range_pa_t;
```

5.5.5.5 `bmp280`press`range`_t`

enum `bmp280_press_range_t` : `uint32_t`

BMP280 pressure output range limits for postcondition validation.

Valid pressure output range per the BMP280 datasheet operating specifications:

- Pressure: 300 hPa to 1100 hPa (in Pa*256 fixed-point units) Derivation: $k_bmp280_press_min_pa * 256 = 30000 * 256 = 7680000$ $k_bmp280_press_max_pa * 256 = 110000 * 256 = 28160000$

See also

[bmp280_temp_range_t](#) Temperature output range limits

[bmp280_press_range_pa_t](#) Base Pa constants from which Pa*256 values are derived

[rx_bmp280_read\(\)](#) Applies these range checks after compensation

Since

Version 1.0.0

Enumerator

<code>k_bmp280_press_min_pa_256</code>	Minimum valid pressure: 300 hPa * 256 (<code>k_bmp280_press_min_pa * 256</code>)
<code>k_bmp280_press_max_pa_256</code>	Maximum valid pressure: 1100 hPa * 256 (<code>k_bmp280_press_max_pa * 256</code>)

Definition at line 181 of file `rx_bmp280.c`.

```
00181      : uint32_t {
00182  k_bmp280_press_min_pa_256 =
00183  7680000U, /**< Minimum valid pressure: 300 hPa * 256 (k_bmp280_press_min_pa * 256) */
00184  k_bmp280_press_max_pa_256 =
00185  28160000U, /**< Maximum valid pressure: 1100 hPa * 256 (k_bmp280_press_max_pa * 256) */
00186 } bmp280_press_range_t;
```

5.5.5.6 bmp280'sizeof't

enum [bmp280_sizeof_t](#) : uint8_t

Expected sizes for static assertions in this module.

Named constants used in static_assert comparisons to avoid magic numbers.

Since

Version 1.0.0

Enumerator

k_bmp280_sizeof_int64	Expected sizeof(int64_t) for overflow-safe pressure math
k_bmp280_calib_tail_bytes	Number of bytes occupied by the final coefficient (P9)

Definition at line 74 of file [rx_bmp280.c](#).

```
00074      : uint8_t {
00075  k_bmp280_sizeof_int64    = 8, /**< Expected sizeof(int64_t) for overflow-safe pressure math */
00076  k_bmp280_calib_tail_bytes = 2, /**< Number of bytes occupied by the final coefficient (P9) */
00077 } bmp280_sizeof_t;
```

5.5.5.7 bmp280'temp'comp't

enum [bmp280_temp_comp_t](#) : int32_t

Integer shift and scale constants for BMP280 temperature compensation formula.

Constants used exclusively by [internal_compensate_temp\(\)](#). Placed at module scope to avoid -Werror=unused-local-typedefs when the GNURX cross-compiler sees the typedef name unused inside the function body.

Since

Version 1.0.0

Enumerator

k_temp_shift_adc_3	adc_T right shift for T1 subtraction step
k_temp_shift_t1_1	dig_T1 left shift in first var1 step
k_temp_shift_11	Right shift for var1 final step
k_temp_shift_adc_4	adc_T right shift for T1 comparison steps
k_temp_shift_12	Right shift for squared difference
k_temp_shift_14	Right shift for var2 final step
k_temp_fine_scale	Scale factor in fine-to-output conversion
k_temp_round_add	Rounding constant in fine-to-output conversion

<code>k_temp_shift_out</code>	Right shift to produce 0.01 degC output
-------------------------------	---

Definition at line 452 of file [rx_bmp280.c](#).

```

00452         : int32_t {
00453     k_temp_shift_adc_3 = 3,  /**< adc_T right shift for T1 subtraction step */
00454     k_temp_shift_t1_1 = 1,  /**< dig_T1 left shift in first var1 step */
00455     k_temp_shift_11 = 11,  /**< Right shift for var1 final step */
00456     k_temp_shift_adc_4 = 4,  /**< adc_T right shift for T1 comparison steps */
00457     k_temp_shift_12 = 12,  /**< Right shift for squared difference */
00458     k_temp_shift_14 = 14,  /**< Right shift for var2 final step */
00459     k_temp_fine_scale = 5,  /**< Scale factor in fine-to-output conversion */
00460     k_temp_round_add = 128, /**< Rounding constant in fine-to-output conversion */
00461     k_temp_shift_out = 8,  /**< Right shift to produce 0.01 degC output */
00462 } bmp280_temp_comp_t;

```

5.5.5.8 bmp280`temp`range``t`

enum [bmp280_temp_range_t](#) : int32_t

BMP280 temperature output range limits for postcondition validation.

Valid temperature output range per the BMP280 datasheet operating specifications:

- Temperature: -40.00 degC to +85.00 degC (in centi-degC units)

See also

[bmp280_press_range_t](#) Pressure output range limits

[rx_bmp280_read\(\)](#) Applies these range checks after compensation

Since

Version 1.0.0

Enumerator

<code>k_bmp280_temp_min_cdegc_impl</code>	Minimum valid temperature: -40.00 degC in centi-degC
<code>k_bmp280_temp_max_cdegc_impl</code>	Maximum valid temperature: +85.00 degC in centi-degC

Definition at line 160 of file [rx_bmp280.c](#).

```

00160         : int32_t {
00161     k_bmp280_temp_min_cdegc_impl = -4000, /**< Minimum valid temperature: -40.00 degC in centi-degC */
00162     k_bmp280_temp_max_cdegc_impl = 8500,  /**< Maximum valid temperature: +85.00 degC in centi-degC */
00163 } bmp280_temp_range_t;

```

5.5.5.9 press_comp_constants_t

```
enum press\_comp\_constants\_t : int32_t
```

Bosch BMP280 pressure compensation formula constants (datasheet appendix A).

Integer shift and scale constants used exclusively by [internal_compensate_pressure\(\)](#) and [internal_finalize_pressure_q8\(\)](#). Placed at module scope so both functions can reference the same named constants without redundancy.

Since

Version 1.0.0

Enumerator

k_press_t_offset	t_fine offset in pressure formula
k_press_p_scale	ADC scaling constant
k_press_shift_31	Shift for 31-bit alignment
k_press_shift_47	Shift for 47-bit alignment
k_press_shift_8	Output shift for Q8 fixed-point Pa*256
k_press_mul_3125	Bosch formula scaling factor
k_press_shift_35	Shift for 35-bit intermediate
k_press_shift_25	Shift for 25-bit intermediate
k_press_shift_19	Shift for 19-bit intermediate
k_press_shift_13	Shift for P9 coefficient shift
k_press_shift_17	Shift for P5 coefficient step
k_press_shift_12	Right shift for dig_P3 step
k_press_shift_33	Divisor shift for var1
k_press_shift_4	Shift for P7 addition

Definition at line 531 of file [rx_bmp280.c](#).

```
00531      : int32_t {
00532  k_press_t_offset = 128000, /**< t_fine offset in pressure formula */
00533  k_press_p_scale  = 1048576, /**< ADC scaling constant */
00534  k_press_shift_31 = 31,     /**< Shift for 31-bit alignment */
00535  k_press_shift_47 = 47,     /**< Shift for 47-bit alignment */
00536  k_press_shift_8  = 8,      /**< Output shift for Q8 fixed-point Pa*256 */
00537  k_press_mul_3125 = 3125,   /**< Bosch formula scaling factor */
00538  k_press_shift_35 = 35,     /**< Shift for 35-bit intermediate */
00539  k_press_shift_25 = 25,     /**< Shift for 25-bit intermediate */
00540  k_press_shift_19 = 19,     /**< Shift for 19-bit intermediate */
00541  k_press_shift_13 = 13,     /**< Shift for P9 coefficient shift */
00542  k_press_shift_17 = 17,     /**< Shift for P5 coefficient step */
00543  k_press_shift_12 = 12,     /**< Right shift for dig_P3 step */
00544  k_press_shift_33 = 33,     /**< Divisor shift for var1 */
00545  k_press_shift_4  = 4,      /**< Shift for P7 addition */
00546 } press_comp_constants_t;
```

5.5.6 Function Documentation

5.5.6.1 internal_assemble_adc20()

```
RX_STATIC_TESTABLE int32_t internal_assemble_adc20 (
    const uint8_t * buf)
```

Assemble a 20-bit signed ADC value from a 3-byte MSB/LSB/XLSB buffer.

The BMP280 outputs pressure and temperature as 20-bit raw values stored in three consecutive bytes with the following layout:

- buf[0] (MSB): bits[19:12] of the ADC value
- buf[1] (LSB): bits[11:4] of the ADC value
- buf[2] (XLSB): bits[3:0] of the ADC value in the upper nibble (bits [7:4])

Assembly formula: $(\text{MSB} \ll 12) \mid (\text{LSB} \ll 4) \mid (\text{XLSB} \gg 4)$

Precondition

buf non-NULL, capacity ≥ 3

Postcondition

Return value is a valid 20-bit non-negative integer

Note

Inline; zero overhead after optimization

See also

[internal_compensate_temp\(\)](#) Consumes the assembled temperature ADC value

[internal_compensate_pressure\(\)](#) Consumes the assembled pressure ADC value

Since

Version 1.0.0

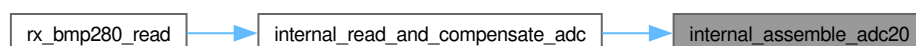
Definition at line 418 of file [rx_bmp280.c](#).

```
00419 {
00420     /* Pre-condition: buf is a 6-byte ADC read buffer (never NULL).
00421      * Post-condition: result is in [0, 0xFFFFF] because only 20 bits are
00422      * assembled (MSB contributes bits[19:12], LSB bits[11:4], XLSB bits[3:0]). */
00423     RX_ASSERT_PRE(buf != NULL, "buf must not be NULL");
00424     #ifndef UNIT_TEST
00425     if (buf == NULL) {
00426         return 0;
00427     }
00428     #endif
00429     /* Assembly: (MSB « 12) | (LSB « 4) | (XLSB » 4) produces 20-bit value */
00430     const int32_t result =
00431         (int32_t)((uint32_t)buf[k_bmp280_adc20_msb_idx] « k_bmp280_shift_msb) |
00432         ((uint32_t)buf[k_bmp280_adc20_lsb_idx] « k_bmp280_shift_lsb_left) |
00433         ((uint32_t)buf[k_bmp280_adc20_xlsb_idx] » k_bmp280_shift_xlsb_right));
00434     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(20-bit mask guarantees range) */
00435     RX_ASSERT_POST(result >= 0 && result <= (int32_t)k_bmp280_adc_20bit_max,
00436         "ADC20 result out of 20-bit range");
00437     /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00438     return result;
00439 }
```

References [k_bmp280_adc20_lsb_idx](#), [k_bmp280_adc20_msb_idx](#), [k_bmp280_adc20_xlsb_idx](#), [k_bmp280_adc_20bit_max](#), [k_bmp280_shift_lsb_left](#), [k_bmp280_shift_msb](#), and [k_bmp280_shift_xlsb_right](#).

Referenced by [internal_read_and_compensate_adc\(\)](#).

Here is the caller graph for this function:



5.5.6.2 `internal_compensate_pressure()`

```
RX_STATIC_TESTABLE rx_err_t internal_compensate_pressure (
    int32_t adc_pressure,
    int32_t t_fine,
    uint32_t * press_out)
```

Apply Bosch integer pressure compensation formula (returns Pa*256 in `press_out`).

Apply Bosch integer pressure compensation formula.

Implements the Bosch BMP280 integer pressure compensation algorithm verbatim from datasheet v1.19 appendix A. Uses `t_fine` from temperature compensation and trimming parameters from `s_calib`.

Returns `k_rx_err_invalid_state` if the internal `var1` divisor would be zero (prevents division by zero as required by NASA Power of 10 Rule 7).

Output unit: Pa * 256 (fixed-point Q8.0 format). Divide by 256.0 for Pa.

Precondition

- `adc_P` is valid 20-bit ADC output from BMP280
- `t_fine` was computed by [internal_compensate_temp\(\)](#) for same measurement
- `press_out` non-NULL

Postcondition

- `*press_out` in range [7680000, 28160000] (300*256 hPa to 1100*256 hPa, Pa*256) on success
- Returns `k_rx_err_invalid_state` without modifying `*press_out` when `var1 == 0`

Note

- Uses 64-bit arithmetic to prevent overflow at intermediate values

See also

- BMP280 datasheet v1.19 appendix A, `compensate_P_int64`

Since

- Version 1.0.0

Definition at line 613 of file [rx_bmp280.c](#).

```
00616 {
00617  /* Pre-conditions: dig_P1 non-zero is validated by rx_bmp280_init before any read is possible;
00618   * adc_pressure is a 20-bit value from internal_assemble_adc20 (always >= 0);
00619   * press_out is a local stack variable passed by internal_read_and_compensate_adc (never NULL). */
00620  RX_ASSERT_PRE(s_calib.dig_P1 != 0U, "dig_P1 must be non-zero (factory calibration)");
00621  RX_ASSERT_PRE(adc_pressure >= 0, "adc_P must be non-negative (20-bit unsigned ADC value)");
00622  RX_ASSERT_PRE(press_out != NULL, "press_out must not be NULL");
00623  #ifdef UNIT_TEST
00624      if (press_out == NULL) {
00625          return k_rx_err_null_ptr;
00626      }
00627  #endif
00628
00629  /* k_press_base_one must be int64_t (not enum) because the expression
00630   * (k_press_base_one « k_press_shift_47) requires a 48-bit value.
```

```

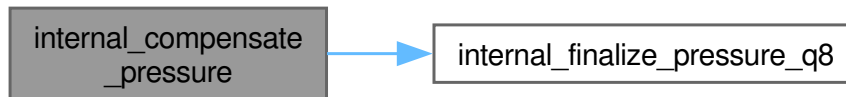
00631  * A 32-bit integer cannot hold (1 « 47); int64_t is required to
00632  * prevent undefined behavior from shifting beyond the type width.
00633  * C23 typed enums cannot have int64_t as underlying type on RX72N. */
00634  static const int64_t k_press_base_one = 1;
00635
00636  int64_t var1 = ((int64_t)t_fine) - k_press_t_offset;
00637  int64_t var2 = var1 * var1 * (int64_t)s_calib.dig_P6;
00638  var2 += (var1 * (int64_t)s_calib.dig_P5) « k_press_shift_17;
00639  var2 += (((int64_t)s_calib.dig_P4) « k_press_shift_35;
00640  var1 = ((var1 * var1 * (int64_t)s_calib.dig_P3) » k_press_shift_8) +
00641  ((var1 * (int64_t)s_calib.dig_P2) « k_press_shift_12);
00642  var1 = (((k_press_base_one « k_press_shift_47) + var1)) * ((int64_t)s_calib.dig_P1) »
00643  k_press_shift_33;
00644
00645  /* Division by zero guard (NASA Power of 10 Rule 7) */
00646  if (var1 == 0) {
00647      rx_log_error(s_tag, "Pressure compensation: var1==0 (dig_P1 fault)");
00648      return k_rx_err_invalid_state;
00649  }
00650
00651  int64_t p = (int64_t)k_press_p_scale - adc_pressure;
00652  p = (((p « k_press_shift_31) - var2) * k_press_mul_3125) / var1;
00653
00654  var1 = (((int64_t)s_calib.dig_P9) * (p » k_press_shift_13) * (p » k_press_shift_13)) »
00655  k_press_shift_25;
00656  var2 = (((int64_t)s_calib.dig_P8) * p) » k_press_shift_19;
00657
00658  *press_out = internal_finalize_pressure_q8(p, var1, var2);
00659  return k_rx_ok;
00660 }

```

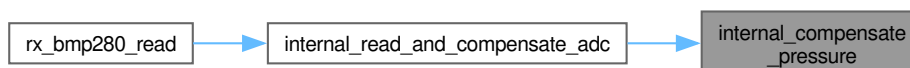
References [internal_finalize_pressure_q8\(\)](#), [k_press_mul_3125](#), [k_press_p_scale](#), [k_press_shift_12](#), [k_press_shift_13](#), [k_press_shift_17](#), [k_press_shift_19](#), [k_press_shift_25](#), [k_press_shift_31](#), [k_press_shift_33](#), [k_press_shift_35](#), [k_press_shift_47](#), [k_press_shift_8](#), [k_press_t_offset](#), [s_calib](#), and [s_tag](#).

Referenced by [internal_read_and_compensate_adc\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.5.6.3 internal_compensate_temp()

```

RX_STATIC_TESTABLE int32_t internal_compensate_temp (
    int32_t adc_temp,
    int32_t * t_fine_out)

```

Apply Bosch integer temperature compensation formula (returns centi-degC, sets `t_fine`).

Apply Bosch integer temperature compensation formula.

Implements the Bosch BMP280 integer temperature compensation algorithm verbatim from datasheet v1.19 appendix A. Uses trimming parameters from `s_calib` and computes the fine temperature value `t_fine` which is required for pressure compensation.

Algorithm:

```

var1 = (((adc_T » 3) - (dig_T1 « 1)) * dig_T2) » 11
var2 = (((adc_T » 4) - dig_T1) * ((adc_T » 4) - dig_T1)) » 12 * dig_T3 » 14
t_fine = var1 + var2
T = (t_fine * 5 + 128) » 8 // Unit: centi-degrees (0.01 degC)

```

Precondition

adc_T is valid 20-bit ADC output from BMP280
t_fine_out non-NULL

Postcondition

*t_fine_out set to intermediate temperature value
Return value in range [-4000, 8500] for valid operating range

Note

Pure integer arithmetic, no floating-point

See also

BMP280 datasheet v1.19 appendix A, compensate_T_int32

Since

Version 1.0.0

Definition at line 493 of file [rx_bmp280.c](#).

```

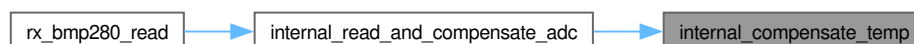
00494 {
00495     /* Pre-conditions: t_fine_out is a local stack variable (never NULL);
00496      * adc_temp is produced by internal_assemble_adc20 which masks to 20 bits. */
00497     RX_ASSERT_PRE(t_fine_out != NULL, "t_fine_out must be non-NULL");
00498     #ifndef UNIT_TEST
00499     if (t_fine_out == NULL) {
00500         return 0;
00501     }
00502     #endif
00503     RX_ASSERT_PRE(adc_temp >= 0 && adc_temp <= (int32_t)k_bmp280_adc_20bit_max,
00504                  "adc_T must be a 20-bit ADC value");
00505     const int32_t var1 =
00506         (((adc_temp » k_temp_shift_adc_3) - ((int32_t)s_calib.dig_T1 « k_temp_shift_t1_1))) *
00507         ((int32_t)s_calib.dig_T2)) »
00508         k_temp_shift_11;
00509     const int32_t var2 = (((((adc_temp » k_temp_shift_adc_4) - ((int32_t)s_calib.dig_T1)) *
00510                             ((adc_temp » k_temp_shift_adc_4) - ((int32_t)s_calib.dig_T1))) »
00511                             k_temp_shift_12) *
00512                             ((int32_t)s_calib.dig_T3)) »
00513         k_temp_shift_14;
00514
00515     *t_fine_out = var1 + var2;
00516
00517     return (*t_fine_out * k_temp_fine_scale + k_temp_round_add) » k_temp_shift_out;
00518 }

```

References [k_bmp280_adc_20bit_max](#), [k_temp_fine_scale](#), [k_temp_round_add](#), [k_temp_shift_11](#), [k_temp_shift_12](#), [k_temp_shift_14](#), [k_temp_shift_adc_3](#), [k_temp_shift_adc_4](#), [k_temp_shift_out](#), [k_temp_shift_t1_1](#), and [s_calib](#).

Referenced by [internal_read_and_compensate_adc\(\)](#).

Here is the caller graph for this function:



5.5.6.4 internal`finalize`pressure`q8()

```
RX_STATIC_TESTABLE uint32_t internal_finalize_pressure_q8 (
    int64_t p,
    int64_t var1,
    int64_t var2)
```

Apply final Q8 fixed-point scaling and dig_P7 offset to pressure intermediate.

Implements the last step of the Bosch BMP280 pressure compensation formula (datasheet Section 4.2.3): right-shifts the accumulated intermediate value to produce Pa*256 (Q8 fixed-point), then adds the dig_P7 offset term.

Extracted from [internal_compensate_pressure\(\)](#) to keep that function within the 60-line NASA Power of 10 Rule 4 guideline.

Precondition

s_calib populated by [internal_parse_calibration\(\)](#) (dig_P1 != 0)

Postcondition

Return value is pressure * 256 in Pascal units (Q8 format)

Note

Accesses s_calib.dig_P7 directly (module-level static)

See also

[internal_compensate_pressure\(\)](#) Sole caller

Since

Version 1.0.0

Definition at line 569 of file [rx_bmp280.c](#).

```
00570 {
00571  /* Pre-conditions: dig_P1 non-zero is guaranteed by the rx_bmp280_init post-condition check;
00572   * p is non-negative by construction from the BMP280 datasheet algorithm (var1 > 0 guard). */
00573  RX_ASSERT_PRE(s_calib.dig_P1 != 0U, "s_calib must be initialized before finalizing pressure");
00574  RX_ASSERT_PRE(p >= 0, "p must be non-negative before final Q8 scaling");
00575  static_assert((bool)((sizeof(int64_t) == k_bmp280_sizeof_int64) != 0),
00576               "int64_t must be 64-bit for overflow-safe pressure math");
00577  /* Bosch BMP280 datasheet formula (Section 4.2.3):
00578   * Step 1: shift (p + fine adjustments) right by 8 to get Q8 fixed-point Pa*256
00579   * Step 2: add dig_P7 offset (left-shifted 4) AFTER the right shift
00580   * Note: shifting the dig_P7 term together with the rest would incorrectly
00581   *       scale it by an additional factor of 256. */
00582  const int64_t p_scaled = (p + var1 + var2) » k_press_shift_8;
00583  const int64_t dig_p7   = ((int64_t)s_calib.dig_P7) « k_press_shift_4;
00584  return (uint32_t)(p_scaled + dig_p7);
00585 }
```

References [k_bmp280_sizeof_int64](#), [k_press_shift_4](#), [k_press_shift_8](#), and [s_calib](#).

Referenced by [internal_compensate_pressure\(\)](#).

Here is the caller graph for this function:



5.5.6.5 internal_parse_calibration()

```
RX_STATIC_TESTABLE void internal_parse_calibration (
    const uint8_t * buf)
```

Parse 24-byte OTP calibration block into s_calib coefficients.

Parses the 24-byte factory-calibrated trimming parameters read from OTP registers k_bmp280_reg_calib_start..k_bmp280_reg_calib_end (0x88-0x9F). All coefficients are stored little-endian (LSB first) in the OTP block.

Precondition

buf non-NULL

buf contains exactly k_bmp280_calib_byte_count bytes from OTP registers

Postcondition

s_calib.dig_T1..T3 and dig_P1..P9 populated from buf

Caller must verify dig_T1 and dig_P1 are non-zero before use

Note

dig_T1 and dig_P1 may still be zero if OTP is blank; caller validates

See also

[rx_bmp280_init\(\)](#) Sole caller; validates non-zero after this returns

Since

Version 1.0.0

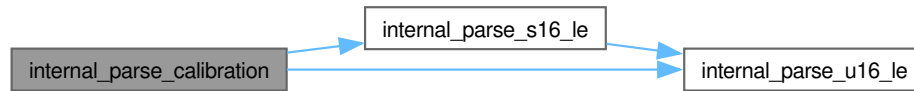
Definition at line 686 of file [rx_bmp280.c](#).

```
00687 {
00688     /* Pre-condition: buf is a stack-allocated calibration buffer in rx_bmp280_init (never NULL). */
00689     RX_ASSERT_PRE(buf != NULL, "calibration buffer must not be NULL");
00690     #ifdef UNIT_TEST
00691     if (buf == NULL) {
00692         return;
00693     }
00694     #endif
00695     static_assert(
00696         (bool)((k_bmp280_calib_p9_lsb + k_bmp280_calib_tail_bytes == k_bmp280_calib_byte_count) != 0),
00697         "P9 coefficient must occupy the final two bytes of the calibration block");
00698     s_calib.dig_T1 = internal_parse_u16_le(&buf[k_bmp280_calib_t1_lsb]);
00699     s_calib.dig_T2 = internal_parse_s16_le(&buf[k_bmp280_calib_t2_lsb]);
00700     s_calib.dig_T3 = internal_parse_s16_le(&buf[k_bmp280_calib_t3_lsb]);
00701     s_calib.dig_P1 = internal_parse_u16_le(&buf[k_bmp280_calib_p1_lsb]);
00702     s_calib.dig_P2 = internal_parse_s16_le(&buf[k_bmp280_calib_p2_lsb]);
00703     s_calib.dig_P3 = internal_parse_s16_le(&buf[k_bmp280_calib_p3_lsb]);
00704     s_calib.dig_P4 = internal_parse_s16_le(&buf[k_bmp280_calib_p4_lsb]);
00705     s_calib.dig_P5 = internal_parse_s16_le(&buf[k_bmp280_calib_p5_lsb]);
00706     s_calib.dig_P6 = internal_parse_s16_le(&buf[k_bmp280_calib_p6_lsb]);
00707     s_calib.dig_P7 = internal_parse_s16_le(&buf[k_bmp280_calib_p7_lsb]);
00708     s_calib.dig_P8 = internal_parse_s16_le(&buf[k_bmp280_calib_p8_lsb]);
00709     s_calib.dig_P9 = internal_parse_s16_le(&buf[k_bmp280_calib_p9_lsb]);
00710     /* Post-condition: caller (rx_bmp280_init) validates dig_T1 and dig_P1 are non-zero */
00711 }
```

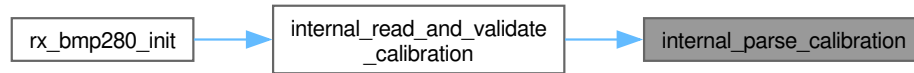
References [internal_parse_s16_le\(\)](#), [internal_parse_u16_le\(\)](#), [k_bmp280_calib_byte_count](#), [k_bmp280_calib_p1_lsb](#), [k_bmp280_calib_p2_lsb](#), [k_bmp280_calib_p3_lsb](#), [k_bmp280_calib_p4_lsb](#), [k_bmp280_calib_p5_lsb](#), [k_bmp280_calib_p6_lsb](#), [k_bmp280_calib_p7_lsb](#), [k_bmp280_calib_p8_lsb](#), [k_bmp280_calib_p9_lsb](#), [k_bmp280_calib_t1_lsb](#), [k_bmp280_calib_t2_lsb](#), [k_bmp280_calib_t3_lsb](#), [k_bmp280_calib_tail_bytes](#), and [s_calib](#).

Referenced by [internal_read_and_validate_calibration\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.5.6.6 internal`parse`s16`le()

```
RX_STATIC_TESTABLE int16_t internal_parse_s16_le (
    const uint8_t * buf)
```

Assemble a signed 16-bit little-endian value from two consecutive bytes.

Reads buf[0] (LSB) and buf[1] (MSB) and reinterprets the assembled uint16_t as a two's complement int16_t value. Matches BMP280 signed calibration coefficient layout.

Precondition

buf non-NULL, capacity ≥ 2

Postcondition

Return value correctly represents two's complement signed value

No side effects; buf and global state are not modified

Note

Inline; zero overhead after optimization

See also

[internal_parse_u16_le\(\)](#) Unsigned variant

Since

Version 1.0.0

Definition at line 381 of file [rx_bmp280.c](#).

```

00382 {
00383     /* Pre-condition: buf is always a valid calibration buffer pointer (never NULL). */
00384     RX_ASSERT_PRE(buf != NULL, "buf must not be NULL");
00385     #ifdef UNIT_TEST
00386     if (buf == NULL) {
00387         return 0;
00388     }
00389     #endif
00390     static_assert((bool)((sizeof(int16_t) == sizeof(uint16_t)) != 0),
00391         "int16_t and uint16_t must be same size for safe reinterpret cast");
00392     return (int16_t)internal_parse_u16_le(buf);
00393 }
```

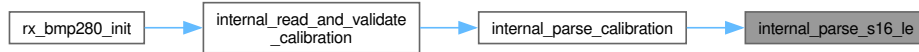
References [internal_parse_u16_le\(\)](#).

Referenced by [internal_parse_calibration\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.5.6.7 internal_parse_u16_le()

```

RX_STATIC_TESTABLE uint16_t internal_parse_u16_le (
    const uint8_t * buf)
  
```

Assemble an unsigned 16-bit little-endian value from two consecutive bytes.

Reads buf[0] (LSB) and buf[1] (MSB) and combines them into a uint16_t value using little-endian byte order (LSB first), matching BMP280 calibration register layout.

Precondition

buf non-NULL, capacity >= 2

Postcondition

Return value == (uint16_t)buf[0] | ((uint16_t)buf[1] << 8)

No side effects; buf and global state are not modified

Note

Inline; zero overhead after optimization

See also

[internal_parse_s16_le\(\)](#) Signed variant

Since

Version 1.0.0

Definition at line 347 of file [rx_bmp280.c](#).

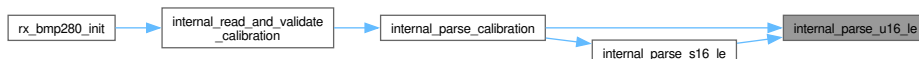
```

00348 {
00349     /* Pre-condition: buf is always a valid pointer to a calibration or ADC buffer;
00350      * all callers pass stack-allocated arrays (never NULL). */
00351     RX_ASSERT_PRE(buf != NULL, "buf must not be NULL");
00352     #ifndef UNIT_TEST
00353     if (buf == NULL) {
00354         return 0U;
00355     }
00356     #endif
00357     static_assert((bool)(((unsigned)k_bmp280_byte_shift == (unsigned)k_bmp280_sizeof_int64) != 0),
00358         "byte shift must be 8 for LE 16-bit assembly");
00359     return (uint16_t)((uint16_t)buf[k_bmp280_le16_lsb_idx] |
00360         ((uint16_t)buf[k_bmp280_le16_msb_idx] « k_bmp280_byte_shift));
00361 }
  
```

References [k_bmp280_byte_shift](#), [k_bmp280_le16_lsb_idx](#), [k_bmp280_le16_msb_idx](#), and [k_bmp280_sizeof_int64](#).

Referenced by [internal_parse_calibration\(\)](#), and [internal_parse_s16_le\(\)](#).

Here is the caller graph for this function:



5.5.6.8 `internal_read_and_compensate_adc()`

```
RX_STATIC_TESTABLE rx_err_t internal_read_and_compensate_adc (
    bmp280_data_t * out)
```

Read ADC registers, assemble values, apply compensation, and validate range.

Read ADC registers, assemble values, apply Bosch compensation, and validate range.

Performs the ADC read + compensation + range-check portion of the measurement cycle, after the forced-mode trigger and status poll have completed. Specifically:

1. Read 6 bytes of ADC data from `k_bmp280_reg_press_msb` (0xF7)
2. Assemble 20-bit pressure and temperature ADC values via `internal_assemble_adc20()`
3. Apply Bosch integer temperature compensation (`internal_compensate_temp()`)
4. Apply Bosch integer pressure compensation (`internal_compensate_pressure()`)
5. Validate temperature in [-40.00, +85.00] degC (centi-degC units)
6. Validate pressure in [300, 1100] hPa (Pa*256 units)

Precondition

`s_manager` non-NULL (`rx_bmp280_init` succeeded)
`s_initialized` == true
`out` non-NULL
 Forced measurement complete (status poll cleared by caller)

Postcondition

`out->temp_centi_degc` in [-4000, 8500] on `k_rx_ok`
`out->press_pa_256` in [7680000, 28160000] on `k_rx_ok`

Note

Not thread-safe

See also

`rx_bmp280_read()` Caller that triggers measurement and polls status before calling this
`internal_compensate_temp()` Temperature compensation algorithm
`internal_compensate_pressure()` Pressure compensation algorithm

Since

Version 1.0.0

Definition at line 875 of file [rx_bmp280.c](#).

```

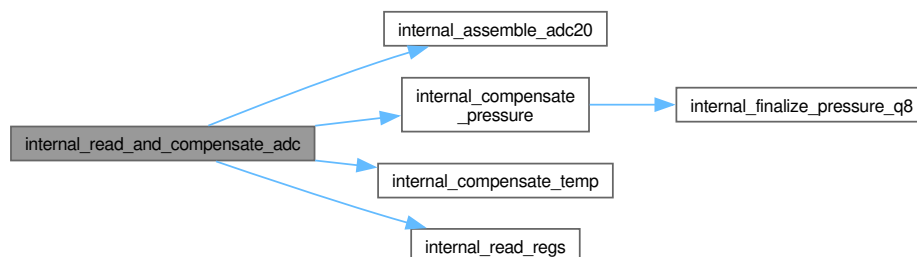
00876 {
00877     /* Pre-conditions: out is a local stack variable in rx_bmp280_read (never NULL);
00878      * s_initialized is true because rx_bmp280_read checks it via RX_CHECK_INITIALIZED. */
00879     RX_ASSERT_PRE(out != NULL, "out must be non-NULL");
00880     #ifndef UNIT_TEST
00881     if (out == NULL) {
00882         return k_rx_err_null_ptr;
00883     }
00884     #endif
00885     RX_ASSERT_PRE(s_initialized, "BMP280 must be initialized before ADC read");
00886
00887     /* Step 1: Read 6 bytes of ADC data: pressure[3] + temperature[3] */
00888     uint8_t adc_buf[k_bmp280_adc_buf_size];
00889     const rx_err_t err = internal_read_regs(k_bmp280_reg_press_msb, adc_buf, k_bmp280_adc_buf_size);
00890     if (err != k_rx_ok) {
00891         rx_log_error(s_tag, "ADC data read failed");
00892         return err;
00893     }
00894
00895     /* Step 2: Assemble 20-bit raw ADC values */
00896     const int32_t adc_pressure = internal_assemble_adc20(&adc_buf[k_bmp280_press_msb_idx]);
00897     const int32_t adc_temp = internal_assemble_adc20(&adc_buf[k_bmp280_temp_msb_idx]);
00898
00899     /* Step 3: Apply Bosch integer temperature compensation */
00900     int32_t t_fine = 0;
00901     out->temp_centi_degC = internal_compensate_temp(adc_temp, &t_fine);
00902
00903     /* Step 4: Apply Bosch integer pressure compensation */
00904     const rx_err_t press_err = internal_compensate_pressure(adc_pressure, t_fine, &out->press_pa_256);
00905     if (press_err != k_rx_ok) {
00906         rx_log_error(s_tag, "Pressure compensation failed (var1==0)");
00907         return press_err;
00908     }
00909
00910     /* Step 5: Postcondition range checks (BMP280 datasheet operating limits). */
00911     if ((out->temp_centi_degC < k_bmp280_temp_min_cdegC_impl) ||
00912         (out->temp_centi_degC > k_bmp280_temp_max_cdegC_impl)) {
00913         rx_log_error_val(s_tag, "Temperature out of range (centi-degC)", (int32_t)out->temp_centi_degC);
00914         return k_rx_err_invalid_state;
00915     }
00916
00917     if ((out->press_pa_256 < k_bmp280_press_min_pa_256) ||
00918         (out->press_pa_256 > k_bmp280_press_max_pa_256)) {
00919         rx_log_error_val(s_tag, "Pressure out of range (Pa*256)", out->press_pa_256);
00920         return k_rx_err_invalid_state;
00921     }
00922
00923     return k_rx_ok;
00924 }

```

References [internal_assemble_adc20\(\)](#), [internal_compensate_pressure\(\)](#), [internal_compensate_temp\(\)](#), [internal_read_regs\(\)](#), [k_bmp280_adc_buf_size](#), [k_bmp280_press_max_pa_256](#), [k_bmp280_press_min_pa_256](#), [k_bmp280_press_msb_idx](#), [k_bmp280_reg_press_msb](#), [k_bmp280_temp_max_cdegC_impl](#), [k_bmp280_temp_min_cdegC_impl](#), [k_bmp280_temp_msb_idx](#), [bmp280_data_t::press_pa_256](#), [s_initialized](#), [s_tag](#), and [bmp280_data_t::temp_centi_degC](#).

Referenced by [rx_bmp280_read\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.5.6.9 internal`read`and`validate`calibration()

```
rx_err_t internal_read_and_validate_calibration (
    void ) [static]
```

Read, parse, and validate BMP280 factory calibration coefficients.

Reads 24 bytes of factory-calibrated trimming parameters from OTP registers 0x88-0x9F, parses them into `s_calib`, and validates that critical coefficients (`dig_T1` and `dig_P1`) are non-zero. Clears `s_manager` on failure.

Precondition

`s_manager` non-NULL
 "i2c1_baro" bus initialized

Postcondition

`s_calib` populated with valid coefficients on `k_rx_ok`
`s_manager` set to NULL on failure

Note

Not thread-safe

See also

[rx_bmp280_init\(\)](#) Sole caller

Since

Version 1.0.0

Definition at line 772 of file [rx_bmp280.c](#).

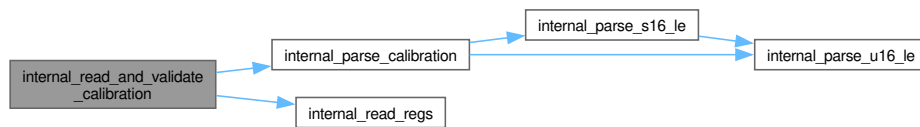
```

00773 {
00774     uint8_t      calib_buf[k_bmp280_calib_byte_count];
00775     const rx_err_t err =
00776         internal_read_regs(k_bmp280_reg_calib_start, calib_buf, k_bmp280_calib_byte_count);
00777     if (err != k_rx_ok) {
00778         rx_log_error(s_tag, "Calibration read failed");
00779         s_manager = NULL;
00780         return err;
00781     }
00782
00783     internal_parse_calibration(calib_buf);
00784
00785     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(dig_T1 short-circuit: tested via dig_P1==0 path) */
00786     if ((s_calib.dig_T1 == 0U) || (s_calib.dig_P1 == 0U)) {
00787         /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00788         rx_log_error(s_tag, "Invalid calibration: dig_T1 or dig_P1 is zero");
00789         s_manager = NULL;
00790         return k_rx_err_invalid_state;
00791     }
00792     return k_rx_ok;
00793 }
```

References [internal_parse_calibration\(\)](#), [internal_read_regs\(\)](#), [k_bmp280_calib_byte_count](#), [k_bmp280_reg_calib_start](#), [s_calib](#), [s_manager](#), and [s_tag](#).

Referenced by [rx_bmp280_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.5.6.10 internal_read_regs()

```

RX_STATIC_TESTABLE rx_err_t internal_read_regs (
    uint8_t reg,
    uint8_t * buf,
    uint8_t len)

```

Burst-read consecutive registers from BMP280 via I2C write-read.

Issues a combined I2C write-read transaction: write starting register address, then read len bytes. BMP280 auto-increments register address.

Precondition

- s_manager non-NULL
- "i2c1_baro" bus initialized in s_manager
- buf capacity >= len

Postcondition

- buf[0..len-1] contain register data on k_rx_ok
- Bus transaction completes before function returns

Note

- Not thread-safe

See also

- rx_bus_i2c_write_read()

Since

Version 1.0.0

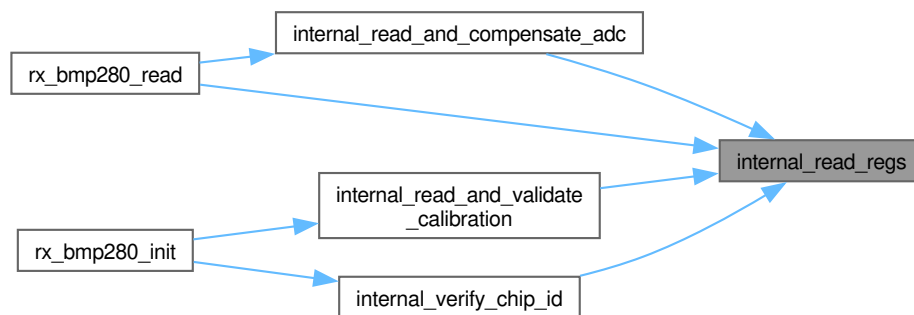
Definition at line 319 of file [rx_bmp280.c](#).

```
00320 {
00321  /* Pre-conditions: s_manager is non-NULL (only called after successful init),
00322   * buf is a local stack buffer (never NULL), len is an enum constant (always > 0). */
00323  RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before read");
00324  RX_ASSERT_PRE(buf != NULL, "buf must be non-NULL for read operation");
00325  RX_ASSERT_PRE(len > 0U, "len must be positive for read operation");
00326  return rx_bus_i2c_write_read(s_manager, s_bus_name, &reg, k_bmp280_read_cmd_size, buf, len);
00327 }
```

References [k_bmp280_read_cmd_size](#), [s_bus_name](#), and [s_manager](#).

Referenced by [internal_read_and_compensate_adc\(\)](#), [internal_read_and_validate_calibration\(\)](#), [internal_verify_chip_id\(\)](#), and [rx_bmp280_read\(\)](#).

Here is the caller graph for this function:



5.5.6.11 internal`verify`chip`id()

```
rx_err_t internal_verify_chip_id (
    void ) [static]
```

Verify BMP280 chip ID register reads the expected value.

Reads the chip ID register (0xD0) and compares it to `k_bmp280_chip_id_expected`. Clears `s_manager` on failure so the driver remains in a safe state.

Precondition

`s_manager` non-NULL
 "i2c1_baro" bus initialized

Postcondition

`s_manager` set to NULL on failure
 No state change on success

Note

Not thread-safe

See also

[rx_bmp280_init\(\)](#) Sole caller

Since

Version 1.0.0

Definition at line 736 of file [rx_bmp280.c](#).

```
00737 {
00738     uint8_t      chip_id = 0U;
00739     const rx_err_t err    = internal_read_regs(k_bmp280_reg_chip_id, &chip_id, k_bmp280_single_byte);
00740     if (err != k_rx_ok) {
00741         rx_log_error(s_tag, "Chip ID read failed");
00742         s_manager = NULL;
00743         return err;
00744     }
00745     if (chip_id != k_bmp280_chip_id_expected) {
00746         rx_log_error_val(s_tag, "Unexpected chip ID", (uint32_t)chip_id);
00747         s_manager = NULL;
00748         return k_rx_err_invalid_state;
00749     }
00750     return k_rx_ok;
00751 }
```

References [internal_read_regs\(\)](#), [k_bmp280_chip_id_expected](#), [k_bmp280_reg_chip_id](#), [k_bmp280_single_byte](#), [s_manager](#), and [s_tag](#).

Referenced by [rx_bmp280_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.5.6.12 internal_write_reg()

```
RX_STATIC_TESTABLE rx_err_t internal_write_reg (
    uint8_t reg,
    uint8_t val)
```

Write a single byte to a BMP280 register via I2C.

Sends a 2-byte I2C write transaction [reg, val] to the BMP280 at device address 0x76 (embedded in the "i2c1_baro" bus configuration).

Precondition

`s_manager` non-NULL (set by `rx_bmp280_init`)
 "i2c1_baro" bus initialized

Postcondition

Register contains val on k_rx_ok
 Bus transaction completes before function returns

Note

Not thread-safe

See also

`rx_bus_i2c_write()`

Since

Version 1.0.0

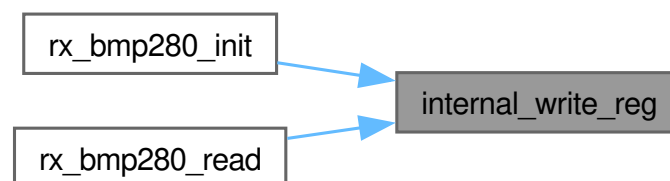
Definition at line 286 of file `rx_bmp280.c`.

```
00287 {
00288     /* Pre-conditions: s_manager and s_bus_name are non-NULL by construction;
00289      * internal_write_reg is only called after rx_bmp280_init stores a valid
00290      * manager pointer and s_bus_name is a compile-time string constant. */
00291     RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before write");
00292     RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL");
00293     uint8_t buf[k_bmp280_write_buf_size];
00294     buf[k_bmp280_write_idx_reg] = reg;
00295     buf[k_bmp280_write_idx_val] = val;
00296     return rx_bus_i2c_write(s_manager, s_bus_name, buf, k_bmp280_write_buf_size);
00297 }
```

References `k_bmp280_write_buf_size`, `k_bmp280_write_idx_reg`, `k_bmp280_write_idx_val`, `s_bus_name`, and `s_manager`.

Referenced by `rx_bmp280_init()`, and `rx_bmp280_read()`.

Here is the caller graph for this function:



5.5.6.13 rx`bmp280`init()

```
rx_err_t rx_bmp280_init (
    rx_bus_manager_t * manager) [nodiscard]
```

Initialize BMP280 sensor and read factory calibration coefficients.

Verifies chip ID, reads and validates 24 bytes of factory-calibrated trimming parameters from OTP registers, and configures the IIR filter by writing to the config register 0xF5.

Precondition

manager non-NULL, "i2c1_baro" registered and initialized
 BMP280 powered on RIIC1 bus

Postcondition

s_calib contains valid OTP coefficients
 s_initialized == true on success

Note

Not thread-safe

See also

[rx_bmp280_read\(\)](#) For triggered measurements

Since

Version 1.0.0

Definition at line 815 of file [rx_bmp280.c](#).

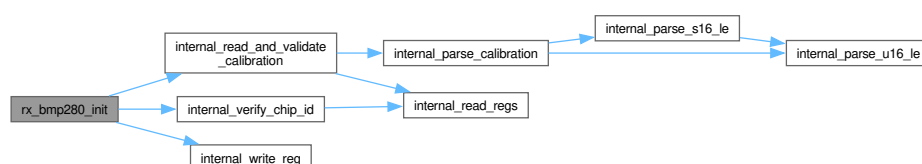
```

00816 {
00817     RX_CHECK_NULL_PTR(manager, s_tag, "Bus manager is NULL");
00818     RX_ASSERT_PRE(s_bus_name != NULL, "Bus name must not be NULL");
00819
00820     s_manager = manager;
00821     s_initialized = false;
00822
00823     rx_err_t err = internal_verify_chip_id();
00824     if (err != k_rx_ok) {
00825         return err;
00826     }
00827
00828     err = internal_read_and_validate_calibration();
00829     if (err != k_rx_ok) {
00830         return err;
00831     }
00832
00833     err = internal_write_reg(k_bmp280_reg_config, k_bmp280_config_val);
00834     if (err != k_rx_ok) {
00835         rx_log_error(s_tag, "Config register write failed");
00836         s_manager = NULL;
00837         return err;
00838     }
00839
00840     s_initialized = true;
00841     rx_log_info(s_tag, "BMP280 initialized, calibration loaded");
00842
00843     return k_rx_ok;
00844 }

```

References [internal_read_and_validate_calibration\(\)](#), [internal_verify_chip_id\(\)](#), [internal_write_reg\(\)](#), [k_bmp280_config_val](#), [k_bmp280_reg_config](#), [s_bus_name](#), [s_initialized](#), [s_manager](#), and [s_tag](#).

Here is the call graph for this function:



5.5.6.14 rx`bmp280`read()

```
rx_err_t rx_bmp280_read (
    bmp280_data_t * out) [nodiscard]
```

Trigger forced measurement and read compensated pressure and temperature.

Trigger a forced measurement and read compensated pressure and temperature.

Executes the complete forced-mode measurement cycle:

1. Trigger forced measurement (write ctrl_meas)
2. Poll status register for completion (bounded loop)
3. Read ADC data, apply compensation, validate range (internal_read_and_compensate_adc)

Precondition

rx_bmp280_init() succeeded
out non-NULL

Postcondition

out populated with compensated measurement on k_rx_ok
Sensor in sleep mode (forced measurement complete)

Note

Not thread-safe

See also

internal_read_and_compensate_adc() ADC read, compensation, and range validation

Since

Version 1.0.0

Definition at line 947 of file rx_bmp280.c.

```
00948 {
00949     RX_CHECK_NULL_PTR(out, s_tag, "Output data pointer is NULL");
00950
00951     if (!s_initialized) {
00952         return k_rx_err_not_initialized;
00953     }
00954
00955     /* Step 1: Write ctrl_meas to trigger forced measurement */
00956     rx_err_t err = internal_write_reg(k_bmp280_reg_ctrl_meas, k_bmp280_ctrl_meas_val);
00957     if (err != k_rx_ok) {
00958         rx_log_error(s_tag, "ctrl_meas write failed");
00959         return err;
00960     }
00961
00962     /* Step 2: Poll status register until measuring bit (bit 3) clears */
00963     uint32_t poll_count = 0;
00964     while (poll_count < (uint32_t)k_bmp280_poll_max) {
00965         uint8_t status_byte = 0;
00966         err = internal_read_regs(k_bmp280_reg_status, &status_byte, k_bmp280_single_byte);
```

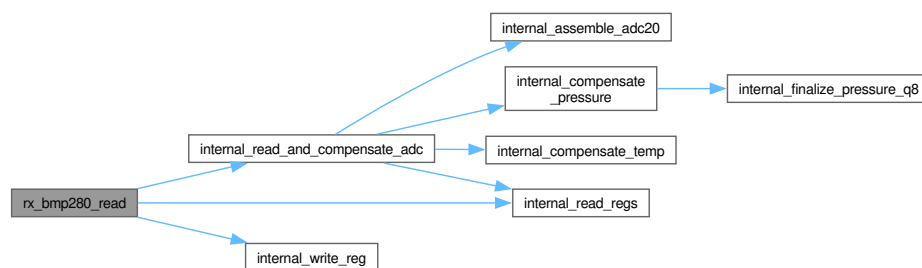
```

00967     if (err != k_rx_ok) {
00968         rx_log_error(s_tag, "Status register read failed");
00969         return err;
00970     }
00971
00972     if ((status_byte & k_bmp280_status_meas_mask) == 0U) {
00973         break; /* Measurement complete */
00974     }
00975
00976     (void)tx_thread_sleep(k_bmp280_poll_sleep_ticks); /* Yield CPU while sensor is measuring */
00977     poll_count++;
00978 }
00979
00980 if (poll_count >= (uint32_t)k_bmp280_poll_max) {
00981     rx_log_error(s_tag, "Status poll timeout");
00982     return k_rx_err_timeout;
00983 }
00984
00985 /* Step 3: Read ADC, apply compensation, validate output range */
00986 return internal_read_and_compensate_adc(out);
00987 }

```

References [internal_read_and_compensate_adc\(\)](#), [internal_read_regs\(\)](#), [internal_write_reg\(\)](#), [k_bmp280_ctrl_meas_val](#), [k_bmp280_poll_max](#), [k_bmp280_poll_sleep_ticks](#), [k_bmp280_reg_ctrl_meas](#), [k_bmp280_reg_status](#), [k_bmp280_single_byte](#), [k_bmp280_status_meas_mask](#), [s_initialized](#), and [s_tag](#).

Here is the call graph for this function:



5.5.7 Variable Documentation

5.5.7.1 s'bus' name

```
const char* const s_bus_name = "i2c1_baro" [static]
```

I2C bus name for BMP280 (RIIC1, device addr 0x76, registered as "i2c1_baro" in main.c).

Note

Must match the bus name registered via `rx_bus_manager_add_bus()` in main.c

See also

main.c Bus registration with name "i2c1_baro"

Since

Version 1.0.0

Definition at line 231 of file [rx_bmp280.c](#).

Referenced by [internal_read_regs\(\)](#), [internal_write_reg\(\)](#), and [rx_bmp280_init\(\)](#).

5.5.7.2 s`calib

[bmp280_calib_t](#) s_calib = {} [static]

Factory calibration coefficients read from BMP280 OTP during init.

Note

Read-only after [rx_bmp280_init\(\)](#); access outside init is read-only

Invariant

s_initialized == true => s_calib contains valid OTP factory calibration coefficients

See also

[rx_bmp280_init\(\)](#) Reads 24-byte OTP block into this struct
BMP280 datasheet section 4.2.2 for coefficient descriptions

Since

Version 1.0.0

Definition at line 213 of file [rx_bmp280.c](#).

00213 {};

Referenced by [internal_compensate_pressure\(\)](#), [internal_compensate_temp\(\)](#), [internal_finalize_pressure_q8\(\)](#), [internal_parse_calibration\(\)](#), and [internal_read_and_validate_calibration\(\)](#).

5.5.7.3 s`initialized

bool s_initialized = false [static]

Guard flag: true after successful [rx_bmp280_init\(\)](#).

Note

Only set to true in [rx_bmp280_init\(\)](#) after all init steps succeed

Invariant

Once true, s_calib and s_manager are valid and non-NULL

Since

Version 1.0.0

Definition at line 220 of file [rx_bmp280.c](#).

Referenced by [internal_read_and_compensate_adc\(\)](#), [rx_bmp280_init\(\)](#), and [rx_bmp280_read\(\)](#).

5.5.7.4 s`manager

```
rx_bus_manager_t* s_manager = NULL [static]
```

Bus manager pointer stored during [rx_bmp280_init\(\)](#).

Note

Set once in [rx_bmp280_init\(\)](#); NULL before init and on error

See also

[rx_bmp280_init\(\)](#) Sets this pointer

Since

Version 1.0.0

Definition at line 204 of file [rx_bmp280.c](#).

Referenced by [internal_read_and_validate_calibration\(\)](#), [internal_read_regs\(\)](#), [internal_verify_chip_id\(\)](#), [internal_write_reg\(\)](#), and [rx_bmp280_init\(\)](#).

5.5.7.5 s`tag

```
const char* const s_tag = "BMP280" [static]
```

Log tag for this module.

Note

Constant; never modified after initialization

Since

Version 1.0.0

Definition at line 197 of file [rx_bmp280.c](#).

Referenced by [internal_compensate_pressure\(\)](#), [internal_read_and_compensate_adc\(\)](#), [internal_read_and_validate_c](#), [internal_verify_chip_id\(\)](#), [rx_bmp280_init\(\)](#), and [rx_bmp280_read\(\)](#).

5.6 rx_bmp280.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_bmp280.c
00003  * @brief BMP280 Digital Barometric Pressure and Temperature Sensor Driver Implementation
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * Complete driver implementation for the Bosch BMP280 barometric pressure
00009  * sensor connected via I2C on RIIC1. Implements the Bosch integer-only
00010  * compensation algorithm (datasheet section 4.2.3) for accurate temperature
00011  * and pressure calculation without floating-point operations.
00012  *
00013  * # Integer Compensation Algorithm
00014  *
00015  * The Bosch compensation formulas use 64-bit integer arithmetic for pressure
00016  * to maintain precision across the full operating range. The intermediate
00017  * variable t_fine is computed from the temperature ADC value and reused
00018  * in the pressure formula for temperature compensation.
00019  *
00020  * Formula source: BMP280 datasheet v1.19, section 4.2.3
00021  * "Compensation formulas in double precision floating point"
00022  * (Integer version provided in appendix A)
00023  *
00024  * # NASA Power of 10 Compliance
00025  *
00026  * | Rule | Status | Notes |
00027  * |-----|-----|-----|
00028  * | 1. No goto | [PASS] | Structured if/while only |
00029  * | 2. Bounded loops | [PASS] | Status poll bounded by k_bmp280_poll_max |
00030  * | 3. No dynamic memory | [PASS] | All buffers on stack |
00031  * | 4. Short functions | [PASS] | Max function ~40 lines |
00032  * | 5. Assertions | [PASS] | 2+ checks per function |
00033  * | 6. Data scope | [PASS] | Locals at point of use |
00034  * | 7. Check returns | [PASS] | All I2C returns validated |
00035  * | 8. Limit preprocessor | [PASS] | C23 typed enums only |
00036  * | 9. Pointer restrictions | [WARN] | bus_manager function pointers (DIP) |
00037  * | 10. Compile warnings | [PASS] | -Wall -Wextra -Werror |
00038  *
00039  * @see rx_bmp280.h Public API
00040  * @see rx_bmp280_regs.h Register and constant definitions
00041  *
00042  * @author STAR Team
00043  * @date 2026-03-04
00044  * @since Version 1.0.0
00045  * @copyright Copyright (c) 2026 Locked Inc.
00046  * SPDX-License-Identifier: MIT
00047  */
00048
00049 #include "rx_bmp280.h"
00050
00051 #include <stddef.h>
00052 #include <stdint.h>
00053
00054 #include "rx_bmp280_regs.h"
00055 #include "rx_bus_i2c.h"
00056 #include "rx_check.h"
00057 #include "rx_log.h"
00058 #include "tx_api.h"
00059
00060 /**
00061  * =====
00062  * Module-level Constants
00063  * =====
00064  */
00065 /**
00066  * @enum bmp280_sizeof_t
00067  * @brief Expected sizes for static assertions in this module
00068  *
00069  * @details
00070  * Named constants used in static_assert comparisons to avoid magic numbers.
00071  *
00072  * @since Version 1.0.0
00073  */
00074 typedef enum : uint8_t {
00075     k_bmp280_sizeof_int64 = 8, /**< Expected sizeof(int64_t) for overflow-safe pressure math */
00076     k_bmp280_calib_tail_bytes = 2, /**< Number of bytes occupied by the final coefficient (P9) */
00077 } bmp280_sizeof_t;
00078
00079 /**
00080  * @enum bmp280_le16_idx_t

```

```

00081 * @brief Byte indices for little-endian 16-bit pair (LSB first)
00082 *
00083 * @details
00084 * Index constants for accessing LSB and MSB within a 2-byte little-endian pair.
00085 * Used by internal_parse_u16_le() and internal_parse_s16_le().
00086 *
00087 * @since Version 1.0.0
00088 */
00089 typedef enum : uint8_t {
00090     k_bmp280_le16_lsb_idx = 0, /**< Index of least-significant byte in a 2-byte LE pair */
00091     k_bmp280_le16_msb_idx = 1, /**< Index of most-significant byte in a 2-byte LE pair */
00092 } bmp280_le16_idx_t;
00093
00094 /**
00095 * @enum bmp280_adc20_idx_t
00096 * @brief Byte indices for 3-byte 20-bit ADC buffer (MSB first)
00097 *
00098 * @details
00099 * Index constants for accessing MSB, LSB, and XLSB within a 3-byte ADC buffer.
00100 * Used by internal_assemble_adc20().
00101 *
00102 * @since Version 1.0.0
00103 */
00104 typedef enum : uint8_t {
00105     k_bmp280_adc20_msb_idx = 0, /**< Index of MSB byte in 3-byte ADC buffer */
00106     k_bmp280_adc20_lsb_idx = 1, /**< Index of LSB byte in 3-byte ADC buffer */
00107     k_bmp280_adc20_xlsb_idx = 2, /**< Index of XLSB byte in 3-byte ADC buffer */
00108 } bmp280_adc20_idx_t;
00109
00110 /**
00111 * @enum bmp280_adc20_range_t
00112 * @brief Valid range for 20-bit ADC values assembled by internal_assemble_adc20()
00113 *
00114 * @details
00115 * The BMP280 ADC output is a 20-bit non-negative integer.
00116 * Maximum value is  $2^{20} - 1 = 0xFFFFF = 1048575$ .
00117 * Used as a postcondition bound in internal_assemble_adc20().
00118 *
00119 * @since Version 1.0.0
00120 */
00121 typedef enum : uint32_t {
00122     k_bmp280_adc_20bit_max = 0xFFFFFU, /**< Maximum valid 20-bit ADC value ( $2^{20} - 1 = 1048575$ ) */
00123 } bmp280_adc20_range_t;
00124
00125 /**
00126 * @enum bmp280_press_range_pa_t
00127 * @brief BMP280 valid pressure range in base Pascal units
00128 *
00129 * @details
00130 * Valid pressure limits per the BMP280 datasheet operating specifications:
00131 * - 300 hPa = 30000 Pa (minimum, e.g. high altitude)
00132 * - 1100 hPa = 110000 Pa (maximum, e.g. sea level)
00133 *
00134 * These base-Pa values are multiplied by 256 for the fixed-point Pa*256 format
00135 * used by the Bosch integer compensation formula output.
00136 * See bmp280_press_range_t for the Pa*256 scaled limits.
00137 *
00138 * @see bmp280_press_range_t Pa*256 scaled limits used in postcondition check
00139 *
00140 * @since Version 1.0.0
00141 */
00142 typedef enum : uint32_t {
00143     k_bmp280_press_min_pa = 30000U, /**< Minimum valid pressure: 300 hPa in Pa */
00144     k_bmp280_press_max_pa = 110000U, /**< Maximum valid pressure: 1100 hPa in Pa */
00145 } bmp280_press_range_pa_t;
00146
00147 /**
00148 * @enum bmp280_temp_range_t
00149 * @brief BMP280 temperature output range limits for postcondition validation
00150 *
00151 * @details
00152 * Valid temperature output range per the BMP280 datasheet operating specifications:
00153 * - Temperature: -40.00 degC to +85.00 degC (in centi-degC units)
00154 *
00155 * @see bmp280_press_range_t Pressure output range limits
00156 * @see rx_bmp280_read() Applies these range checks after compensation
00157 *
00158 * @since Version 1.0.0
00159 */
00160 typedef enum : int32_t {
00161     k_bmp280_temp_min_cdegc_impl = -4000, /**< Minimum valid temperature: -40.00 degC in centi-degC */
00162     k_bmp280_temp_max_cdegc_impl = 8500, /**< Maximum valid temperature: +85.00 degC in centi-degC */
00163 } bmp280_temp_range_t;
00164
00165 /**
00166 * @enum bmp280_press_range_t
00167 * @brief BMP280 pressure output range limits for postcondition validation

```



```

00168 *
00169 * @details
00170 * Valid pressure output range per the BMP280 datasheet operating specifications:
00171 * - Pressure: 300 hPa to 1100 hPa (in Pa*256 fixed-point units)
00172 * Derivation: k_bmp280_press_min_pa * 256 = 30000 * 256 = 7680000
00173 * k_bmp280_press_max_pa * 256 = 110000 * 256 = 28160000
00174 *
00175 * @see bmp280_temp_range_t Temperature output range limits
00176 * @see bmp280_press_range_pa_t Base Pa constants from which Pa*256 values are derived
00177 * @see rx_bmp280_read() Applies these range checks after compensation
00178 *
00179 * @since Version 1.0.0
00180 */
00181 typedef enum : uint32_t {
00182     k_bmp280_press_min_pa_256 =
00183         7680000U, /**< Minimum valid pressure: 300 hPa * 256 (k_bmp280_press_min_pa * 256) */
00184     k_bmp280_press_max_pa_256 =
00185         28160000U, /**< Maximum valid pressure: 1100 hPa * 256 (k_bmp280_press_max_pa * 256) */
00186 } bmp280_press_range_t;
00187
00188 /*
=====
00189 * Module-Static State
00190 *
=====
00191 */
00192
00193 /** @brief Log tag for this module
00194 * @note Constant; never modified after initialization
00195 * @since Version 1.0.0
00196 */
00197 static const char* const s_tag = "BMP280";
00198
00199 /** @brief Bus manager pointer stored during rx_bmp280_init()
00200 * @note Set once in rx_bmp280_init(); NULL before init and on error
00201 * @see rx_bmp280_init() Sets this pointer
00202 * @since Version 1.0.0
00203 */
00204 static rx_bus_manager_t* s_manager = NULL;
00205
00206 /** @brief Factory calibration coefficients read from BMP280 OTP during init
00207 * @note Read-only after rx_bmp280_init(); access outside init is read-only
00208 * @invariant s_initialized == true => s_calib contains valid OTP factory calibration coefficients
00209 * @see rx_bmp280_init() Reads 24-byte OTP block into this struct
00210 * @see BMP280 datasheet section 4.2.2 for coefficient descriptions
00211 * @since Version 1.0.0
00212 */
00213 static bmp280_calib_t s_calib = {};
00214
00215 /** @brief Guard flag: true after successful rx_bmp280_init()
00216 * @note Only set to true in rx_bmp280_init() after all init steps succeed
00217 * @invariant Once true, s_calib and s_manager are valid and non-NULL
00218 * @since Version 1.0.0
00219 */
00220 static bool s_initialized = false;
00221
00222 /** @brief I2C bus name for BMP280 (RIIC1, device addr 0x76, registered as "i2c1_baro" in main.c)
00223 * @note Must match the bus name registered via rx_bus_manager_add_bus() in main.c
00224 * @see main.c Bus registration with name "i2c1_baro"
00225 * @since Version 1.0.0
00226 */
00227 #ifdef UNIT_TEST
00228 static const char* s_bus_name =
00229     "i2c1_baro"; /* NOLINT(cppcoreguidelines-avoid-non-const-global-variables) */
00230 #else
00231 static const char* const s_bus_name = "i2c1_baro";
00232 #endif
00233
00234 /*
=====
00235 * Forward Declarations
00236 *
=====
00237 */
00238
00239 /** @brief Write a single byte to a BMP280 register via I2C */
00240 RX_STATIC_TESTABLE rx_err_t internal_write_reg(uint8_t reg, uint8_t val);
00241 /** @brief Burst-read consecutive registers from BMP280 via I2C write-read */
00242 RX_STATIC_TESTABLE rx_err_t internal_read_regs(uint8_t reg, uint8_t* buf, uint8_t len);
00243 /** @brief Apply Bosch integer temperature compensation formula (returns centi-degC, sets t_fine) */
00244 RX_STATIC_TESTABLE int32_t internal_compensate_temp(int32_t adc_temp, int32_t* t_fine_out);
00245 /** @brief Apply Bosch integer pressure compensation formula (returns Pa*256 in press_out) */
00246 RX_STATIC_TESTABLE rx_err_t internal_compensate_pressure(int32_t adc_pressure,
00247     int32_t t_fine,
00248     uint32_t* press_out);
00249 /** @brief Assemble an unsigned 16-bit little-endian value from two consecutive bytes */
00250 RX_STATIC_TESTABLE uint16_t internal_parse_u16_le(const uint8_t* buf);

```

```

00251 /** @brief Assemble a signed 16-bit little-endian value from two consecutive bytes */
00252 RX_STATIC_TESTABLE int16_t internal_parse_s16_le(const uint8_t* buf);
00253 /** @brief Assemble a 20-bit signed ADC value from a 3-byte MSB/LSB/XLSB buffer */
00254 RX_STATIC_TESTABLE int32_t internal_assemble_adc20(const uint8_t* buf);
00255 /** @brief Parse 24-byte OTP calibration block into s_calib coefficients */
00256 RX_STATIC_TESTABLE void internal_parse_calibration(const uint8_t* buf);
00257 /** @brief Apply final Q8 fixed-point scaling and dig_P7 offset to pressure intermediate */
00258 RX_STATIC_TESTABLE uint32_t internal_finalize_pressure_q8(int64_t p, int64_t var1, int64_t var2);
00259 /** @brief Read ADC registers, assemble values, apply compensation, and validate range */
00260 RX_STATIC_TESTABLE rx_err_t internal_read_and_compensate_adc(bmp280_data_t* out);
00261
00262 /*
=====
00263 * Internal Helpers
00264 *
=====
00265 */
00266
00267 /**
00268 * @brief Write a single byte to a BMP280 register via I2C
00269 *
00270 * @details
00271 * Sends a 2-byte I2C write transaction [reg, val] to the BMP280 at
00272 * device address 0x76 (embedded in the "i2c1_baro" bus configuration).
00273 *
00274 *
00275 *
00276 * @pre s_manager non-NULL (set by rx_bmp280_init)
00277 * @pre "i2c1_baro" bus initialized
00278 * @post Register contains val on k_rx_ok
00279 * @post Bus transaction completes before function returns
00280 *
00281 * @note Not thread-safe
00282 * @see rx_bus_i2c_write()
00283 *
00284 * @since Version 1.0.0
00285 */
00286 RX_STATIC_TESTABLE rx_err_t internal_write_reg(uint8_t reg, uint8_t val)
00287 {
00288     /* Pre-conditions: s_manager and s_bus_name are non-NULL by construction;
00289      * internal_write_reg is only called after rx_bmp280_init stores a valid
00290      * manager pointer and s_bus_name is a compile-time string constant. */
00291     RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before write");
00292     RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL");
00293     uint8_t buf[k_bmp280_write_buf_size];
00294     buf[k_bmp280_write_idx_reg] = reg;
00295     buf[k_bmp280_write_idx_val] = val;
00296     return rx_bus_i2c_write(s_manager, s_bus_name, buf, k_bmp280_write_buf_size);
00297 }
00298
00299 /**
00300 * @brief Burst-read consecutive registers from BMP280 via I2C write-read
00301 *
00302 * @details
00303 * Issues a combined I2C write-read transaction: write starting register
00304 * address, then read len bytes. BMP280 auto-increments register address.
00305 *
00306 *
00307 *
00308 * @pre s_manager non-NULL
00309 * @pre "i2c1_baro" bus initialized in s_manager
00310 * @pre buf capacity >= len
00311 * @post buf[0..len-1] contain register data on k_rx_ok
00312 * @post Bus transaction completes before function returns
00313 *
00314 * @note Not thread-safe
00315 * @see rx_bus_i2c_write_read()
00316 *
00317 * @since Version 1.0.0
00318 */
00319 RX_STATIC_TESTABLE rx_err_t internal_read_regs(uint8_t reg, uint8_t* buf, uint8_t len)
00320 {
00321     /* Pre-conditions: s_manager is non-NULL (only called after successful init),
00322      * buf is a local stack buffer (never NULL), len is an enum constant (always > 0). */
00323     RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before read");
00324     RX_ASSERT_PRE(buf != NULL, "buf must be non-NULL for read operation");
00325     RX_ASSERT_PRE(len > 0U, "len must be positive for read operation");
00326     return rx_bus_i2c_write_read(s_manager, s_bus_name, &reg, k_bmp280_read_cmd_size, buf, len);
00327 }
00328
00329 /**
00330 * @brief Assemble an unsigned 16-bit little-endian value from two consecutive bytes
00331 *
00332 * @details
00333 * Reads buf[0] (LSB) and buf[1] (MSB) and combines them into a uint16_t value
00334 * using little-endian byte order (LSB first), matching BMP280 calibration register layout.
00335 */

```

```

00336 *
00337 *
00338 * @pre buf non-NULL, capacity >= 2
00339 * @post Return value == (uint16_t)buf[0] | ((uint16_t)buf[1] « 8)
00340 * @post No side effects; buf and global state are not modified
00341 *
00342 * @note Inline; zero overhead after optimization
00343 * @see internal_parse_s16_le() Signed variant
00344 *
00345 * @since Version 1.0.0
00346 */
00347 RX_STATIC_TESTABLE uint16_t internal_parse_u16_le(const uint8_t* buf)
00348 {
00349     /* Pre-condition: buf is always a valid pointer to a calibration or ADC buffer;
00350      * all callers pass stack-allocated arrays (never NULL). */
00351     RX_ASSERT_PRE(buf != NULL, "buf must not be NULL");
00352     #ifndef UNIT_TEST
00353     if (buf == NULL) {
00354         return 0U;
00355     }
00356     #endif
00357     static_assert((bool)((unsigned)k_bmp280_byte_shift == (unsigned)k_bmp280_sizeof_int64) != 0,
00358                  "byte shift must be 8 for LE 16-bit assembly");
00359     return (uint16_t)((uint16_t)buf[k_bmp280_le16_lsb_idx] |
00360                      ((uint16_t)buf[k_bmp280_le16_msb_idx] « k_bmp280_byte_shift));
00361 }
00362 /**
00363  * @brief Assemble a signed 16-bit little-endian value from two consecutive bytes
00364  *
00365  * @details
00366  * Reads buf[0] (LSB) and buf[1] (MSB) and reinterprets the assembled uint16_t
00367  * as a two's complement int16_t value. Matches BMP280 signed calibration coefficient layout.
00368  *
00369  *
00370  *
00371  *
00372  * @pre buf non-NULL, capacity >= 2
00373  * @post Return value correctly represents two's complement signed value
00374  * @post No side effects; buf and global state are not modified
00375  *
00376  * @note Inline; zero overhead after optimization
00377  * @see internal_parse_u16_le() Unsigned variant
00378  *
00379  * @since Version 1.0.0
00380 */
00381 RX_STATIC_TESTABLE int16_t internal_parse_s16_le(const uint8_t* buf)
00382 {
00383     /* Pre-condition: buf is always a valid calibration buffer pointer (never NULL). */
00384     RX_ASSERT_PRE(buf != NULL, "buf must not be NULL");
00385     #ifndef UNIT_TEST
00386     if (buf == NULL) {
00387         return 0;
00388     }
00389     #endif
00390     static_assert((bool)((sizeof(int16_t) == sizeof(uint16_t)) != 0),
00391                  "int16_t and uint16_t must be same size for safe reinterpret cast");
00392     return (int16_t)internal_parse_u16_le(buf);
00393 }
00394 /**
00395  * @brief Assemble a 20-bit signed ADC value from a 3-byte MSB/LSB/XLSB buffer
00396  *
00397  * @details
00398  * The BMP280 outputs pressure and temperature as 20-bit raw values stored in
00399  * three consecutive bytes with the following layout:
00400  * - buf[0] (MSB): bits[19:12] of the ADC value
00401  * - buf[1] (LSB): bits[11:4] of the ADC value
00402  * - buf[2] (XLSB): bits[3:0] of the ADC value in the upper nibble (bits [7:4])
00403  *
00404  * Assembly formula: (MSB « 12) | (LSB « 4) | (XLSB » 4)
00405  *
00406  *
00407  *
00408  *
00409  * @pre buf non-NULL, capacity >= 3
00410  * @post Return value is a valid 20-bit non-negative integer
00411  *
00412  * @note Inline; zero overhead after optimization
00413  * @see internal_compensate_temp() Consumes the assembled temperature ADC value
00414  * @see internal_compensate_pressure() Consumes the assembled pressure ADC value
00415  *
00416  * @since Version 1.0.0
00417 */
00418 RX_STATIC_TESTABLE int32_t internal_assemble_adc20(const uint8_t* buf)
00419 {
00420     /* Pre-condition: buf is a 6-byte ADC read buffer (never NULL).
00421      * Post-condition: result is in [0, 0xFFFFF] because only 20 bits are
00422      * assembled (MSB contributes bits[19:12], LSB bits[11:4], XLSB bits[3:0]). */

```

```

00423 RX_ASSERT_PRE(buf != NULL, "buf must not be NULL");
00424 #ifndef UNIT_TEST
00425 if (buf == NULL) {
00426     return 0;
00427 }
00428 #endif
00429 /* Assembly: (MSB « 12) | (LSB « 4) | (XLSB » 4) produces 20-bit value */
00430 const int32_t result =
00431     (int32_t)((((uint32_t)buf[k_bmp280_adc20_msb_idx] « k_bmp280_shift_msb) |
00432         ((uint32_t)buf[k_bmp280_adc20_lsb_idx] « k_bmp280_shift_lsb_left) |
00433         ((uint32_t)buf[k_bmp280_adc20_xlsb_idx] » k_bmp280_shift_xlsb_right)));
00434 /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(20-bit mask guarantees range) */
00435 RX_ASSERT_POST(result >= 0 && result <= (int32_t)k_bmp280_adc_20bit_max,
00436     "ADC20 result out of 20-bit range");
00437 /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00438 return result;
00439 }
00440
00441 /**
00442  * @enum bmp280_temp_comp_t
00443  * @brief Integer shift and scale constants for BMP280 temperature compensation formula
00444  *
00445  * @details
00446  * Constants used exclusively by internal_compensate_temp(). Placed at module scope
00447  * to avoid -Werror=unused-local-typedefs when the GNURX cross-compiler sees the
00448  * typedef name unused inside the function body.
00449  *
00450  * @since Version 1.0.0
00451  */
00452 typedef enum : int32_t {
00453     k_temp_shift_adc_3 = 3, /**< adc_T right shift for T1 subtraction step */
00454     k_temp_shift_t1_1 = 1, /**< dig_T1 left shift in first var1 step */
00455     k_temp_shift_11 = 11, /**< Right shift for var1 final step */
00456     k_temp_shift_adc_4 = 4, /**< adc_T right shift for T1 comparison steps */
00457     k_temp_shift_12 = 12, /**< Right shift for squared difference */
00458     k_temp_shift_14 = 14, /**< Right shift for var2 final step */
00459     k_temp_fine_scale = 5, /**< Scale factor in fine-to-output conversion */
00460     k_temp_round_add = 128, /**< Rounding constant in fine-to-output conversion */
00461     k_temp_shift_out = 8, /**< Right shift to produce 0.01 degC output */
00462 } bmp280_temp_comp_t;
00463
00464 /**
00465  * @brief Apply Bosch integer temperature compensation formula
00466  *
00467  * @details
00468  * Implements the Bosch BMP280 integer temperature compensation algorithm
00469  * verbatim from datasheet v1.19 appendix A. Uses trimming parameters
00470  * from s_calib and computes the fine temperature value t_fine which
00471  * is required for pressure compensation.
00472  *
00473  * Algorithm:
00474  * @code
00475  * var1 = (((adc_T » 3) - (dig_T1 « 1)) * dig_T2) » 11
00476  * var2 = (((((adc_T » 4) - dig_T1) * ((adc_T » 4) - dig_T1)) » 12) * dig_T3) » 14
00477  * t_fine = var1 + var2
00478  * T = (t_fine * 5 + 128) » 8 // Unit: centi-degrees (0.01 degC)
00479  * @endcode
00480  *
00481  *
00482  *
00483  * @pre adc_T is valid 20-bit ADC output from BMP280
00484  * @pre t_fine_out non-NULL
00485  * @post *t_fine_out set to intermediate temperature value
00486  * @post Return value in range [-4000, 8500] for valid operating range
00487  *
00488  * @note Pure integer arithmetic, no floating-point
00489  * @see BMP280 datasheet v1.19 appendix A, compensate_T_int32
00490  *
00491  * @since Version 1.0.0
00492  */
00493 RX_STATIC_TESTABLE int32_t internal_compensate_temp(int32_t adc_temp, int32_t* t_fine_out)
00494 {
00495     /* Pre-conditions: t_fine_out is a local stack variable (never NULL);
00496      * adc_temp is produced by internal_assemble_adc20 which masks to 20 bits. */
00497     RX_ASSERT_PRE(t_fine_out != NULL, "t_fine_out must be non-NULL");
00498     #ifndef UNIT_TEST
00499     if (t_fine_out == NULL) {
00500         return 0;
00501     }
00502     #endif
00503     RX_ASSERT_PRE(adc_temp >= 0 && adc_temp <= (int32_t)k_bmp280_adc_20bit_max,
00504         "adc_T must be a 20-bit ADC value");
00505     const int32_t var1 =
00506         (((adc_temp » k_temp_shift_adc_3) - ((int32_t)s_calib.dig_T1 « k_temp_shift_t1_1))) *
00507         ((int32_t)s_calib.dig_T2)) »
00508         k_temp_shift_11;
00509     const int32_t var2 = (((((adc_temp » k_temp_shift_adc_4) - ((int32_t)s_calib.dig_T1)) *

```

```

00510             ((adc_temp » k_temp_shift_adc_4) - ((int32_t)s_calib.dig_T1))) »
00511             k_temp_shift_12) *
00512             ((int32_t)s_calib.dig_T3)) »
00513             k_temp_shift_14;
00514
00515 *t_fine_out = var1 + var2;
00516
00517 return (*t_fine_out * k_temp_fine_scale + k_temp_round_add) » k_temp_shift_out;
00518 }
00519
00520 /**
00521  * @enum press_comp_constants_t
00522  * @brief Bosch BMP280 pressure compensation formula constants (datasheet appendix A)
00523  *
00524  * @details
00525  * Integer shift and scale constants used exclusively by internal_compensate_pressure()
00526  * and internal_finalize_pressure_q8(). Placed at module scope so both functions can
00527  * reference the same named constants without redundancy.
00528  *
00529  * @since Version 1.0.0
00530  */
00531 typedef enum : int32_t {
00532     k_press_t_offset = 128000, /**< t_fine offset in pressure formula */
00533     k_press_p_scale = 1048576, /**< ADC scaling constant */
00534     k_press_shift_31 = 31, /**< Shift for 31-bit alignment */
00535     k_press_shift_47 = 47, /**< Shift for 47-bit alignment */
00536     k_press_shift_8 = 8, /**< Output shift for Q8 fixed-point Pa*256 */
00537     k_press_mul_3125 = 3125, /**< Bosch formula scaling factor */
00538     k_press_shift_35 = 35, /**< Shift for 35-bit intermediate */
00539     k_press_shift_25 = 25, /**< Shift for 25-bit intermediate */
00540     k_press_shift_19 = 19, /**< Shift for 19-bit intermediate */
00541     k_press_shift_13 = 13, /**< Shift for P9 coefficient shift */
00542     k_press_shift_17 = 17, /**< Shift for P5 coefficient step */
00543     k_press_shift_12 = 12, /**< Right shift for dig_P3 step */
00544     k_press_shift_33 = 33, /**< Divisor shift for var1 */
00545     k_press_shift_4 = 4, /**< Shift for P7 addition */
00546 } press_comp_constants_t;
00547
00548 /**
00549  * @brief Apply final Q8 fixed-point scaling and dig_P7 offset to pressure intermediate
00550  *
00551  * @details
00552  * Implements the last step of the Bosch BMP280 pressure compensation formula
00553  * (datasheet Section 4.2.3): right-shifts the accumulated intermediate value to
00554  * produce Pa*256 (Q8 fixed-point), then adds the dig_P7 offset term.
00555  *
00556  * Extracted from internal_compensate_pressure() to keep that function within
00557  * the 60-line NASA Power of 10 Rule 4 guideline.
00558  *
00559  *
00560  *
00561  * @pre s_calib populated by internal_parse_calibration() (dig_P1 != 0)
00562  * @post Return value is pressure * 256 in Pascal units (Q8 format)
00563  *
00564  * @note Accesses s_calib.dig_P7 directly (module-level static)
00565  * @see internal_compensate_pressure() Sole caller
00566  *
00567  * @since Version 1.0.0
00568  */
00569 RX_STATIC_TESTABLE uint32_t internal_finalize_pressure_q8(int64_t p, int64_t var1, int64_t var2)
00570 {
00571     /* Pre-conditions: dig_P1 non-zero is guaranteed by the rx_bmp280_init post-condition check;
00572      * p is non-negative by construction from the BMP280 datasheet algorithm (var1 > 0 guard). */
00573     RX_ASSERT_PRE(s_calib.dig_P1 != 0U, "s_calib must be initialized before finalizing pressure");
00574     RX_ASSERT_PRE(p >= 0, "p must be non-negative before final Q8 scaling");
00575     static_assert((bool)((sizeof(int64_t) == k_bmp280_sizeof_int64) != 0),
00576                  "int64_t must be 64-bit for overflow-safe pressure math");
00577     /* Bosch BMP280 datasheet formula (Section 4.2.3):
00578      * Step 1: shift (p + fine adjustments) right by 8 to get Q8 fixed-point Pa*256
00579      * Step 2: add dig_P7 offset (left-shifted 4) AFTER the right shift
00580      * Note: shifting the dig_P7 term together with the rest would incorrectly
00581      *       scale it by an additional factor of 256. */
00582     const int64_t p_scaled = (p + var1 + var2) » k_press_shift_8;
00583     const int64_t dig_p7 = ((int64_t)s_calib.dig_P7) « k_press_shift_4;
00584     return (uint32_t)(p_scaled + dig_p7);
00585 }
00586
00587 /**
00588  * @brief Apply Bosch integer pressure compensation formula
00589  *
00590  * @details
00591  * Implements the Bosch BMP280 integer pressure compensation algorithm
00592  * verbatim from datasheet v1.19 appendix A. Uses t_fine from temperature
00593  * compensation and trimming parameters from s_calib.
00594  *
00595  * Returns k_rx_err_invalid_state if the internal var1 divisor would be zero
00596  * (prevents division by zero as required by NASA Power of 10 Rule 7).

```

```

00597 *
00598 * Output unit: Pa * 256 (fixed-point Q8.0 format). Divide by 256.0 for Pa.
00599 *
00600 *
00601 *
00602 * @pre adc_P is valid 20-bit ADC output from BMP280
00603 * @pre t_fine was computed by internal_compensate_temp() for same measurement
00604 * @pre press_out non-NULL
00605 * @post *press_out in range [7680000, 28160000] (300*256 hPa to 1100*256 hPa, Pa*256) on success
00606 * @post Returns k_rx_err_invalid_state without modifying *press_out when var1 == 0
00607 *
00608 * @note Uses 64-bit arithmetic to prevent overflow at intermediate values
00609 * @see BMP280 datasheet v1.19 appendix A, compensate_P_int64
00610 *
00611 * @since Version 1.0.0
00612 */
00613 RX_STATIC_TESTABLE rx_err_t internal_compensate_pressure(int32_t adc_pressure,
00614                                                         int32_t t_fine,
00615                                                         uint32_t* press_out)
00616 {
00617     /* Pre-conditions: dig_P1 non-zero is validated by rx_bmp280_init before any read is possible;
00618      * adc_pressure is a 20-bit value from internal_assemble_adc20 (always >= 0);
00619      * press_out is a local stack variable passed by internal_read_and_compensate_adc (never NULL). */
00620     RX_ASSERT_PRE(s_calib.dig_P1 != 0U, "dig_P1 must be non-zero (factory calibration)");
00621     RX_ASSERT_PRE(adc_pressure >= 0, "adc_P must be non-negative (20-bit unsigned ADC value)");
00622     RX_ASSERT_PRE(press_out != NULL, "press_out must not be NULL");
00623 #ifdef UNIT_TEST
00624     if (press_out == NULL) {
00625         return k_rx_err_null_ptr;
00626     }
00627 #endif
00628     /* k_press_base_one must be int64_t (not enum) because the expression
00629      * (k_press_base_one « k_press_shift_47) requires a 48-bit value.
00630      * A 32-bit integer cannot hold (1 « 47); int64_t is required to
00631      * prevent undefined behavior from shifting beyond the type width.
00632      * C23 typed enums cannot have int64_t as underlying type on RX72N. */
00633     static const int64_t k_press_base_one = 1;
00634
00635     int64_t var1 = ((int64_t)t_fine) - k_press_t_offset;
00636     int64_t var2 = var1 * var1 * (int64_t)s_calib.dig_P6;
00637     var2 += (var1 * (int64_t)s_calib.dig_P5) « k_press_shift_17;
00638     var2 += ((int64_t)s_calib.dig_P4) « k_press_shift_35;
00639     var1 = ((var1 * var1 * (int64_t)s_calib.dig_P3) » k_press_shift_8) +
00640           ((var1 * (int64_t)s_calib.dig_P2) « k_press_shift_12);
00641     var1 = (((k_press_base_one « k_press_shift_47) + var1)) * ((int64_t)s_calib.dig_P1) »
00642           k_press_shift_33;
00643
00644     /* Division by zero guard (NASA Power of 10 Rule 7) */
00645     if (var1 == 0) {
00646         rx_log_error(s_tag, "Pressure compensation: var1==0 (dig_P1 fault)");
00647         return k_rx_err_invalid_state;
00648     }
00649
00650     int64_t p = (int64_t)k_press_p_scale - adc_pressure;
00651     p = (((p « k_press_shift_31) - var2) * k_press_mul_3125) / var1;
00652
00653     var1 = (((int64_t)s_calib.dig_P9) * (p » k_press_shift_13) * (p » k_press_shift_13)) »
00654           k_press_shift_25;
00655     var2 = (((int64_t)s_calib.dig_P8) * p) » k_press_shift_19;
00656
00657     *press_out = internal_finalize_pressure_q8(p, var1, var2);
00658     return k_rx_ok;
00659 }
00660
00661 /*
00662 =====
00663 * Calibration Parsing
00664 =====
00665 */
00666 /**
00667  * @brief Parse 24-byte OTP calibration block into s_calib coefficients
00668  *
00669  * @details
00670  * Parses the 24-byte factory-calibrated trimming parameters read from OTP
00671  * registers k_bmp280_reg_calib_start..k_bmp280_reg_calib_end (0x88-0x9F).
00672  * All coefficients are stored little-endian (LSB first) in the OTP block.
00673  *
00674  * @pre buf non-NULL
00675  * @pre buf contains exactly k_bmp280_calib_byte_count bytes from OTP registers
00676  * @post s_calib.dig_T1..T3 and dig_P1..P9 populated from buf
00677  * @post Caller must verify dig_T1 and dig_P1 are non-zero before use
00678  *
00679  * @note dig_T1 and dig_P1 may still be zero if OTP is blank; caller validates

```



```

00682 * @see rx_bmp280_init() Sole caller; validates non-zero after this returns
00683 *
00684 * @since Version 1.0.0
00685 */
00686 RX_STATIC_TESTABLE void internal_parse_calibration(const uint8_t* buf)
00687 {
00688     /* Pre-condition: buf is a stack-allocated calibration buffer in rx_bmp280_init (never NULL). */
00689     RX_ASSERT_PRE(buf != NULL, "calibration buffer must not be NULL");
00690     #ifndef UNIT_TEST
00691     if (buf == NULL) {
00692         return;
00693     }
00694     #endif
00695     static_assert(
00696         (bool)((k_bmp280_calib_p9_lsb + k_bmp280_calib_tail_bytes == k_bmp280_calib_byte_count) != 0),
00697         "P9 coefficient must occupy the final two bytes of the calibration block");
00698     s_calib.dig_T1 = internal_parse_u16_le(&buf[k_bmp280_calib_t1_lsb]);
00699     s_calib.dig_T2 = internal_parse_s16_le(&buf[k_bmp280_calib_t2_lsb]);
00700     s_calib.dig_T3 = internal_parse_s16_le(&buf[k_bmp280_calib_t3_lsb]);
00701     s_calib.dig_P1 = internal_parse_u16_le(&buf[k_bmp280_calib_p1_lsb]);
00702     s_calib.dig_P2 = internal_parse_s16_le(&buf[k_bmp280_calib_p2_lsb]);
00703     s_calib.dig_P3 = internal_parse_s16_le(&buf[k_bmp280_calib_p3_lsb]);
00704     s_calib.dig_P4 = internal_parse_s16_le(&buf[k_bmp280_calib_p4_lsb]);
00705     s_calib.dig_P5 = internal_parse_s16_le(&buf[k_bmp280_calib_p5_lsb]);
00706     s_calib.dig_P6 = internal_parse_s16_le(&buf[k_bmp280_calib_p6_lsb]);
00707     s_calib.dig_P7 = internal_parse_s16_le(&buf[k_bmp280_calib_p7_lsb]);
00708     s_calib.dig_P8 = internal_parse_s16_le(&buf[k_bmp280_calib_p8_lsb]);
00709     s_calib.dig_P9 = internal_parse_s16_le(&buf[k_bmp280_calib_p9_lsb]);
00710     /* Post-condition: caller (rx_bmp280_init) validates dig_T1 and dig_P1 are non-zero */
00711 }
00712
00713 /*
=====
00714 * Public API Implementation
00715 *
=====
00716 */
00717
00718 /**
00719 * @brief Verify BMP280 chip ID register reads the expected value
00720 *
00721 * @details
00722 * Reads the chip ID register (0xD0) and compares it to k_bmp280_chip_id_expected.
00723 * Clears s_manager on failure so the driver remains in a safe state.
00724 *
00725 *
00726 * @pre s_manager non-NULL
00727 * @pre "i2c1_baro" bus initialized
00728 * @post s_manager set to NULL on failure
00729 * @post No state change on success
00730 *
00731 * @note Not thread-safe
00732 * @see rx_bmp280_init() Sole caller
00733 *
00734 * @since Version 1.0.0
00735 */
00736 static rx_err_t internal_verify_chip_id(void)
00737 {
00738     uint8_t chip_id = 0U;
00739     const rx_err_t err = internal_read_regs(k_bmp280_reg_chip_id, &chip_id, k_bmp280_single_byte);
00740     if (err != k_rx_ok) {
00741         rx_log_error(s_tag, "Chip ID read failed");
00742         s_manager = NULL;
00743         return err;
00744     }
00745     if (chip_id != k_bmp280_chip_id_expected) {
00746         rx_log_error_val(s_tag, "Unexpected chip ID", (uint32_t)chip_id);
00747         s_manager = NULL;
00748         return k_rx_err_invalid_state;
00749     }
00750     return k_rx_ok;
00751 }
00752
00753 /**
00754 * @brief Read, parse, and validate BMP280 factory calibration coefficients
00755 *
00756 * @details
00757 * Reads 24 bytes of factory-calibrated trimming parameters from OTP registers
00758 * 0x88-0x9F, parses them into s_calib, and validates that critical coefficients
00759 * (dig_T1 and dig_P1) are non-zero. Clears s_manager on failure.
00760 *
00761 *
00762 * @pre s_manager non-NULL
00763 * @pre "i2c1_baro" bus initialized
00764 * @post s_calib populated with valid coefficients on k_rx_ok
00765 * @post s_manager set to NULL on failure
00766 *

```

```

00767 * @note Not thread-safe
00768 * @see rx_bmp280_init() Sole caller
00769 *
00770 * @since Version 1.0.0
00771 */
00772 static rx_err_t internal_read_and_validate_calibration(void)
00773 {
00774     uint8_t      calib_buf[k_bmp280_calib_byte_count];
00775     const rx_err_t err =
00776         internal_read_regs(k_bmp280_reg_calib_start, calib_buf, k_bmp280_calib_byte_count);
00777     if (err != k_rx_ok) {
00778         rx_log_error(s_tag, "Calibration read failed");
00779         s_manager = NULL;
00780         return err;
00781     }
00782     internal_parse_calibration(calib_buf);
00783
00784     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(dig_T1 short-circuit: tested via dig_P1==0 path) */
00785     if ((s_calib.dig_T1 == 0U) || (s_calib.dig_P1 == 0U)) {
00786         /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00787         rx_log_error(s_tag, "Invalid calibration: dig_T1 or dig_P1 is zero");
00788         s_manager = NULL;
00789         return k_rx_err_invalid_state;
00790     }
00791     return k_rx_ok;
00792 }
00793
00794 /**
00795  * @brief Initialize BMP280 sensor and read factory calibration coefficients
00796  *
00797  * @details
00798  * Verifies chip ID, reads and validates 24 bytes of factory-calibrated trimming
00799  * parameters from OTP registers, and configures the IIR filter by writing to
00800  * the config register 0xF5.
00801  *
00802  * @pre manager non-NULL, "i2c1_baro" registered and initialized
00803  * @pre BMP280 powered on RIIC1 bus
00804  * @post s_calib contains valid OTP coefficients
00805  * @post s_initialized == true on success
00806  *
00807  * @note Not thread-safe
00808  * @see rx_bmp280_read() For triggered measurements
00809  *
00810  * @since Version 1.0.0
00811  */
00812 rx_err_t rx_bmp280_init(rx_bus_manager_t* manager)
00813 {
00814     RX_CHECK_NULL_PTR(manager, s_tag, "Bus manager is NULL");
00815     RX_ASSERT_PRE(s_bus_name != NULL, "Bus name must not be NULL");
00816     s_manager = manager;
00817     s_initialized = false;
00818     rx_err_t err = internal_verify_chip_id();
00819     if (err != k_rx_ok) {
00820         return err;
00821     }
00822     err = internal_read_and_validate_calibration();
00823     if (err != k_rx_ok) {
00824         return err;
00825     }
00826     err = internal_write_reg(k_bmp280_reg_config, k_bmp280_config_val);
00827     if (err != k_rx_ok) {
00828         rx_log_error(s_tag, "Config register write failed");
00829         s_manager = NULL;
00830         return err;
00831     }
00832     s_initialized = true;
00833     rx_log_info(s_tag, "BMP280 initialized, calibration loaded");
00834     return k_rx_ok;
00835 }
00836
00837 /**
00838  * @brief Read ADC registers, assemble values, apply Bosch compensation, and validate range
00839  *
00840  * @details
00841  * Performs the ADC read + compensation + range-check portion of the measurement cycle,
00842  * after the forced-mode trigger and status poll have completed. Specifically:
00843  * 1. Read 6 bytes of ADC data from k_bmp280_reg_press_msb (0xF7)
00844  * 2. Assemble 20-bit pressure and temperature ADC values via internal_assemble_adc20()
00845  */

```



```

00854 * 3. Apply Bosch integer temperature compensation (internal_compensate_temp())
00855 * 4. Apply Bosch integer pressure compensation (internal_compensate_pressure())
00856 * 5. Validate temperature in [-40.00, +85.00] degC (centi-degC units)
00857 * 6. Validate pressure in [300, 1100] hPa (Pa*256 units)
00858 *
00859 *
00860 *
00861 * @pre s_manager non-NULL (rx_bmp280_init succeeded)
00862 * @pre s_initialized == true
00863 * @pre out non-NULL
00864 * @pre Forced measurement complete (status poll cleared by caller)
00865 * @post out->temp_centi_degC in [-4000, 8500] on k_rx_ok
00866 * @post out->press_pa_256 in [7680000, 28160000] on k_rx_ok
00867 *
00868 * @note Not thread-safe
00869 * @see rx_bmp280_read() Caller that triggers measurement and polls status before calling this
00870 * @see internal_compensate_temp() Temperature compensation algorithm
00871 * @see internal_compensate_pressure() Pressure compensation algorithm
00872 *
00873 * @since Version 1.0.0
00874 */
00875 RX_STATIC_TESTABLE rx_err_t internal_read_and_compensate_adc(bmp280_data_t* out)
00876 {
00877     /* Pre-conditions: out is a local stack variable in rx_bmp280_read (never NULL);
00878      * s_initialized is true because rx_bmp280_read checks it via RX_CHECK_INITIALIZED. */
00879     RX_ASSERT_PRE(out != NULL, "out must be non-NULL");
00880     #ifdef UNIT_TEST
00881     if (out == NULL) {
00882         return k_rx_err_null_ptr;
00883     }
00884     #endif
00885     RX_ASSERT_PRE(s_initialized, "BMP280 must be initialized before ADC read");
00886
00887     /* Step 1: Read 6 bytes of ADC data: pressure[3] + temperature[3] */
00888     uint8_t adc_buf[k_bmp280_adc_buf_size];
00889     const rx_err_t err = internal_read_regs(k_bmp280_reg_press_msb, adc_buf, k_bmp280_adc_buf_size);
00890     if (err != k_rx_ok) {
00891         rx_log_error(s_tag, "ADC data read failed");
00892         return err;
00893     }
00894
00895     /* Step 2: Assemble 20-bit raw ADC values */
00896     const int32_t adc_pressure = internal_assemble_adc20(&adc_buf[k_bmp280_press_msb_idx]);
00897     const int32_t adc_temp = internal_assemble_adc20(&adc_buf[k_bmp280_temp_msb_idx]);
00898
00899     /* Step 3: Apply Bosch integer temperature compensation */
00900     int32_t t_fine = 0;
00901     out->temp_centi_degC = internal_compensate_temp(adc_temp, &t_fine);
00902
00903     /* Step 4: Apply Bosch integer pressure compensation */
00904     const rx_err_t press_err = internal_compensate_pressure(adc_pressure, t_fine, &out->press_pa_256);
00905     if (press_err != k_rx_ok) {
00906         rx_log_error(s_tag, "Pressure compensation failed (var1==0)");
00907         return press_err;
00908     }
00909
00910     /* Step 5: Postcondition range checks (BMP280 datasheet operating limits). */
00911     if ((out->temp_centi_degC < k_bmp280_temp_min_cdegC_impl) ||
00912         (out->temp_centi_degC > k_bmp280_temp_max_cdegC_impl)) {
00913         rx_log_error_val(s_tag, "Temperature out of range (centi-degC)", (int32_t)out->temp_centi_degC);
00914         return k_rx_err_invalid_state;
00915     }
00916
00917     if ((out->press_pa_256 < k_bmp280_press_min_pa_256) ||
00918         (out->press_pa_256 > k_bmp280_press_max_pa_256)) {
00919         rx_log_error_val(s_tag, "Pressure out of range (Pa*256)", out->press_pa_256);
00920         return k_rx_err_invalid_state;
00921     }
00922
00923     return k_rx_ok;
00924 }
00925
00926 /**
00927 * @brief Trigger forced measurement and read compensated pressure and temperature
00928 *
00929 * @details
00930 * Executes the complete forced-mode measurement cycle:
00931 * 1. Trigger forced measurement (write ctrl_meas)
00932 * 2. Poll status register for completion (bounded loop)
00933 * 3. Read ADC data, apply compensation, validate range (internal_read_and_compensate_adc)
00934 *
00935 *
00936 *
00937 * @pre rx_bmp280_init() succeeded
00938 * @pre out non-NULL
00939 * @post out populated with compensated measurement on k_rx_ok
00940 * @post Sensor in sleep mode (forced measurement complete)

```

```

00941 *
00942 * @note Not thread-safe
00943 * @see internal_read_and_compensate_adc() ADC read, compensation, and range validation
00944 *
00945 * @since Version 1.0.0
00946 */
00947 rx_err_t rx_bmp280_read(bmp280_data_t* out)
00948 {
00949     RX_CHECK_NULL_PTR(out, s_tag, "Output data pointer is NULL");
00950
00951     if (!s_initialized) {
00952         return k_rx_err_not_initialized;
00953     }
00954
00955     /* Step 1: Write ctrl_meas to trigger forced measurement */
00956     rx_err_t err = internal_write_reg(k_bmp280_reg_ctrl_meas, k_bmp280_ctrl_meas_val);
00957     if (err != k_rx_ok) {
00958         rx_log_error(s_tag, "ctrl_meas write failed");
00959         return err;
00960     }
00961
00962     /* Step 2: Poll status register until measuring bit (bit 3) clears */
00963     uint32_t poll_count = 0;
00964     while (poll_count < (uint32_t)k_bmp280_poll_max) {
00965         uint8_t status_byte = 0;
00966         err = internal_read_regs(k_bmp280_reg_status, &status_byte, k_bmp280_single_byte);
00967         if (err != k_rx_ok) {
00968             rx_log_error(s_tag, "Status register read failed");
00969             return err;
00970         }
00971
00972         if ((status_byte & k_bmp280_status_meas_mask) == 0U) {
00973             break; /* Measurement complete */
00974         }
00975
00976         (void)tx_thread_sleep(k_bmp280_poll_sleep_ticks); /* Yield CPU while sensor is measuring */
00977         poll_count++;
00978     }
00979
00980     if (poll_count >= (uint32_t)k_bmp280_poll_max) {
00981         rx_log_error(s_tag, "Status poll timeout");
00982         return k_rx_err_timeout;
00983     }
00984
00985     /* Step 3: Read ADC, apply compensation, validate output range */
00986     return internal_read_and_compensate_adc(out);
00987 }
00988
00989 #ifdef UNIT_TEST
00990 /**
00991  * @brief Reset BMP280 driver static state for unit test isolation
00992  *
00993  * @details
00994  *   Clears s_initialized to false and zeroes the calibration struct so
00995  *   each test starts from the same known state regardless of execution
00996  *   order. Available only in UNIT_TEST builds.
00997  *
00998  * @pre Called from test setUp() before each test
00999  * @pre No concurrent access from other threads (test runs single-threaded)
01000  * @post s_initialized == false; s_calib zeroed
01001  * @post s_manager == NULL; driver in pre-init state
01002  *
01003  * @note Test-only helper; not compiled into production firmware
01004  * @see setUp() in test_rx_bmp280.c Calls this function
01005  *
01006  * @since Version 1.0.0
01007  */
01008 void rx_bmp280_test_reset_state(void)
01009 {
01010     RX_ASSERT(true, "rx_bmp280_test_reset_state: test-only function");
01011     s_manager = NULL;
01012     s_initialized = false;
01013     const bmp280_calib_t zero_calib = {};
01014     s_calib = zero_calib;
01015 }
01016
01017 /**
01018  * @brief Set s_initialized and s_manager for direct internal function testing
01019  *
01020  * @details
01021  *   Allows tests to bypass the init sequence and set the module state
01022  *   directly so internal functions like internal_read_and_compensate_adc
01023  *   can be called without going through rx_bmp280_init().
01024  *
01025  * @pre Called from test setup
01026  * @pre No concurrent access from other threads (test runs single-threaded)

```

```

01028 * @post s_initialized == init_val; s_manager == manager
01029 * @post Calibration struct unchanged (only s_initialized and s_manager are modified)
01030 *
01031 * @note Test-only helper; not compiled into production firmware
01032 *
01033 * @since Version 1.0.0
01034 */
01035 void rx_bmp280_test_set_state(rx_bus_manager_t* manager, bool init_val)
01036 {
01037     s_manager = manager;
01038     s_initialized = init_val;
01039 }
01040
01041 /**
01042 * @brief Force s_calib.dig_P1 to zero for division-by-zero coverage testing
01043 *
01044 * @details
01045 * After init, s_calib.dig_P1 is guaranteed non-zero. This test helper
01046 * corrupts dig_P1 to zero so that internal_compensate_pressure() hits
01047 * the var1==0 guard (lines 607-608) when called via
01048 * internal_read_and_compensate_adc().
01049 *
01050 * @pre rx_bmp280_test_set_state() or rx_bmp280_init() called
01051 * @pre No concurrent access from other threads (test runs single-threaded)
01052 * @post s_calib.dig_P1 == 0; other calibration fields unchanged
01053 * @post internal_compensate_pressure() will return k_rx_err_invalid_state when called
01054 *
01055 * @note Test-only helper; not compiled into production firmware
01056 *
01057 * @since Version 1.0.0
01058 */
01059 void rx_bmp280_test_zero_calib_p1(void)
01060 {
01061     s_calib.dig_P1 = 0U;
01062 }
01063
01064 /**
01065 * @brief Override s_bus_name for branch coverage testing
01066 *
01067 * @details
01068 * Sets s_bus_name to the given pointer so that the RX_ASSERT_PRE checks
01069 * on s_bus_name can be tested with NULL. Caller must restore the original
01070 * value after the test.
01071 *
01072 *
01073 * @pre UNIT_TEST defined (s_bus_name is non-const in test builds)
01074 * @post s_bus_name == name
01075 *
01076 * @note Test-only helper; not compiled into production firmware
01077 * @since Version 1.0.0
01078 */
01079 void rx_bmp280_test_set_bus_name(const char* name)
01080 {
01081     s_bus_name = name;
01082 }
01083
01084 #endif /* UNIT_TEST */

```

